UNIVERSIDAD DE GUANAJUATO
División de Ingenierías · Campus Irapuato-Salamanca

Manual y documentación completa del proyecto: el carro con navegación asistida

Control distribuido en tres nodos, percepción multimodal, cinemática C99 portátil y protocolo de tiempo real

Materia: Fundamentos de los Sistemas de Información (FSI)
Profesor Titular: Dr. Horacio Rostro González
Fecha de entrega y exposición: 1 de diciembre de 2026
Integrantes del equipo:
• Jaime Fernando Guerrero Segoviano
• Melanie Contreras Solis
• Vitto Eduardo Rangel Fortuna
• Azul Abril Cadena Campos

Versión del documento: 1.0 · **Commit:** dd518c3 · **Rama:** main

Documentación técnica de ingeniería — Prohibida su alteración sin verificación de código

Índice General del Manual

Estructura temática y paginación del documento consolidado:

1. Resumen ejecutivo y objetivos — *Propósito pedagógico, justificación y alcance* **Pág. 3**

2. Qué hace el sistema — *Comportamiento funcional y capacidades del vehículo* **Pág. 4**

3. Arquitectura general del sistema — *Topología de tres nodos, enlaces de red y jerarquía* **Pág. 5**

4. Hardware, componentes y conexiones — *Placas, motores, I2C, cableado y lista de materiales* **Pág. 9**

5. Firmware del carro (ESP32 DevKit) — *Capas, cola protegida, watchdog y temporizaciones* **Pág. 17**

6. Firmware de la cámara (Freenove ESP32-WROVER) — *Sensor GC0308, compresión JPEG, PSRAM y modos* **Pág. 23**

7. Protocolo de comunicación — *Comandos WebSocket, telemetría SENS y respuestas* **Pág. 28**

8. Núcleo de navegación en C (nucleo_c) — *Máquina de estados, cinemática, pose y calibración* **Pág. 32**

9. Aplicación móvil Android — *Interfaz de usuario, sesiones persistentes y puente JNI* **Pág. 38**

10. Visión por computadora y percepción — *Corredor trapezoidal, segmentación e inferencia TFLite* **Pág. 45**

11. Compilación, instalación y verificación — *Cadenas de compilación, flasheo y comprobación SHA-256* **Pág. 49**

12. Pruebas, calidad y simulador — *2045 casos en PC, pruebas de estrés y modelo virtual* **Pág. 53**

13. Historia del proyecto y decisiones de diseño — *Evolución por jornadas (J1-J15) y lo descartado* **Pág. 61**

14. Resultados y estado de validación — *Desglose estricto: medido en hardware, simulado y NO MEDIDO* **Pág. 64**

15. Límites, riesgos y trabajo futuro — *Restricciones operativas y líneas de desarrollo* **Pág. 66**

16. Guía de operación y seguridad física — *Regla de 90 min / 20 corridas, protocolo de encendido* **Pág. 67**

17. Preguntas frecuentes de la exposición oral — *Banco de 9 preguntas críticas del sínodo con respuestas* **Pág. 71**

18. Solución de problemas y mantenimiento — *Matriz de síntomas, fallas comunes y conservación* **Pág. 74**

19. Glosario técnico de términos y unidades — *Definiciones formales y vocabulario de ingeniería* **Pág. 77**

20. Apéndices técnicos — *Comandos, pines, sumas criptográficas y pendientes* **Pág. 80**

1. Resumen ejecutivo y objetivos

El proyecto **Carro FSI** aborda el diseño, construcción y programación de un vehículo robótico móvil capaz de ejecutar misiones de navegación terrestre asistida empleando exclusivamente componentes de bajo costo y microcontroladores embebidos accesibles en el ámbito universitario.

El sistema demuestra que no se requieren costosos sensores LiDAR industriales ni procesadores de gama alta para lograr comportamientos autónomos seguros: mediante una cuidadosa **arquitectura distribuida en tres nodos**, el desacoplamiento estricto de la lógica cinemática en C99 puro, el control de orientación por giroscopio digital y la supervisión de proximidad por láser ToF, el carro planifica trayectorias sobre una cuadrícula de 20 cm, sortea obstáculos en su trayectoria y mantiene informado al operador humano en tiempo real.

Objetivos principales del proyecto

• **Objetivo 1 (Arquitectónico):** Demostrar la viabilidad de separar la computación de visión (cámara) del control de tracción en tiempo real (motores y giroscopio) en placas microcontroladoras ESP32 independientes, evitando latencias de procesamiento.

• **Objetivo 2 (Portabilidad y rigor):** Desarrollar el núcleo de navegación en C estándar (C99) libre de dependencias externas, permitiendo compilarlo tanto en la app Android (vía JNI) como en PC para validación automatizada mediante suites de pruebas unitarias.

• **Objetivo 3 (Seguridad robusta):** Diseñar mecanismos de protección multinivel ante fallos, incluyendo un watchdog de comunicación de 1000 ms, un watchdog de hardware de 5000 ms y la autoridad prioritaria del sensor láser ToF para detención instantánea.

• **Objetivo 4 (Honestidad metrológica):** Caracterizar experimentalmente los límites físicos del chasis, reportando con total transparencia los datos verificados en pista física, los resultados obtenidos en simulación y los aspectos no medidos.

2. Qué hace el sistema (Descripción general)

El vehículo ofrece dos modos principales de interacción operativa:

1. **Teleoperación manual supervisada:** El operador conduce el vehículo desde la pantalla táctil de su teléfono móvil mediante una cruceta direccional sensible. El teléfono transmite órdenes repetidas de avance cada 250 ms por un canal WebSocket. Mientras el usuario mantenga el botón pulsado, el vehículo avanza aplicando el factor de balance calibrado. Al soltar el control, el carro frena de inmediato. La pantalla exhibe el video en vivo emitido por la cámara a ~12.4 cuadros por segundo.
2. **Navegación asistida semiautónoma:** El operador traza una ruta sobre una cuadrícula métrica de 20 cm en la app Android. Al ordenar la ejecución, el núcleo en C toma el mando:

• Convierte los segmentos en pulsos temporizados calculados con la constante de milisegundos por centímetro.

• En cada cambio de dirección, orienta el chasis utilizando el ángulo medido por el giroscopio MPU6050.

• Analiza continuamente el corredor de seguridad frontal: si la cámara o el láser detectan una caja de 20×20 cm obstruyendo la marcha, el carro se detiene, evalúa el espacio lateral, se aparta 40 cm hacia el lado despejado, supera la caja y busca un punto de reincorporación 30 cm por delante antes de continuar la ruta trazada.

3. Arquitectura general del sistema

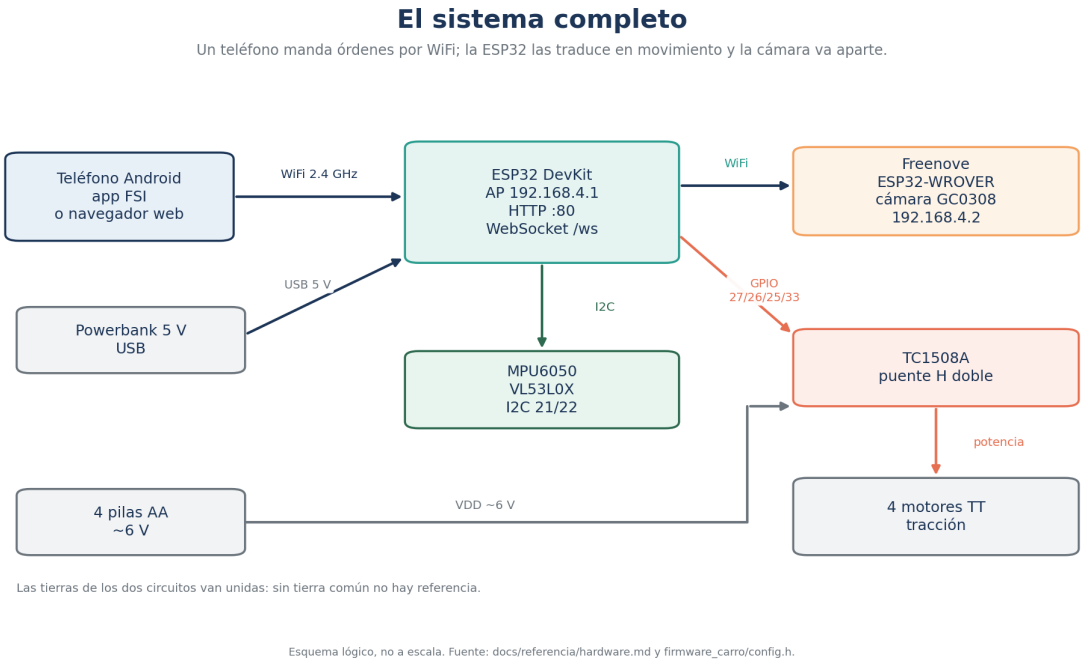


Figura 1: El sistema completo. Teléfono, ESP32, cámara, sensores, driver y dos circuitos de alimentación. (Esquema lógico, no a escala.)

Arquitectura general del sistema

Materia: Fundamentos de los Sistemas de Información (FSI)

Proyecto: Carro con navegación asistida por visión y sensores

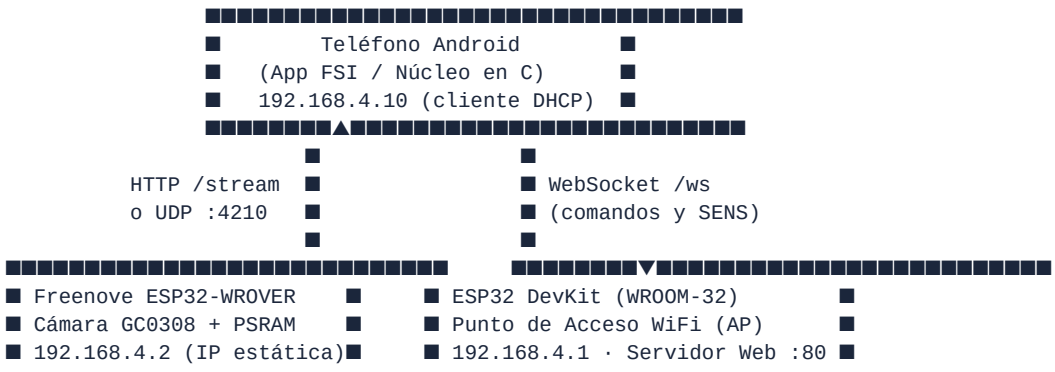
Fecha de consolidación: 27 de octubre de 2026

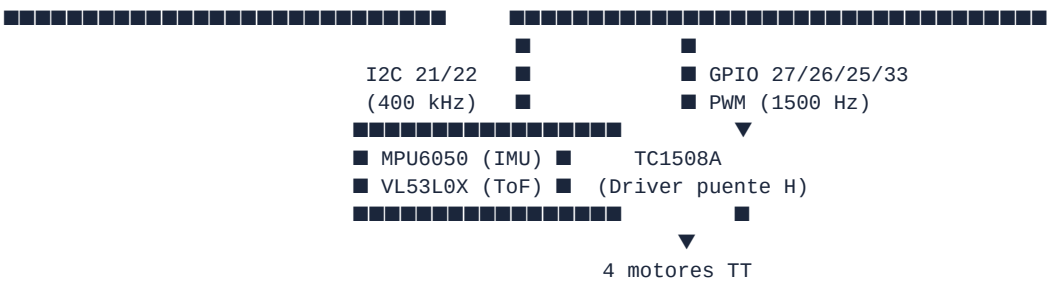
---

1. Visión general del vehículo

El sistema consiste en un vehículo terrestre de cuatro ruedas motrices capaz de navegar rutas planificadas en una cuadrícula bidimensional, sortear obstáculos mediante visión computacional y sensores de proximidad, y reportar telemetría en tiempo real a una aplicación móvil Android.

El procesamiento no se concentra en una única computadora monolítica, sino que se distribuye estratégicamente entre tres nodos autónomos comunicados mediante una red inalámbrica local:





2. Los tres nodos computacionales

2.1 Nodo de Control y Tracción: ESP32 DevKit

Es el controlador de tiempo real del vehículo, basado en un microcontrolador de 32 bits de doble núcleo (ESP32-D0WD-V3) con 4 MB de memoria flash.

• **Generador de red:** Crea el punto de acceso WiFi local bajo el estándar 802.11b/g/n en canal fijo, asignándose la dirección `192.168.4.1`.

• **Controlador de potencia:** Transforma las órdenes de alto nivel en señales PWM dirigidas a las compuertas lógicas del driver de motores TC1508A.

• **Adquisición sensorial I2C:** Muestrea a 100 Hz el giroscopio MPU6050 (`0x68`) para mantener el cómputo de rumbo e interroga periódicamente el sensor láser VL53L0X (`0x29`).

• **Seguridad autónoma:** Contiene la lógica del watchdog de comunicación (1000 ms) y la parada de emergencia por proximidad física del láser.

2.2 Nodo de Visión: Freenove ESP32-WROVER v3.0

Es una placa independiente montada en la parte frontal superior del chasis, equipada con un sensor de imagen GalaxyCore GC0308 y 4 MB de memoria PSRAM.

• **Operación como cliente:** Se conecta como estación (STA) a la red del carro con la IP estática `192.168.4.2`.

• **Transmisión de cuadros:** Captura imágenes en formato RGB565 a resolución HQVGA (240×176 píxeles), comprime cuadros en JPEG por software gracias a la memoria PSRAM y los sirve por HTTP multipart (`/stream`) o por paquetes compactos UDP (`:4210`).

• **Aislamiento de tareas:** Al ejecutarse en un procesador separado, las exigentes rutinas de lectura de sensor y compresión JPEG no interfieren con la precisión de los tiempos de PWM ni con el muestreo del giroscopio del carro.

2.3 Nodo Supervisor: Teléfono Android

Dispositivo móvil montado en el chasis o sostenido por el operador que ejecuta la aplicación del proyecto.

• **Planificación de rutas:** Proporciona un lienzo interactivo con cuadrícula de 20 cm donde se trazan los puntos de destino.

• **Núcleo de navegación (JNI):** Ejecuta nativamente los algoritmos escritos en C99 (`navegador.c`), calculando rumbos, corrigiendo desviaciones laterales y gestionando maniobras de esquive de obstáculos.

• **Inferencia visual:** Analiza el flujo de video en tiempo real mediante segmentación cromática del piso y modelos convolucionales TensorFlow Lite para confirmar la presencia de obstáculos en el corredor de paso.

3. Topología de red y protocolos

Toda la comunicación ocurre dentro de una subred cerrada clase C (`192.168.4.0/24`):

Dispositivo	Rol en red	Dirección IP	Puerto / Ruta	Protocolo	Contenido transmitido

ESP32 DevKit	Servidor AP	`192.168.4.1`	`:80 /ws`	WebSocket	Comandos (`FORWARD`, `STOP`, `GIRAR`), telemetría periódica (`SENS`) y pings.
ESP32 DevKit	Servidor AP	`192.168.4.1`	`:80 /`	HTTP	Página web básica de teleoperación almacenada en flash (`pagina.h`).
Cámara Freenove	Cliente STA	`192.168.4.2`	`:80 /stream`	HTTP	Transmisión de video continuo MJPEG (240×176 px a ~12 fps).
Cámara Freenove	Cliente STA	`192.168.4.2`	`:4210`	UDP	Paquetes de celdas de piso (`CEL1`) de 1244 bytes.
Teléfono móvil	Cliente STA	Asignación DHCP (`.10` a `.20`)	Efímero	TCP/UDP	Conexión bidireccional a los dos nodos del carro.

4. Esquema de alimentación separada

Para evitar que el ruido inductivo y las caídas bruscas de tensión generadas por los motores reinicien los microcontroladores (efecto *brownout*), la alimentación está físicamente desacoplada en dos circuitos:

1. **Circuito de control (Electrónica):**

• Una batería portátil USB (powerbank) de 5 V entrega corriente limpia a la placa ESP32 DevKit y a la cámara Freenove a través de sus puertos micro-USB.

• El regulador de tensión interno de la placa ESP32 baja los 5 V a 3.3 V regulados para alimentar los sensores I2C (MPU6050 y VL53L0X).

2. **Circuito de potencia (Motores):**

• Un portapilas con 4 pilas AA entrega aproximadamente 6 V nominales directamente al pin `VDD` del driver TC1508A.

• La corriente de arranque y bloqueo de los motores fluye exclusivamente a través de las pilas AA.

3. **Masa común (GND):**

• Las tierras (GND) del circuito de la powerbank y del portapilas de motores están soldadas o unidas rígidamente en un bus común. Esto garantiza una referencia de voltaje compartida de 0 V para que las señales lógicas PWM enviadas desde el ESP32 al driver sean interpretadas correctamente.

5. Jerarquía de control y autoridad de parada

La toma de decisiones de movimiento sigue un esquema de seguridad por capas:

1. **Capa 1: App y Navegador Nativo:**

Determina hacia dónde avanzar y planifica desvíos de 40 cm si la cámara detecta un obstáculo sobre la ruta trazada. 2. **Capa 2: Firmware del Carro (`carro.c`):** Aplica rampas de aceleración a plena potencia (250 ms), balance de potencia por lado y apaga los motores si el canal WebSocket pierde señal por más de 1000 ms (Watchdog). 3. **Capa 3: Sensor ToF Láser (Máxima prioridad física):** El sensor VL53L0X realiza lecturas directas cada ~33 ms. Si detecta un objeto al frente a menos de la distancia crítica de seguridad, el firmware fuerza una parada inmediata (`TOF_PARO`), anulando cualquier comando de avance activo emitido por la app.

6. Referencias visuales

• **`docs/figuras/fig_01_sistema.png`**: Diagrama esquemático integral del sistema.

• **`docs/figuras/fig_02_capas_firmware.png`**: Separación de capas lógicas y de hardware.

• **`docs/figuras/fig_10_alimentacion.png`**: Esquema de conexiones de tierra y circuitos de alimentación.

4. Hardware, componentes y conexiones

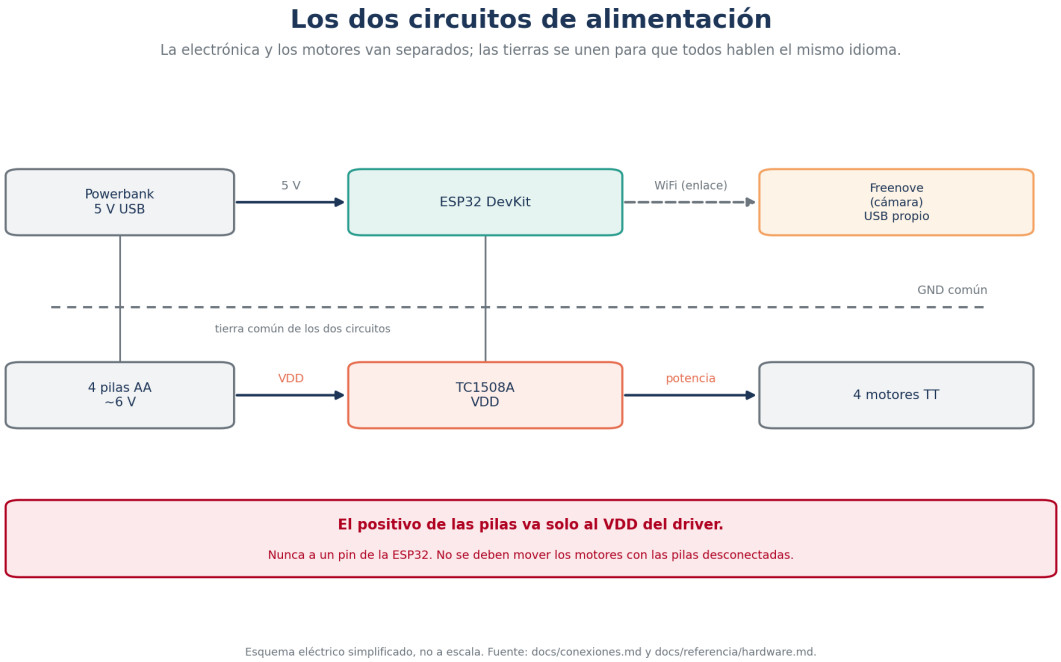
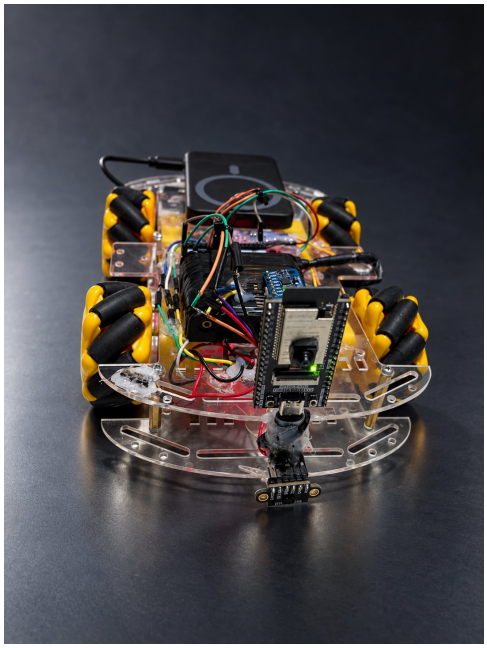
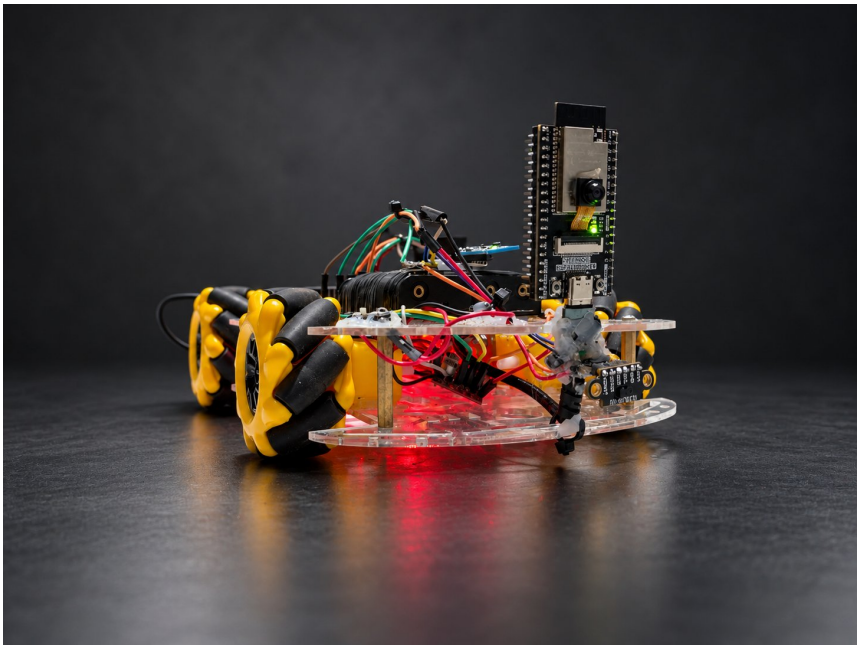


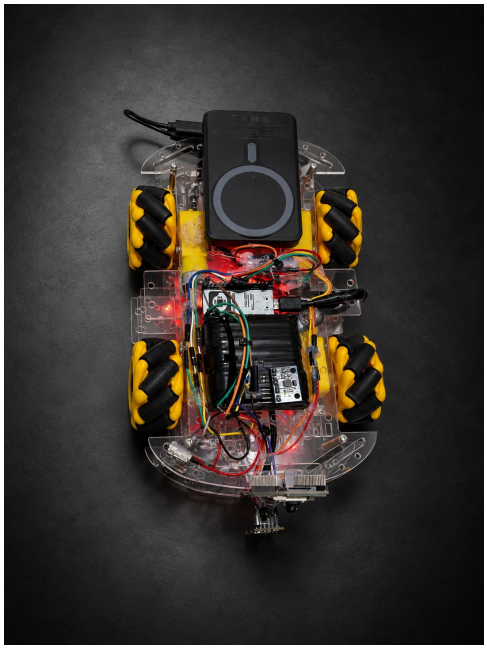
Figura 10: Los dos circuitos de alimentación. La electrónica va por USB y los motores por cuatro pilas AA; las tierras van unidas y el positivo de las pilas no toca la ESP32. (Esquema eléctrico simplificado, no a escala.)



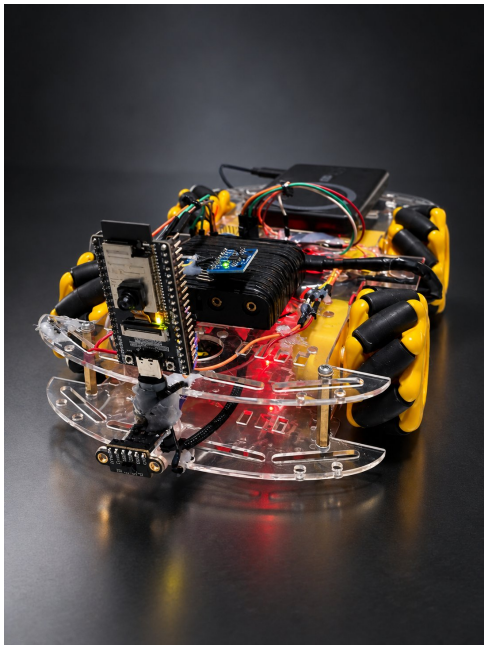
Fotografía: *Vista frontal completa del carro. Archivo: foto_01_carro_frontal.jpg*



Fotografía: *Vista lateral del vehículo. Archivo: foto_02_carro_lateral.jpg*



Fotografía: *Vista superior cenital. Archivo: foto_03_carro_superior.jpg*



Fotografía: *Electrónica abierta con rótulos. Archivo: foto_04_electronica_abierta.jpg*

Referencia de hardware

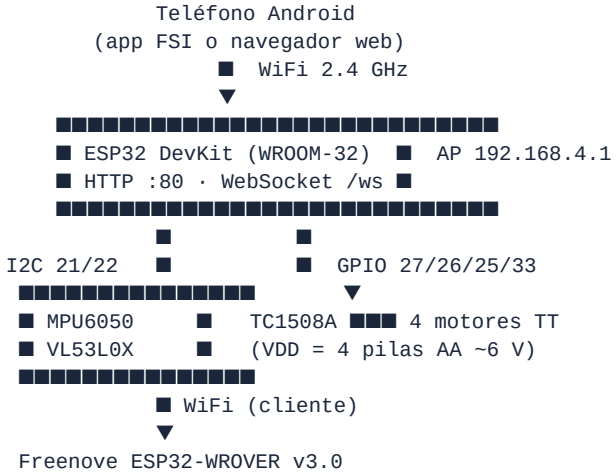
Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Fuentes principales: `firmware_carro/config.h`, `docs/conexiones.md`, `docs/sensores.md`, `docs/camara_prueba.md`, `docs/hardware_cargar.md` y los reportes J13/J14/J15.

Este documento describe lo que el repositorio permite afirmar sobre el hardware del carro. No sustituye a las hojas de datos ni a una medición con multímetro: cuando un dato no está comprobado, se dice. Las etiquetas **VERIFICADO**, **SIMULADO**, **HIPÓTESIS**, **NO MEDIDO** y **PENDIENTE** se explican en `docs/plan_documentacion.md`, sección 1.3.

1. Qué es el carro

El carro es un vehículo de cuatro ruedas, todas motrices, sobre un chasis tipo TT. Lo controla una placa ESP32 DevKit por WiFi. Lleva dos sensores conectados por un bus llamado I2C (un giroscopio MPU6050 y un sensor de distancia VL53L0X) y una segunda placa con cámara (Freenove ESP32-WROVER v3.0) que transmite video por su propia página web. La alimentación está separada: una batería portátil USB (powerbank) alimenta la electrónica, y un portapilas de cuatro pilas AA alimenta solo los motores a través de un driver TC1508A.



```
                    cámara GC0308 · 192.168.4.2 · HTTP :80
```

El diagrama es un esquema lógico, no un plano de montaje. El cableado físico paso a paso está en **`docs/conexiones.md`**, secciones 2 y 6.

1.1 Palabras básicas

Estas palabras aparecen muchas veces en el resto de la documentación. Aquí va su significado en una línea, sin dar por sabido nada:

Palabra	Qué es, en sencillo
ESP32	Un pequeño computador con WiFi y Bluetooth. Es el «cerebro» del carro.
DevKit	La placa que lleva la ESP32 y los conectores para soldar o enchufar cables.
Driver / puente H	Un interruptor electrónico que deja pasar la corriente a los motores en uno u otro sentido. Sin él, la ESP32 no puede mover un motor.
PWM	Una señal que se enciende y apaga muy rápido. Cambiando el porcentaje de tiempo encendida se cambia la velocidad del motor.
I2C	Un bus de dos cables (SDA y SCL) por el que varios sensores comparten datos con la ESP32.
AP (punto de acceso)	Modo en el que la ESP32 crea su propia red WiFi, como un router pequeño.
DHCP	El servicio que reparte direcciones IP a los dispositivos que se conectan a esa red.
IP	El número que identifica a cada dispositivo dentro de la red (por ejemplo, 192.168.4.1).
PSRAM	Memoria adicional de la placa de la cámara, necesaria para guardar imágenes grandes.
Powerbank	Batería portátil con salida USB, la que da energía a la electrónica.

2. Lista de materiales

La columna «Evidencia» dice de dónde sale cada elemento. No se encontró en el repositorio una lista de compra con marcas, modelos exactos ni precios: esa lista es **PENDIENTE**.

Elemento	Cantidad	Para qué	Evidencia
ESP32 DevKit (ESP32-D0WD-V3, 38 pines, 4 MB de flash, sin PSRAM)	1	Control, WiFi, servidor HTTP/WebSocket	**`docs/hardware_cargar.md`** §«Qué se probó»; **`config.h`**
Módulo driver TC1508A	1	Puente H doble para los cuatro motores	**`docs/conexiones.md`** §1 y §5; **`config.h:66-76`**
Motor TT con reductora y rueda	4	Tracción (2 por canal, en paralelo)	**`docs/conexiones.md`** §5; **`config.h:68-70`**
Chasis con portapilas de 4 AA	1	Estructura y alimentación de motores	**`docs/conexiones.md`** §2 y §4
Powerbank USB 5 V	1	Alimentación de la ESP32 (y de la cámara)	**`docs/conexiones.md`** §4; **`config.h:145-148`**
MPU6050 en módulo GY-521	1	Giroscopio de tres ejes; se usa el eje Z	**`config.h:156-174`**; **`docs/sensores.md`**

VL53L0X en módulo VL53LXX-V2	1	Distancia al frente (láser)	`config.h:156-165`, `223-240`; `docs/sensores.md`
Freenove ESP32-WROVER v3.0 con cámara GC0308 y PSRAM de 4 MB	1	Video por WiFi	`docs/camara_prueba.md` §«Primera parte (H2)»
Teléfono Android	1	App de control, visión y rutas	`app_android/`; `docs/entrega.md`
Cables dupont, soldadura o regleta para I2C	—	Unir SDA/SCL de los dos módulos	`docs/conexiones.md` §8
Cable USB de datos	2	Cargar y alimentar las placas	`docs/hardware_cargar.md` §«Si no aparece ningún puerto COM»

Los cuatro motores van **dos por canal y en paralelo**, con la misma polaridad: los izquierdos a OUTA/OUTB y los derechos a OUTC/OUTD (`docs/conexiones.md` §5). El TC1508A es un **puente H doble**: tiene dos canales y cada uno puede girar su par de motores hacia adelante o hacia atrás. Admite 1.8 A continuos y 2.5 A de pico por canal según su hoja de datos; el consumo real de estos motores no está medido (**NO MEDIDO**).

3. Pines usados por la ESP32

Todos los números salen de `firmware_carro/config.h`. Los pines 25, 26, 27 y 33 no son pines de arranque; el GPIO 2 sí lo es y por eso es el delicado del proyecto.

GPIO	Función	Dirección	Nota
27	Motor izquierdo, entrada A	Salida PWM	Va a INA del TC1508A (`config.h:73`)
26	Motor izquierdo, entrada B	Salida PWM	Va a INB; el motor izquierdo **avanza** con este pin (`config.h:74`)
25	Motor derecho, entrada A	Salida PWM	Va a INC; el motor derecho **avanza** con este pin (`config.h:75`)
33	Motor derecho, entrada B	Salida PWM	Va a IND (`config.h:76`)
2	Pulso para la powerbank	Salida	50 ms en alto cada 2 s (`config.h:141-148`)
21	I2C SDA	Bidireccional	MPU6050 y VL53L0X (`config.h:160`)
22	I2C SCL	Salida	MPU6050 y VL53L0X (`config.h:161`)

Reglas de conexión:

• El positivo de las pilas va **solo** al VDD del TC1508A; nunca a un pin de la ESP32 (`docs/conexiones.md` §4).

• El negativo de las pilas, el GND del driver y el GND de la ESP32 van unidos. Sin tierra común no hay referencia (`docs/conexiones.md` §4).

• No se mueven los motores con las pilas desconectadas: una señal de 3.3 V sobre un driver sin alimentar puede superar su tensión máxima de entrada (`docs/conexiones.md` §4).

• Nunca se pone PWM en las dos entradas del mismo motor a la vez (`config.h:71`).

La señal PWM se repite 1500 veces por segundo y se puede graduar en 256 pasos (8 bits), de 0 a 255 (`config.h:81-82`). Un valor de 255 significa «encendido todo el tiempo»; 128, «la mitad». La velocidad base es 255 y los perfiles aplican topes y balance: MANUAL 255/252 y AUTO 210/207 (`config.h:109-128`). Los pivotes usan un solo motor a 255 (`config.h:88`).

4. Alimentación

Hay dos circuitos de alimentación separados a propósito, para que el ruido y el consumo de los motores no reinicien la electrónica:

1. **Electrónica:** powerbank de 5 V por USB a la ESP32 DevKit; la cámara Freenove se alimenta por su propio USB (la documentación no dice si comparte la misma powerbank; **PENDIENTE** de confirmar en el montaje).
2. **Motores:** cuatro pilas AA en serie (alrededor de 6 V nominales) al VDD/VCC del TC1508A. El driver no tiene alimentación lógica separada: VDD alimenta lógica y motores (`docs/conexiones.md` §4).

Consecuencias y límites:

• El TC1508A admite de 2.4 a 7.2 V; 7.2 V es el máximo absoluto. Cuatro alcalinas nuevas rondan 6 V, así que hay poco margen; no se debe añadir una quinta pila (`docs/conexiones.md` §4).

• La ESP32 se configura a 160 MHz y el WiFi a 13 dBm para reducir picos de corriente (`config.h:54-64`). El AP usa el canal 1 fijo (`config.h:61`).

• El GPIO 2 produce un pulso de 50 ms cada 2000 ms para que la powerbank no corte la salida (`config.h:141-148` y `carro.c`). Qué hay conectado realmente a ese pin —resistencia, LED o nada— es **NO MEDIDO** y **PENDIENTE** (`docs/conexiones.md` §3).

• La corriente de arranque y de motor bloqueado de los TT no está medida (**NO MEDIDO**). Con dos motores por canal, el cálculo del documento de conexiones pide medir antes de exigir el driver al máximo (`docs/conexiones.md` §5).

• No hay medición registrada del consumo total ni de la autonomía. Cualquier cifra de duración sería una **HIPÓTESIS**.

El orden seguro de conexión, con las ruedas en el aire y sin pilas al principio, está en `docs/conexiones.md` §6. Resumido: tierras primero, señales después, un solo motor, revisar cortos con multímetro, encender la ESP32 sin pilas, probar un motor, y solo al final los cuatro y el carro en el suelo.

5. Sensores I2C

Los dos módulos comparten el mismo bus I2C de la DevKit y se alimentan a 3V3. El giroscopio mide cuánto y hacia dónde gira el carro; el láser mide la distancia al objeto que tiene delante. Cada módulo trae sus propias resistencias de pull-up, que mantienen los dos cables en un nivel conocido cuando nadie está hablando (`config.h:156-159`).

Señal	GPIO	GY-521 (MPU6050)	VL53LXX-V2 (VL53L0X)
SDA	21	SDA	SDA
SCL	22	SCL	SCL
Alimentación	3V3	VCC	VIN
Tierra	GND	GND	GND

• Frecuencia del bus: 400 kHz; tiempo máximo por transacción: 5 ms (`config.h:162-163`).

• Direcciones: MPU6050 en 0x68 y VL53L0X en 0x29 (`config.h:164-165`). El firmware acepta además los identificadores 0x70 y 0x72 de algunos clones del MPU6050 (`config.h:167-170`).

• Montaje: el giroscopio va plano sobre la caja de pilas, con la flecha X hacia el frente y el eje Z hacia arriba; el láser va al frente, a 6-8 cm del piso, inclinado 5-8° hacia arriba (`docs/conexiones.md` §8).

• El sentido del ángulo se corrige con `IMU_SIGNO` (`config.h:172-174`). El valor actual es 1. Si en una prueba física el sentido sale invertido, se cambia el signo en la configuración; no se cambia el cableado.

• Los empalmes deben ser soldados o con regleta: un SCL flojo ya dejó al giroscopio invisible aunque el LED del módulo encendiera (`docs/conexiones.md` §8).

Para el detalle de mensajes, calibración y diagnóstico, ver `docs/sensores.md` y `docs/referencia/firmware.md`. Los comandos `I2C?`, `PINES?` y `PRUEBAI2C?` permiten revisar el bus por USB (`docs/conexiones.md` §8).

6. Cámara

La cámara es una placa **Freenove ESP32-WROVER v3.0** con sensor **GC0308** de 0.3 MP, PSRAM de 4 MB y chip USB-serie CH340 (`docs/camara_prueba.md` §«Primera parte (H2)»). No es una ESP32-CAM genérica con OV2640.

• El GC0308 **no genera JPEG**: entrega RGB565 o YUV422 y la ESP32 comprime cada cuadro por software. Con el formato JPEG la inicialización falla con `0x106 ESP_ERR_NOT_SUPPORTED` (`camara_prueba.ino:2-15`; `docs/camara_prueba.md` §H2).

• Mapa de pines: `CAMERA_MODEL_WROVER_KIT`; XCLK 21, SIOD 26, SIOC 27, datos Y2..Y9 = 4, 5, 18, 19, 36, 39, 34, 35, VSYNC 25, HREF 23, PCLK 22, sin PWDN ni RESET (`docs/camara_prueba.md` §H2).

• Se conecta como cliente a la red del carro con IP fija **192.168.4.2** y puerto HTTP 80 (`camara_prueba.ino:89-91`). El DHCP del DevKit reparte de .10 a .20 para no chocar con ella (`config.h:42-52`).

• Configuración por defecto: 20 MHz de reloj, RGB565, HQVGA 240×176, JPEG de calidad 50 (`camara_prueba.ino:105-108`). El documento histórico H3 recomendaba QVGA; el código actual usa HQVGA. La diferencia se explica en `docs/referencia/camara.md`.

• La placa se alimenta por USB y se comunica por WiFi; no comparte pines con la DevKit. El montaje físico de la cámara sobre el carro no está documentado (**PENDIENTE**).

7. Qué está verificado y qué no

Afirmación	Estado	Evidencia
La DevKit es un ESP32-D0WD-V3 con 4 MB de flash y sin PSRAM	VERIFICADO	`docs/hardware_cargar.md` §«Qué se probó» (13 oct 2026)
Los pines de motor, I2C y keep-alive son los de la tabla	VERIFICADO	`config.h:66-76`, `160-161`, `148`
El carro se mueve con los cuatro motores y responde a los comandos	VERIFICADO en J13/J14 (pruebas físicas B0-B6)	`docs/reporte_J14.md`
El ajuste recto MANUAL 252/255 dio 3 cm de desvío a 150 cm	VERIFICADO según `config.h:103-104`	Comentario de configuración, sin registro de medición aparte
El ajuste recto AUTO 207/210 está calibrado en el carro	NO MEDIDO	`config.h:96` lo declara pendiente
La corriente de los motores y del driver es segura en todos los casos	NO MEDIDO	`docs/conexiones.md` §5
El GPIO 2 necesita una carga y de qué valor	NO MEDIDO	`docs/conexiones.md` §3
La cámara GC0308 da 12.4 fps a HQVGA y ~137 ms de edad	VERIFICADO por barrido local	`docs/camara_prueba.md` §«Lo medido»
El retraso total del video en el teléfono	NO MEDIDO (solo una muestra de ~530 ms)	`docs/camara_prueba.md` §«Cómo repetir la prueba del cronómetro»
La autonomía del carro	NO MEDIDO	No hay registro en el repositorio
El comportamiento del driver con dos motores bloqueados	NO MEDIDO / HIPÓTESIS de riesgo	`docs/conexiones.md` §5

8. Pendientes de hardware

1. Medir corriente por motor en libre y bloqueado, y temperatura del TC1508A (`docs/conexiones.md` §5).
2. Confirmar qué hay conectado al GPIO 2 y si la powerbank lo necesita (`docs/conexiones.md` §3).
3. Confirmar el rótulo real de las entradas del módulo (IN1..IN4, INT1..INT4 o INA..IND) (`docs/conexiones.md` §7).
4. Verificar la polaridad y el sentido de cada motor con las ruedas en el aire antes de bajar el carro (`docs/conexiones.md` §6).

5. Documentar con fotos el montaje, el cableado y la sujeción de la cámara. La lista está en **`docs/lista_de_fotos.md`**.
6. Medir la autonomía real con una rutina repetible y anotar fecha, batería y resultado (**NO MEDIDO**).

Los defectos y huecos detectados en el código o en los documentos se anotan, sin corregirlos, en **`docs/pendientes_documentacion.md`**.

5. Firmware del carro (ESP32 DevKit)

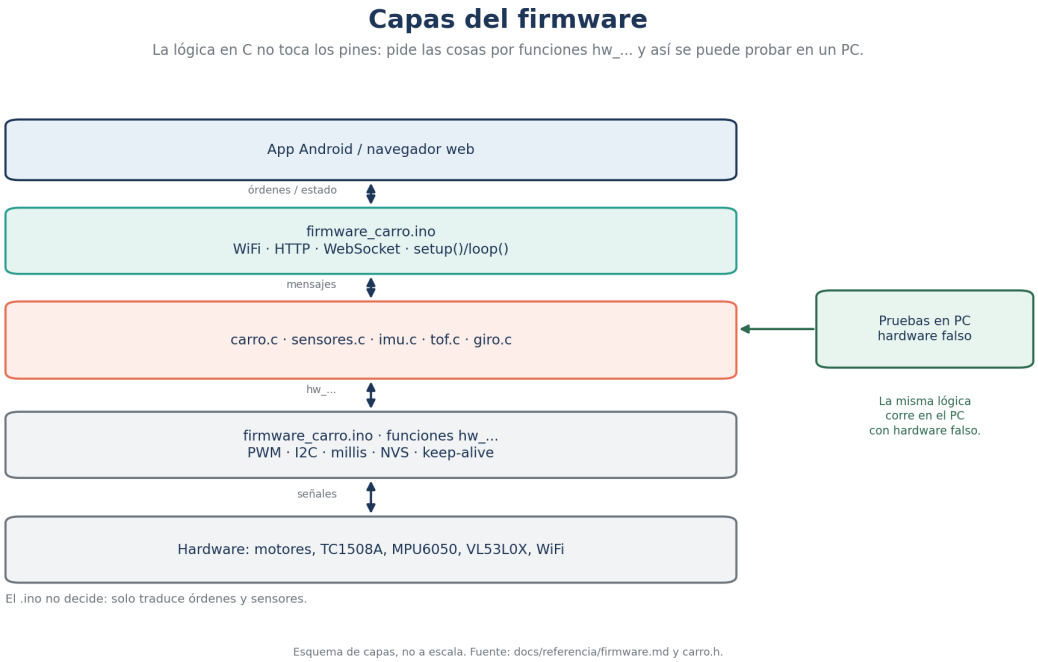


Figura 2: Capas del firmware. La lógica en C se separa del hardware por las funciones hw_..., y por eso se puede probar en el PC. (Esquema de capas, no a escala.)

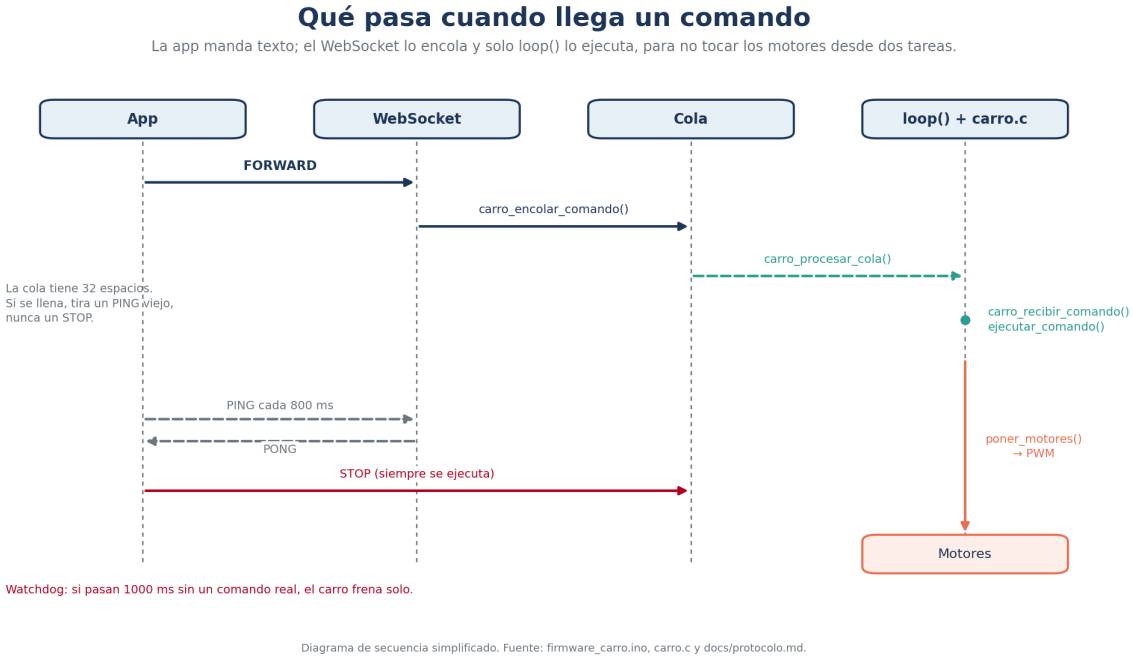


Figura 3: Qué pasa cuando llega un comando. Recorrido de un FORWARD desde la app hasta los motores, y el PING que mantiene viva la conexión. (Diagrama de secuencia simplificado.)

Referencia del firmware del carro

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Código documentado: `firmware_carro/` en `deae8b6`. **Fuentes:** `config.h`, `carro.h`, `carro.c`, `firmware_carro.ino`, `sensores.h`, `sensores.c`, `imu.*`, `tof.*`, `giro.*`, `docs/sensores.md` y `docs/pendientes_firmware.md`.

1. En una frase

El firmware recibe órdenes de texto por WiFi, las convierte en movimiento de cuatro motores y las corta solo si el teléfono deja de dar señales de vida; además lee un giroscopio y un sensor de distancia, y avisa a la app de lo que ve. Ese «cortar solo» es el **watchdog**: un vigilante interno que apaga los motores cuando pasa demasiado tiempo sin recibir una orden real.

La idea central es **separar las decisiones del hardware**. `carro.c` sabe *qué* hacer (avanzar, girar, parar, vigilar el tiempo) pero no sabe encender un pin: para eso llama a funciones `hw_...` que el archivo `.ino` implementa con Arduino. Gracias a ese corte, la lógica se puede probar en un PC con hardware falso, sin placa.

2. Archivos y capas

Archivo	Qué contiene	Se puede probar en PC
`config.h`	Todos los números ajustables: pines, tiempos, velocidades, umbrales	— (solo constantes)
`carro.h` / `carro.c`	Comandos, motores, perfiles, watchdog, cola y keep-alive	Sí, con `pruebas_firmware/`
`sensores.h` / `sensores.c`	Une los sensores con los comandos y los mensajes hacia la app	Sí
`imu.c` / `tof.c` / `giro.c`	Giroscopio, láser y giro por ángulo	Sí
`hw_sensores.h`	Lo que el `.ino` debe darle a los sensores (I2C, tiempo, memoria)	— (interfaz)
`firmware_carro.ino`	WiFi, servidor HTTP, WebSocket, I2C real y `setup()`/`loop()`	No: necesita la placa
`pagina.h`	Página web de control guardada en flash	No

La regla se ve en `carro.h:18-47`: `hw_motor_pwm`, `hw_millis`, `hw_keepalive`, `hw_registrar`, `hw_balance_leer/guardar` y los bloqueos. En el ESP32 las implementa el `.ino` (`firmware_carro.ino:250-350`, entre otras); en las pruebas las implementa un archivo falso.

3. Qué pasa al encender

`setup()` (`firmware_carro.ino:405-434`) hace, en este orden:

1. Abre el puerto serie a 115200 y escribe la versión (`FW_VERSION`).
2. Baja la CPU a 160 MHz para consumir menos (`config.h:63-64`).
3. Configura los cuatro canales PWM a 1500 Hz y 8 bits (`firmware_carro.ino:252-258`).
4. Deja el carro parado y el estado en `STOP` con `carro_iniciar()`.
5. Abre el bus I2C y arranca los sensores (`iniciar_i2c`, `sensores_iniciar`).
6. Configura el watchdog de hardware a 5000 ms.
7. Crea la red WiFi propia y configura el DHCP de .10 a .20.
8. Configura el pin keep-alive.
9. Publica la página web en `/`, el WebSocket en `/ws` y arranca el servidor.

El arranque siempre deja el perfil en **AUTO**, que es el más lento y el más seguro (`carro.c:345`). La app debe pedir `MODO_MANUAL` de forma explícita si quiere velocidad máxima.

4. El bucle principal

`loop()` (**`firmware_carro.ino:535-551`**) se repite sin parar y hace siempre lo mismo, en este orden:

```
revisar el watchdog de hardware (esp_task_wdt_reset)
limpiar clientes WebSocket viejos
leer sensores y mover el giro por ángulo   ← único lugar que toca el I2C
vaciar la cola de comandos
aplicar el watchdog de comunicación
actualizar el pulso de la powerbank
enviar mensajes de sensores a la app
leer comandos de diagnóstico por USB
anotar cuánto tardó esta vuelta (LOOP?)
```

El orden importa: los sensores se leen antes de ejecutar comandos, y el watchdog se revisa después de vaciar la cola. Así un **`STOP`** que acaba de llegar se ejecuta antes de que el vigilante decida frenar por su cuenta.

5. Cómo viaja un comando

La app envía texto por WebSocket, que es un canal abierto entre el teléfono y el carro: una vez conectado, los dos pueden mandar mensajes cuando quieran, sin preguntar y responder cada vez. El camino es:

```
teléfono → WebSocket /ws
        → al_evento_websocket()                (firmware_carro.ino:353)
        → carro_encolar_comando()              (carro.c:389)
        → loop(): carro_procesar_cola()        (carro.c:455)
        → carro_recibir_comando()              (carro.c:509)
        → ejecutar_comando()                   (carro.c:273)
        → avanzar()/retroceder()/girar_...()   (carro.c:115-160)
        → poner_motores() → hw_motor_pwm()      (carro.c:98)
```

La cola existe porque el WebSocket corre en otra tarea del sistema y no debe tocar los motores a la vez que **`loop()`**. Solo **`loop()`** cambia el estado del carro. La cola tiene 32 espacios (**`config.h:154`**) y, si se llena, primero tira un **`PING`** viejo y nunca un **`STOP`** (**`carro.c:399-429`**). Esa protección se probó con 100 000 comandos en estrés, sin perder ningún STOP (ver **`docs/reporte_J14.md`**).

Los comandos repetidos se ignoran (**`carro.c:217-223`**), salvo **`STOP`** y **`EBRAKE`**, que siempre se ejecutan. Así el heartbeat de la página no reinicia el movimiento una y otra vez.

6. Comandos que entiende el firmware

Los comandos son texto plano, sin JSON. La tabla siguiente es la lista completa; los detalles de sensores están en **`docs/sensores.md`** y el protocolo de la app en **`docs/protocolo.md`**.

Comando	Efecto	Respuesta	
`FORWARD`	Avanza con los dos motores	—	
`REVERSE`	Retrocede	—	
`LEFT`	Pivota a la izquierda (solo empuja el motor derecho)	—	
`RIGHT`	Pivota a la derecha (solo empuja el motor izquierdo)	—	
`STOP`	Frena ya	—	
`EBRAKE`	Igual que **`STOP`**	—	
`PING`	Mantiene viva la conexión; **no** renueva el watchdog	**`PONG`**	
`MODO_MANUAL`	Perfil rápido (255/252)	**`ACK_MANUAL`**	

`MODO_AUTO`	Perfil limitado (210/207)	`ACK_AUTO`	
`BAL_AUTO:NNN`	Cambia el balance del perfil AUTO (800-1000) y lo guarda	`ACK_BAL_AUTO:NNN` o `NAK_BAL`	
`BAL_MANUAL:NNN`	Igual para el perfil MANUAL	`ACK_BAL_MANUAL:NNN` o `NAK_BAL`	
`BAL?`	Consulta ambos balances	`BAL:AAA:MMM`	
`GIRAR:<grados>`	Gira un ángulo medido (5 a 200; positivo = izquierda)	`GIRO_OK`, `GIRO_TIMEOUT` o `GIRO_ERROR`	
`CAPS?`	Qué sensores funcionan	`CAPS:...`	
`IMU?`	Ángulo, sesgo, estado e identificación del MPU6050	`IMU:...`	
`TOF?`	Distancia y banderas del láser	`TOF:...`	
`IMU_CAL`	Recalibra el sesgo (solo con el carro quieto)	`ACK_IMU_CAL` o `NAK_IMU_CAL`	
`IMU_CERO`	Pone el ángulo actual en cero	`ACK_IMU_CERO`	
`TOF_USO:<0	1>`	Enciende o apaga la parada de emergencia del láser	`ACK_TOF_USO:<n>`
`LOOP?`	Peor duración de `loop()` y excesos	`LOOP:...`	
`ANT?`	Anticipaciones de giro aprendidas	`ANT:...`	
`ARR?`	Historial de reintentos al arrancar	`ARR:...`	
`ANT_BORRAR`	Borra las anticipaciones aprendidas	`ACK_ANT_BORRAR`	

Además, el firmware manda solo `SENS:<ángulo x10>,<mm>,<banderas>` cada 100 ms mientras hay cliente y el perfil es AUTO (`config.h:242-247`, `sensores.h:11-27`), y `TOF_PARO:<mm>` cuando el láser frena el carro.

Los comandos de diagnóstico `I2C?`, `PINES?` y `PRUEBAI2C?` se aceptan por el **puerto serie USB**, no por WiFi (`firmware_carro.ino:508-533`). `GIRAR` por USB se rechaza con `NAK_SERIE`: los motores solo se mueven por WebSocket (`sensores.c:311-316`).

7. Perfiles, balance y arranque

El carro tiene dos formas de andar:

• **MANUAL:** pensada para conducir con los botones. Tope derecho 255 y factor de balance 988, que deja el izquierdo en 252 (`config.h:120-123`). El ajuste se midió a 150 cm: 3 cm de desvío (`config.h:103-104`).

• **AUTO:** pensada para rutas y para el modo semiautónomo. Tope derecho 210 y factor 986, que deja el izquierdo en 207 (`config.h:125-128`). Este ajuste **no se midió en el carro**; está pendiente (`config.h:96`).

En los dos perfiles, al empezar a avanzar o retroceder los motores van al 100 % durante 250 ms y después se aplica el ajuste de cada lado (`config.h:106-108`, `carro.c:115-142`). El motivo es práctico: desde parado, con menos de 255 los motores solo zumban y no arrancan.

El **balance** es un factor de 800 a 1000 milésimas que solo reduce el lado izquierdo, porque en este carro el izquierdo empuja más (`config.h:112-118`). Se puede cambiar desde la app con `BAL_AUTO`/`BAL_MANUAL`, se guarda en memoria no volátil y se aplica en el siguiente arranque (`carro.c:292-299`, `341-368`).

8. El watchdog: por qué el carro no se escapa

Hay dos vigilantes distintos, y conviene no confundirlos:

1. **Watchdog de comunicación (software, 1000 ms).** Si hay motores en marcha y pasa más de un segundo sin un comando real, `carro_vigilar_comunicacion()` apaga los motores y vuelve al perfil AUTO (`carro.c:589-629`). `PING` no cuenta como comando real: si contara, el carro seguiría avanzando con el botón suelto, porque la página manda `PING` cada 800 ms (`carro.c:506-508`). Las consultas de sensores tampoco renuevan el reloj, salvo `GIRAR`, que sí es una orden de movimiento (`carro.c:520-523`).
2. **Watchdog de hardware (5000 ms).** Si `loop()` se cuelga por completo, el chip se reinicia solo (`firmware_carro.ino:266-285`). Se reconfigura el vigilante que el núcleo de Arduino ya trae, en vez de crear otro.

Durante un giro por ángulo, el watchdog de 1 s **no** actúa: un giro largo puede durar más. El giro tiene su propio tiempo máximo y se aborta con `STOP` o con otro movimiento (`carro.c:311-314`, `619-620`).

Cuando un cliente WebSocket se conecta o se desconecta, el firmware frena y vuelve a AUTO (`carro.c:477-499`, `firmware_carro.ino:356-376`). Si se cae el WiFi, el watchdog de comunicación hace el resto.

9. Sensores, en corto

• **MPU6050 (giroscopio):** mide la velocidad de giro y la integra para estimar el ángulo. Al arrancar promedia 250 lecturas con el carro quieto para calcular el sesgo; si el carro se mueve, reintenta (`config.h:181-188`). El ángulo se usa en `GIRAR` y se envía en `SENS`.

• **VL53L0X (láser):** mide distancia al frente cada 33 ms sin bloquear el bucle. Si hay dos lecturas seguidas por debajo de 120 mm y el perfil es AUTO y el carro avanza, frena con `__STOP_AUTO` y avisa `TOF_PARO:<mm>` (`config.h:223-240`). El comando `TOF_USO:0` apaga esa parada, pero el sensor sigue midiendo.

• **Giro por ángulo:** `GIRAR` usa un solo motor, frena con una anticipación aprendida y después deja asentar el carro para informar el ángulo final (`config.h:202-221`). Las anticipaciones se guardan por sentido y por perfil.

La explicación completa, con mensajes y fallos típicos, está en `docs/referencia/` (sensores) y en `docs/sensores.md`.

10. WiFi y servidor

• La ESP32 crea su propia red (AP). La IP del carro es **192.168.4.1**, el HTTP va en el puerto **80** y el WebSocket en `/ws` (`config.h:38-52`).

• El DHCP reparte de **.10 a .20** para dejar libre la **.2** de la cámara (`firmware_carro.ino:287-310`).

• El canal es fijo (1) y la potencia de radio se baja a 13 dBm para no exigir picos a la powerbank (`config.h:54-64`).

• Solo se atiende **un cliente WebSocket a la vez**: al conectarse uno nuevo, se cierran los demás (`firmware_carro.ino:356-364`).

• La página de control se sirve desde flash y sin caché (`firmware_carro.ino:321-330`).

El nombre y la contraseña de la red **no están en el repositorio**: viven en `firmware_carro/secretos.h`, que está ignorado por git. Si ese archivo no existe, se usa `secretos.ejemplo.h` (`config.h:20-36`). No se debe publicar el binario compilado porque contiene esos datos (ver `docs/entrega.md`).

11. Versión del firmware

`scripts/cargar.ps1` copia el sketch a una carpeta temporal, obtiene el hash corto del commit y la fecha, y escribe `version_compilada.h` con algo como `FW:453ed20-20261027` (`scripts/cargar.ps1:163-173`). `config.h` incluye ese archivo si existe (`config.h:10-18`). Así la versión que se ve en el puerto serie y en la app corresponde al commit con el que se compiló, sin ensuciar el repositorio.

La versión conservada en J15 es **FW:453ed20-20261027**. Su binario no se publica; el hash y el motivo están en `docs/entrega.md` §4. La lectura de la versión en la placa durante J15 quedó pendiente.

12. Límites y cosas que no hace

• No hay encoders: el avance se estima por tiempo, no por distancia medida. Por eso la pose puede desviarse y J15 añadió corrección con el rumbo del giroscopio (`docs/reporte_J15.md`).

• El giroscopio no detecta deslizamiento lateral puro; la corrección de pose no puede arreglar todo (`docs/entrega.md` §6).

• El láser solo mira al frente y a una altura fija; no ve huecos, escalones ni objetos fuera del haz.

• El perfil AUTO no se calibró en el carro: 207/210 es una extrapolación del ajuste manual (**`config.h:96`**).

• El firmware no distingue un comando desconocido de uno válido para el watchdog: un texto raro renueva el reloj y se guarda como comando actual (**`carro.c:330-335`**). Es el comportamiento original y está anotado en **`docs/pendientes_firmware.md`**.

• No se ha medido el consumo, la autonomía ni la temperatura del driver (ver **`docs/referencia/hardware.md`**).

• El pulso keep-alive en GPIO 2 conmuta a nivel alto durante 50 ms cada 2 s en el código, pero existe una discrepancia documental sobre su polaridad lógica y no se ha comprobado físicamente con osciloscopio: queda **PENDIENTE de confirmación física con osciloscopio o multímetro con el carro encendido**.

13. Para seguir leyendo

• **`docs/referencia/firmware.md`** (este documento): vista de conjunto.

• **`docs/codigo/firmware_carro.md`**: recorrido línea a línea de los archivos.

• **`docs/sensores.md`**: MPU6050 y VL53L0X en detalle.

• **`docs/protocolo.md`**: todos los mensajes y el comportamiento esperado de la app.

• **`docs/pendientes_firmware.md`** y **`docs/pendientes_documentacion.md`**: lo que falta o está mal.

6. Firmware de la cámara (Freenove ESP32-WROVER)

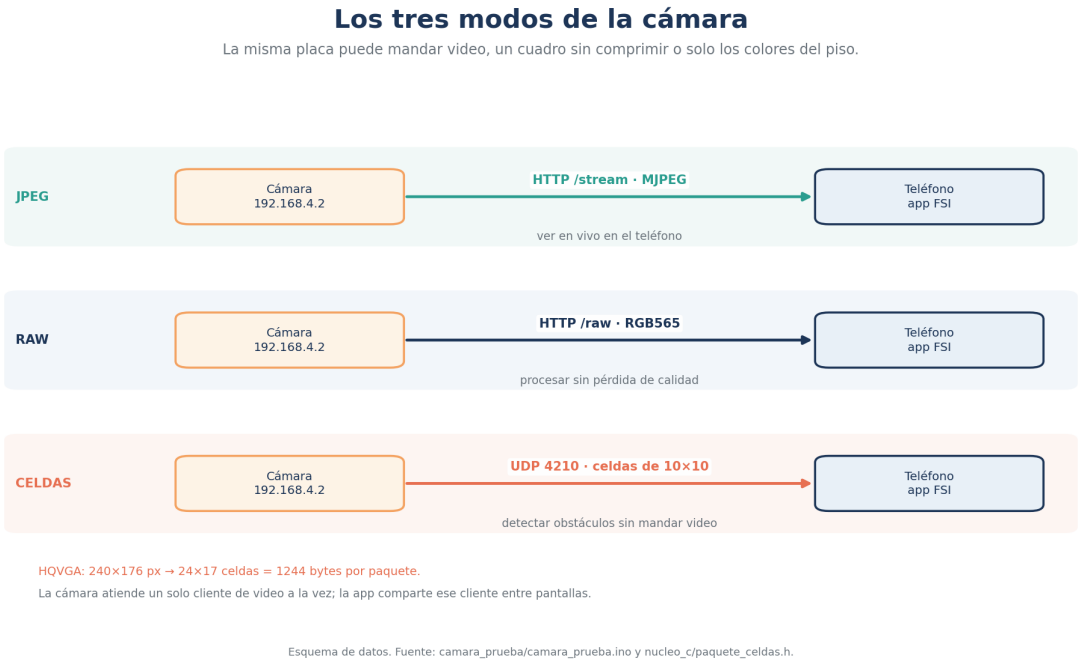


Figura 9: Los tres modos de la cámara. JPEG para ver, RAW para procesar sin pérdida y CELDAS para detectar obstáculos con muy pocos bytes. (Esquema de datos; la cámara atiende un cliente de video a la vez.)

[FOTO PENDIENTE: FOTO_06] — Placa de la cámara Freenove

Encuadre requerido: Detalle de lente GC0308, alimentación micro-USB independiente y antena
Archivo esperado: docs/imagenes/foto_06_camara_freenove.jpg

Referencia de la cámara ESP32

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Código documentado: `camara_prueba/` en `deae8b6`. **Fuentes:** `camara_prueba.ino`, `recuperacion.h/.c`, `docs/camara_prueba.md`, `docs/hardware_cargar.md` y `docs/entrega.md`.

El encargo llama a esta carpeta `firmware_camara`. En el repositorio se llama `camara_prueba/` y así se mantiene; este documento y `camara_prueba/README.md` explican el nombre.

1. En una frase

La cámara es una **placa aparte con WiFi** que mira al frente, comprime lo que ve y lo sirve por HTTP (el mismo protocolo con el que se abren las páginas web) para que el teléfono lo muestre; además puede mandar solo los colores del piso por UDP (un envío rápido, sin confirmación de llegada) para que el detector de obstáculos trabaje sin transmitir video.

No es una ESP32-CAM genérica: es una **Freenove ESP32-WROVER v3.0** con sensor **GC0308**, un sensor que no sabe hacer JPEG. Por eso la placa entrega la imagen en bruto y la ESP32 la comprime por software. Esa decisión explica casi todos sus límites de velocidad.

2. La placa y el sensor

Dato	Valor	Fuente

Placa	Freenove ESP32-WROVER v3.0	`docs/camara_prueba.md` §H2
Sensor	GC0308, 0.3 MP (PID 0x9B)	`docs/camara_prueba.md` §H2
PSRAM	4 MB	Igual
USB-serie	CH340	Igual
Mapa de pines	`CAMERA_MODEL_WROVER_KIT`	`camara_prueba.ino:24-26`
XCLK / SIOD / SIOC	21 / 26 / 27	`docs/camara_prueba.md` §H2
Datos Y2..Y9	4, 5, 18, 19, 36, 39, 34, 35	Igual
VSYNC / HREF / PCLK	25 / 23 / 22	Igual
PWDN / RESET	No usados	Igual

El sensor **no acepta el formato JPEG**: si se intenta, la inicialización falla con `0x106 ESP_ERR_NOT_SUPPORTED`. Por eso el programa pide RGB565 o YUV422 y comprime cada cuadro con `frame2jpg` (`camara_prueba.ino:2-15`). El mapa `AI_THINKER` tampoco sirve en esta placa.

Para que no se den por sabidos los formatos: **JPEG** es una foto comprimida, más pequeña pero con pérdida de calidad; **RGB565** y **YUV422** guardan cada píxel sin comprimir, así que ocupan mucho más pero conservan todo el detalle. La placa no tiene un compresor JPEG en el sensor, y por eso gasta tiempo de procesador en comprimir.

3. La red

La cámara se conecta como **cliente** a la red que crea el carro:

• IP fija **192.168.4.2**, puerta 192.168.4.1, máscara 255.255.255.0 (`camara_prueba.ino:89-91`).

• La IP es fija a propósito: el teléfono siempre sabe dónde buscar el video.

• El DHCP del carro reparte de .10 a .20 para no entregar la .2 (`config.h:42-52`; `firmware_carro.ino:287-310`).

• `WiFi.setSleep(false)` para no dormir la radio y no aumentar el retraso.

• Si la red no está o se cae, reintenta con esperas de 1, 2, 4, 8 y 10 s. A los 60 s sin conectar, se reinicia (`recuperacion.h:22-28`).

La cámara **no crea** su propia red: necesita que la DevKit esté encendida. Los dos dispositivos se pueden encender en cualquier orden; la cámara entra sola cuando aparece la red.

4. Los tres modos de video

El modo se cambia con `/control?modo=...`. Al reiniciar, vuelve siempre a `jpeg` (`camara_prueba.ino:167-171`, `1273-1287`).

Modo	Qué manda	Para qué	Dónde
`jpeg`	Video MJPEG (una sucesión de fotos JPEG) y capturas sueltas	Ver por el teléfono	`/stream`, `/capture`
`raw`	Un cuadro RGB565 sin comprimir	Procesar la imagen en el teléfono sin pérdida	`/raw`
`celdas`	Promedio RGB de cada celda de 10×10 por UDP	Detector de obstáculos sin video	UDP 4210

En modo `celdas`, el JPEG sigue disponible en `/stream`; lo que cambia es que además se manda el paquete de celdas. El paquete tiene la marca `CEL1`, número de cuadro, antigüedad, tamaño, columnas, filas y los colores (`nucleo_c/paquete_celdas.h:1-30`). Con HQVGA son 24×17 celdas = 1244 bytes, que caben en un datagrama UDP.

El teléfono pide las celdas mandando `HOLA` al puerto 4210; la cámara responde `HOLA-OK` y recuerda la dirección. `PING n` se responde con `PONG n` para medir el retardo (`camara_prueba.ino:643-673`).

5. Las rutas HTTP

Ruta	Devuelve	Uso
`/`	Página con el video y los números	Ver en el navegador
`/capture`	Un JPEG	Foto suelta
`/stream`	MJPEG continuo	Video
`/raw`	Un cuadro RGB565	Procesamiento sin compresión
`/control?size=...`	Cambia QQVGA/HQVGA/QVGA/VGA	Resolución
`/control?comp=...`	Cambia el compresor (`esp`, `420`, `444`, `baja`)	Comparar compresores
`/control?modo=...`	Cambia `jpeg`, `raw` o `celdas`	Modo de video
`/stats`	JSON con fps, tiempos, memoria y estado	Diagnóstico
`/salud`	`ok`	Vigilante interno

Cada petición se anota en el puerto serie como `GET <ruta> desde <ip>` (`camara_prueba.ino:21-22`). `/stats` incluye, entre otros: fps de captura y envío, edad del cuadro, ms de compresión y envío, descartados, RSSI, memoria libre, `uptime_s`, reconexiones, último reinicio y estado del modo celdas (`camara_prueba.ino:1315-1335`).

6. Cómo se procesa un cuadro

El programa usa **dos núcleos** para no acumular retraso (`camara_prueba.ino:8-15`):

```
núcleo 0: tarea_captura
   pide un cuadro a la cámara
   lo deja en UNA sola casilla
   si la casilla estaba ocupada, tira el viejo

núcleo 1: tarea_stream / manejar_captura
   toma el cuadro más reciente
   lo comprime con frame2jpg
   lo envía por HTTP
```

La casilla única es la clave: si el teléfono va lento, se pierden cuadros viejos en vez de acumularse. Un cuadro que ya tiene más de 450 ms al salir de la cámara se descarta (`EDAD_MAX_US`, `camara_prueba.ino:95`). Sin demanda de video, la captura se detiene; así no se guarda un cuadro que envejece.

La configuración por defecto en el código es:

```
static const Config CONFIG_DEFECTO = { 20, PIXFORMAT_RGB565, FRAMESIZE_HQVGA, 50 };
```

Es decir, 20 MHz, RGB565, **HQVGA 240×176** y calidad JPEG 50 (`camara_prueba.ino:105-108`). El comentario del código dice que se eligió HQVGA por sus ~12 fps y ~0,32 s de retraso, y que QVGA se siente más lenta.

> **Contradicción detectada:** `docs/camara_prueba.md` §«Lo medido» concluye «se queda QVGA como valor por defecto», pero el código actual usa HQVGA. Se anota en `docs/pendientes_documentacion.md`. Para el manual vale el código; si se prefiere QVGA, se cambia con `/control?size=qvga` o en `CONFIG_DEFECTO`.

7. Rendimiento medido

El barrido de H3 midió 100 cuadros por combinación, sin cliente, con luz intermedia (Y ≈ 124-127). Los números completos están en `docs/camara_prueba.md` §«Lo medido». Los más útiles:

Configuración	fps de salida	Compresión	Edad del cuadro
20 MHz, RGB565, QVGA	8,8	107 ms	252 ms
20 MHz, RGB565, HQVGA	12,4	56 ms	137 ms

20 MHz, RGB565, QQVGA	10,0	26 ms	126 ms
16 MHz, RGB565, HQVGA	12,8	56 ms	135 ms
10 MHz, RGB565, QVGA	5,0	101 ms	301 ms

Qué significan:

• **La resolución manda.** HQVGA tiene el 55 % de los píxeles de QVGA y da ~41 % más fps y ~46 % menos edad.

• **RGB565 comprime más rápido que YUV422** (107 contra 125 ms en QVGA) y genera JPEG algo más chicos.

• **La calidad 50 contra 70 casi no cambia el tiempo** (107 contra 109 ms), pero baja el tamaño ~23 %. Por eso se queda 50.

• **El límite es el sensor y la compresión por software**, no la red. Con un sensor con JPEG por hardware (OV2640) el límite cambiaría.

La edad del cuadro se mide desde que empieza la captura hasta que termina el envío. Incluye la lectura del sensor (~100 ms). El **retraso total hasta la pantalla del teléfono no está medido**: solo hay una muestra histórica de ~530 ms (`docs/camara_prueba.md` §«Cómo repetir la prueba del cronómetro»). Cualquier cifra de latencia de extremo a extremo es **NO MEDIDA**.

8. Autoprueba y recuperación

Al arrancar, con `AUTOPRUEBA 1`, la placa se pide a sí misma `/stats`, `/capture` y `/stream` por 127.0.0.1 (`camara_prueba.ino:84-85`). Eso comprueba que el servidor responde, pero **no mide el WiFi**: el cliente local comparte el procesador y los resultados son pesimistas. En H3 dio 8,2 fps en `/stream` y 250 ms de edad media (`docs/camara_prueba.md` §«Autoprueba local»).

El vigilante de `recuperacion.c` decide, y el `.ino` actúa:

Situación	Qué hace
Se pierde el WiFi	`WiFi.begin` con espera creciente; a los 60 s, reinicio
`/salud` no responde 3 veces seguidas (30 s)	Reinicio
Alguien pide cuadros y no llega ninguno en 5 s	Reinicio
`loop()` no avanza en 8 s	Reinicio

El motivo del reinicio se guarda en memoria RTC y sale en `/stats` (`ultimo_reinicio`) y por el puerto serie. Los códigos y el detalle están en `docs/camara_prueba.md` §«Robustez del WiFi y de la cámara».

Lo que **no** cubre: si el sensor no inicia al arrancar, la placa se queda detenida y no se reinicia sola; si la radio se cuelga pero `WiFi.status()` sigue diciendo «conectado», la placa tampoco lo nota (`docs/camara_prueba.md` §«Qué NO cubre»).

9. Compilar y cargar

```
.\scripts\cargar.ps1 camara -SoloCompilar
.\scripts\cargar.ps1 camara -Puerto COM8
```

Se compila para **ESP32 Wrover Module** (`esp32:esp32:esp32wrover`, con PSRAM). `cargar.ps1` copia el sketch a una carpeta temporal y, si existen, los archivos `obstaculos.c/h` y `paquete_celdas.c/h` de `nucleo_c/` para el modo CELDAS (`scripts/cargar.ps1:176-190`). Sin esos archivos, el sketch compila solo con JPEG y RAW.

La tarjeta microSD de la placa debe estar vacía: no se usa. El secreto de la WiFi vive en `camara_prueba/secretos.h`, ignorado por git; si no existe, se usa `secretos.ejemplo.h`. **No se publica ni se lee en la documentación.**

La cámara ocupa ~83 % de la flash al compilar (`docs/camara_prueba.md` §«Cargar»). El binario no se versiona.

10. Qué se probó y qué no

Afirmación	Estado	Evidencia
La placa es Freenove con GC0308 y PSRAM	VERIFICADO	`docs/camara_prueba.md` §H2
El sensor no hace JPEG	VERIFICADO	Error `0x106` documentado
El barrido de fps y compresión	VERIFICADO en local, sin cliente	`docs/camara_prueba.md` §«Lo medido»
El video se ve en el teléfono	VERIFICADO en pruebas de H3	Documento citado
El retraso real hasta la pantalla	NO MEDIDO (una muestra de ~530 ms)	Documento citado
La recuperación ante caída de WiFi	VERIFICADO en pruebas de PC (34 casos) y en la placa, según el documento	`pruebas_camara.c`, `docs/camara_prueba.md`
El modo CELDAS por UDP en el teléfono	VERIFICADO según la bitácora; no se repitió en el cierre	`docs/camara_prueba.md`, `docs/bitacora.md`
El comportamiento con la cámara montada en el carro en movimiento	NO MEDIDO	No hay registro

11. Límites y cuidados

• **No es la cámara de la app:** la app usa la cámara del teléfono para el detector en `VisionActivity`; la del carro se usa para video.

• **Un solo cliente de video:** si dos dispositivos piden `/stream`, el segundo puede desplazar al primero o quedarse esperando. La app lo comparte entre pantallas, pero no con otro teléfono.

• **La cámara necesita la red del carro.** Si la DevKit está apagada, no hay video.

• **La IP .2 es fija.** Si otro dispositivo la toma, hay choque. El DHCP del carro ya la evita.

• **El modo RAW solo funciona con RGB565**; con YUV422 responde 503 (`camara_prueba.ino:1215`).

• **El modo `celdas` necesita que el sketch se haya compilado con los archivos del núcleo.** Si se abre suelto en el IDE de Arduino, no existe.

• **No se tocan los registros del sensor**: no hay hoja de datos del GC0308 en el proyecto y escribir a ciegas no es seguro (`docs/camara_prueba.md` §H3).

• **La cámara no se ha probado montada y en movimiento**; la vibración y el cambio de luz pueden afectar al detector.

12. Para seguir leyendo

• `docs/codigo/camara_prueba.md`: recorrido por funciones y tareas.

• `docs/camara_prueba.md`: documento histórico con el barrido completo.

• `docs/referencia/hardware.md`: alimentación y montaje.

• `docs/protocolo.md`: cómo pide la app el video y las celdas.

• `camara_prueba/README.md`: arranque rápido y rutas.

7. Protocolo de comunicación

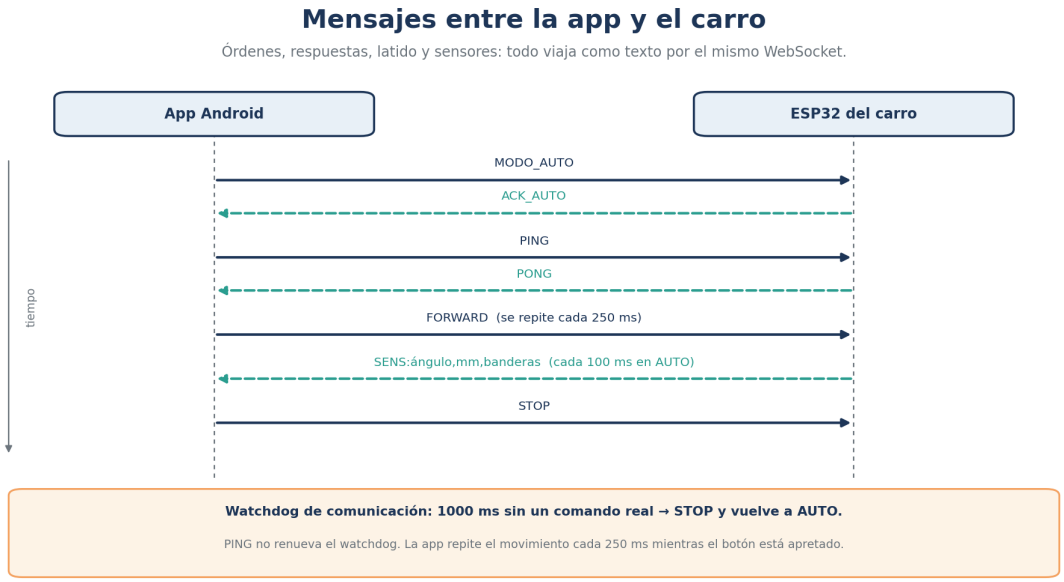


Diagrama de secuencia simplificado. Fuente: app_android/ControlLogica.kt, firmware_carro/carro.c y docs/protocolo.md.

Figura 11: Mensajes entre la app y el carro. Órdenes, ACK, PING/PONG, SENS y el watchdog que frena si no llega un comando real. (Diagrama de secuencia simplificado; los tiempos son los del código.)

Protocolo de comunicación del carro FSI

Materia: Fundamentos de los Sistemas de Información (FSI)

Canal principal: WebSocket sobre TCP/IP (`ws://192.168.4.1/ws`)

Formato: Texto plano en codificación ASCII/UTF-8, sin envoltorios JSON ni binarios

Fecha de referencia: 27 de octubre de 2026

---

1. Principios de diseño

1. **Simplicidad de procesamiento:** El ESP32 es un microcontrolador que no debe desperdiciar ciclos ni memoria dinámica parseando estructuras JSON complejas. Los mensajes son palabras clave sencillas delimitadas por retornos de línea o separadores de dos puntos (`:`).
2. **Asincronía total:** A través de un único socket WebSocket bidireccional, la app envía órdenes de movimiento y el carro emite ráfagas de telemetría de sensores sin requerir un esquema de petición-respuesta bloqueante.
3. **Prioridad absoluta para la seguridad:** Los comandos de detención (**`STOP`** y **`EBRAKE`**) tienen vía libre en la cola y se ejecutan de manera inmediata, incluso si la cola de mensajes se encuentra saturada.

---

2. Comandos de control de movimiento

Comando enviado	Efecto en motores	Respuesta del carro	Notas de seguridad

FORWARD	Avanza en línea recta aplicando el balance de potencia del perfil activo.	Ninguna	Arranca a potencia 255 durante 250 ms para vencer inercia inicial y luego baja al balance.
REVERSE	Retrocede en línea recta.	Ninguna	Mismo comportamiento de arranque pleno y balance.
LEFT	Pivote hacia la izquierda (rueda derecha empuja adelante a potencia 255; rueda izquierda parada).	Ninguna	No se aplica escalonamiento en pivotes.
RIGHT	Pivote hacia la derecha (rueda izquierda empuja adelante a potencia 255; rueda derecha parada).	Ninguna	Giro con un solo lado activo.
STOP	Apaga el PWM de los cuatro pines de motor (0 %).	Ninguna	Se ejecuta SIEMPRE. Si se envía repetido, limpia cualquier residuo en los registros.
EBRAKE	Freno de emergencia (idéntico a **STOP**).	Ninguna	Comando de máxima prioridad para botones de pánico.

3. Modos de conducción y calibración de balance

El carro cuenta con dos perfiles de velocidad independientes (**firmware_carro/config.h**):

Comando enviado	Respuesta	Descripción del perfil
MODO_AUTO	**ACK_AUTO**	Perfil para navegación autónoma y pruebas de ruta. Velocidad base 210, motor izquierdo ajustado a 207. Envío activo de telemetría **SENS**. Si el carro estaba en marcha al conmutar, primero se detiene.
MODO_MANUAL	**ACK_MANUAL**	Perfil de velocidad máxima para conducción manual desde la app (255 derecho / 252 izquierdo medido). No emite ráfagas **SENS** para no saturar el canal de teleoperación.
BAL_AUTO:NNN	**ACK_BAL_AUTO:NNN** o **NAK_BAL**	Ajusta el factor de balance del perfil AUTO en milésimas (rango permitido: 800 a 1000). Se guarda en memoria no volátil del ESP32.
BAL_MANUAL:NNN	**ACK_BAL_MANUAL:NNN** o **NAK_BAL**	Ajusta el factor de balance del perfil MANUAL en milésimas (800 a 1000) y lo almacena.
BAL?	**BAL:AAA:MMM**	Consulta los balances guardados: **AAA** = factor AUTO, **MMM** = factor MANUAL (ej. **BAL:986:988**).

4. Giro de precisión en bucle cerrado

Para realizar giros angulares exactos sin depender de aproximaciones de tiempo, el firmware implementa un control cerrado supervisado por el giroscopio MPU6050:

• **Sintaxis:** **GIRAR:<grados>**

• **Rango admitido:** **5** a **200** grados enteros.

• **Convenio de signo:** Grados positivos indican giro a la **izquierda** (sentido antihorario visto desde arriba); grados negativos indican giro a la **derecha**.

• **Respuestas posibles:**

• **`GIRO_OK`**: El vehículo completó el giro angular dentro de la tolerancia programada.

• **`GIRO_TIMEOUT`**: Expiró el tiempo límite de maniobra antes de alcanzar el ángulo deseado (ej. ruedas patinando o carro atorado).

• **`GIRO_ERROR:<motivo>`**: Falla en la ejecución (ej. **`GIRO_ERROR:sin_imu`** si el giroscopio no está disponible o se encuentra descalibrado).

---

5. Telemetría periódica y eventos de seguridad

5.1 Trama de telemetría `SENS`

En perfil **`AUTO`**, si hay un cliente WebSocket enlazado, el ESP32 transmite periódicamente cada **100 ms** un mensaje de estado compuesto por tres campos:

```
SENS:<angulo_x10>,<distancia_mm>,<banderas>
```

Ejemplo: **`SENS:450,320,7`**

• **Campo 1 (`angulo_x10`):** Ángulo actual acumulado por el giroscopio multiplicado por 10. En el ejemplo, **`450`** representa **`45.0`** grados.

• **Campo 2 (`distancia_mm`):** Lectura milimétrica del sensor de distancia láser VL53L0X. En el ejemplo, **`320`** representa 320 mm (32 cm). Si no hay objeto en rango, reporta un valor fuera de rango.

• **Campo 3 (`banderas`):** Entero decimal que representa una máscara de bits con el estado del subsistema:

• **Bit 0 (valor 1):** MPU6050 inicializado y en estado operativo (**`IMU_LISTO`**).

• **Bit 1 (valor 2):** Sensor láser VL53L0X vivo y respondiendo transacciones I2C.

• **Bit 2 (valor 4):** Medición de distancia válida y dentro de rango óptico confiable.

• **Bit 4 (valor 16):** Maniobra de giro en curso (**`GIRO_ACTIVO`**).

Nota: Un valor de banderas **`7`** (1 + 2 + 4) indica salud perfecta: IMU lista, láser respondiendo y distancia válida.

5.2 Notificación de parada de emergencia ToF

Cuando el sensor láser detecta un obstáculo a una distancia inferior al umbral crítico mientras el vehículo avanza, el firmware detiene los motores de manera autónoma y difunde por el WebSocket:

```
TOF_PARO:<distancia_mm>
```

---

6. Monitoreo de latencia y Watchdog

• **Comando `PING`:** La aplicación Android envía periódicamente **`PING`** para estimar el retardo de la red inalámbrica.

• **Respuesta `PONG`:** El ESP32 responde inmediatamente **`PONG`**.

• **Regla crítica del Watchdog:** El comando **`PING`** **NO renueva el temporizador del watchdog**. Para que el carro mantenga los motores encendidos durante un avance manual, la app debe repetir el comando **`FORWARD`** cada 250 ms. Si pasan **1000 ms** sin recibir un comando de movimiento real, el vigilante apaga los motores automáticamente.

---

7. Consultas de diagnóstico y sensores

Comando	Formato de respuesta	Descripción		
`CAPS?`	`CAPS:IMU=<0\	1>,TOF=<0\	1>`	Reporta qué sensores fueron detectados en el bus I2C durante el arranque.

`IMU?`	`IMU:estado,angulo,sesgo,id`	Diagnóstico del giroscopio: estado, ángulo actual, sesgo estimado y registro WHO_AM_I.		
`TOF?`	`TOF:distancia_mm,valida`	Diagnóstico de distancia cruda del láser.		
`IMU_CAL`	`ACK_IMU_CAL` o `NAK_IMU_CAL`	Fuerza recalibración del sesgo del giroscopio (solo admitido con motores parados por >300 ms).		
`IMU_CERO`	`ACK_IMU_CERO`	Establece el ángulo actual del giroscopio como nueva referencia de 0 grados.		
`TOF_USO:<0\	1>`	`ACK_TOF_USO:<0\	1>`	Activa (`1`) o desactiva temporalmente (`0`) la autoridad de parada de emergencia del láser ToF.
`LOOP?`	`LOOP:peor_us,excesos`	Reporta el peor tiempo de ejecución de una vuelta del bucle `loop()` en microsegundos.		
`ARR?`	`ARR:reintentos,motivo`	Registro de incidencias ocurridas durante la inicialización de periféricos.		

8. Gestión y descarte en la cola de comandos

El firmware dispone de un buffer circular de 32 ranuras (`COLA_COMANDOS_MAX`). Si una ráfaga saturara la cola: 1. Se descartan prioritariamente mensajes `PING` antiguos no despachados. 2. Si continúa saturada, se descartan comandos de consulta de diagnóstico. 3. **Bajo ninguna circunstancia se descarta un comando `STOP` o `EBRAKE`**.

9. Referencias visuales

• `docs/figuras/fig_03_flujo_comando.png`: Recorrido de los mensajes desde la app hasta el hardware.

• `docs/figuras/fig_11_protocolo.png`: Secuencia temporal y periodicidad de mensajes WebSocket.

8. Núcleo de navegación en C (nucleo_c)

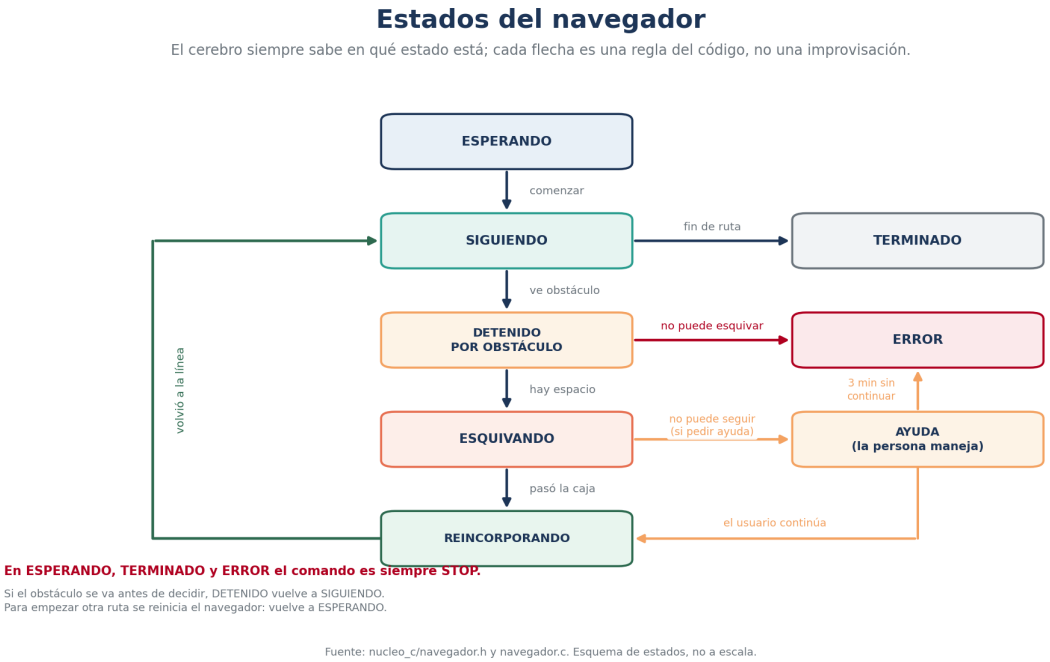


Figura 4: Estados del navegador. El navegador siempre está en un estado conocido y solo cambia con reglas explícitas. (En ESPERANDO, TERMINADO y ERROR el comando es siempre STOP.)

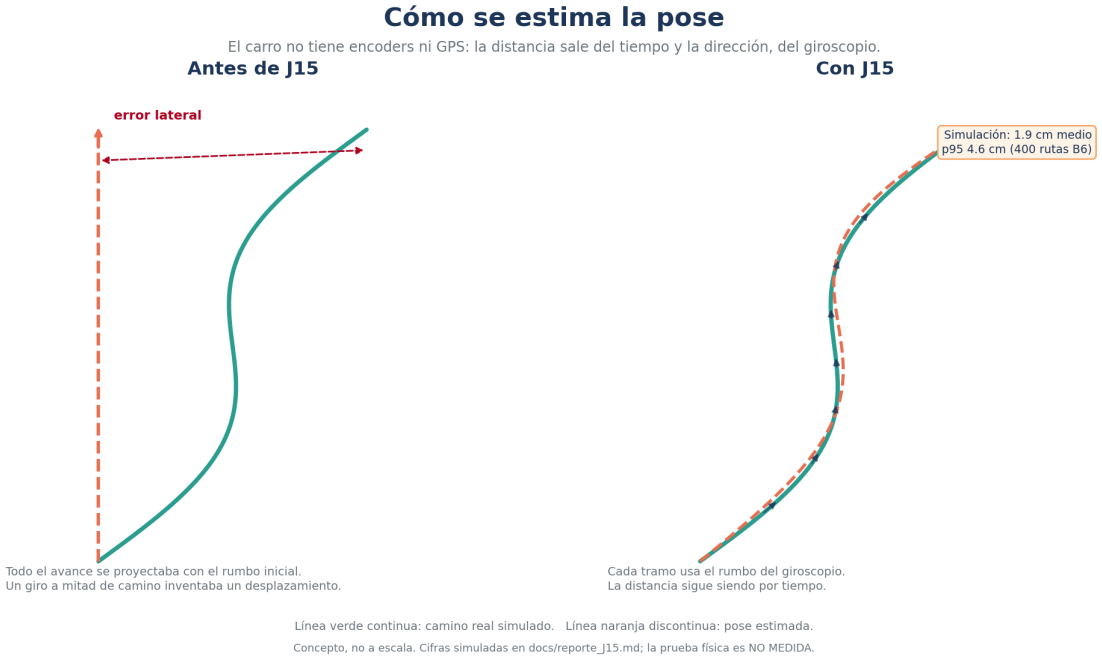
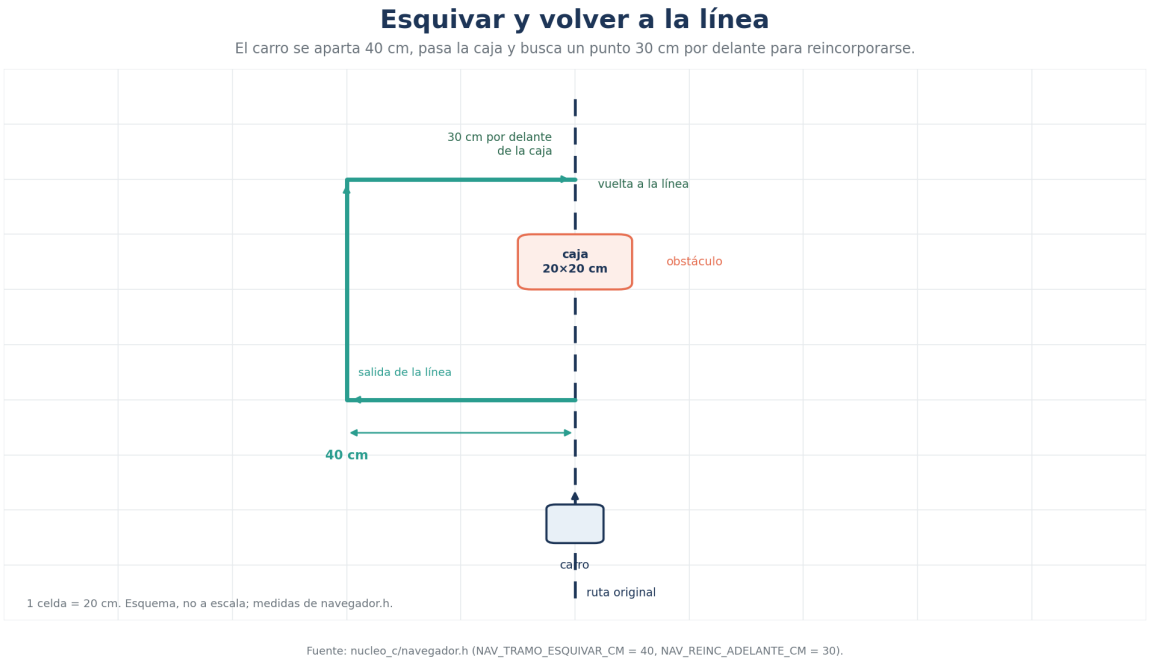


Figura 5: Cómo se estima la pose. Antes todo el avance se proyectaba con un rumbo fijo; J15 integra cada tramo con el rumbo del giroscopio. (Concepto; cifras simuladas en docs/reporte_J15.md. La prueba física es NO MEDIDA.)



Fuente: nucleo_c/navegador.h (NAV_TRAMO_ESQUIVAR_CM = 40, NAV_REINC_ADELANTE_CM = 30).

Figura 6: Esquivar y volver a la línea. El carro se aparta 40 cm, pasa la caja y busca un punto 30 cm por delante para reincorporarse. (Esquema, no a escala; medidas de navegador.h.)



Ejemplo ilustrativo: 50 ms/cm y 10 ms/grado son valores de partida. Los reales se miden con la app (docs/calibracion.md).

Figura 12: Navegar por tiempo. El carro convierte distancia y ángulo en tiempo con la calibración; el arranque y el frenado se miden aparte. (Ejemplo ilustrativo; los valores reales se miden con la app.)

Referencia del núcleo de navegación en C

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Código documentado: `nucleo_c/` en `deae8b6`. **Fuentes:** cabeceras y archivos `.c` del núcleo, `docs/navegador.md`, `docs/obstaculos.md`, `docs/calibracion.md`, `docs/reporte_J15.md` y `nucleo_c/Makefile`.

1. En una frase

El núcleo es el **cerebro que planea** el movimiento: recibe una ruta dibujada en una cuadrícula, la convierte en pasos con tiempos, mira si la cámara ve un obstáculo y decide en cada instante qué comando mandar al carro (`FORWARD`, `LEFT`, `STOP`...).

No habla con el hardware ni con Android: es C estándar, sin librerías externas, sin `malloc` (memoria dinámica: pedirle memoria al sistema mientras el programa corre) y solo con números enteros. Eso permite compilarlo y probarlo en un PC, y usarlo desde la app por JNI (el puente que deja que Kotlin llame a código C) sin cambiar una línea.

2. Los módulos, uno por uno

El núcleo creció por capas. Esta tabla ayuda a ubicar cada archivo; después se explican las capas importantes.

Módulo	Qué resuelve	Usado por
`trayectoria.c`	Convierte una lista de puntos de la cuadrícula en pasos `AVANZAR`/`GIRAR` con su tiempo	App (pantalla de ruta), navegador
`calibracion.c`	Guarda las medidas del carro (arranque, ms/cm, frenado, ms/grado) y convierte distancia/ángulo a tiempo	Trayectoria, navegador, app
`obstaculos.c`	Detector por color de piso: divide la imagen en celdas y dice si hay algo que no es piso	App (visión), cámara, navegador
`corredor.c`	Calcula la franja de la imagen por la que el carro va a pasar (con perspectiva)	App, navegador
`movimiento_visual.c`	Estima cuánto giró el carro comparando dos cuadros de la cámara	App, navegador
`movimiento_piso.c`	Comprueba si el piso se mueve como debería al avanzar; lo que se mueve distinto puede ser un objeto	App
`fusion.c`	Combina el detector de color con el modelo de objetos; el color manda en seguridad	App
`cualquiera.c`	Modo «cualquier objeto»: usa el modelo solo para detectar una caja, sin importar la clase	App
`mascara_vision.c`	Aprende artefactos fijos de la imagen (viñeteo, parte del carro) y los ignora	App
`calgiro.c`	Calibra el giro con el giroscopio: ocho giros guiados por lado	App
`navegador.c`	El cerebro: sigue la ruta, frena, esquiva, vuelve a la línea y pide ayuda	App, simulador
`ruta_editor.c`	Lógica del editor de rutas (vértices, deshacer, plantillas, guardar)	App
`sim_ruta.c` + `simulador_navegador.c`	Carro virtual para probar rutas y esquives sin el carro	App, pruebas
`sesion_compartida.c`	Decides cuándo abrir/cerrar un recurso compartido (video, conexión)	App
`video_stats.c`	Mide fps y huecos del video	App
`latencia_video.c`	Compara el PING con y sin video	App
`paquete_celdas.c`	Formato UDP del video resumido en celdas	Cámara, app
`pisos_sinteticos.c`	Genera pisos difíciles para pruebas	Solo pruebas

3. La cadena principal: de una ruta a un comando

```
puntos de la cuadrícula
        ■  trayectoria_calcular_desde_cal()
        ▼
pasos AVANZAR/GIRAR con tiempo_ms
        ■  navegador_cargar_ruta() + navegador_comenzar()
        ▼
navegador_tick() cada ~50 ms  ← resultado del detector / cámara
        ■
        ▼
FORWARD / REVERSE / LEFT / RIGHT / STOP
        ■
        ▼
la app lo manda por WebSocket (o el simulador lo ejecuta)
```

La ruta se dibuja en celdas de **20 cm** (`trayectoria.h:19`). Cada tramo produce como máximo un giro y un avance; una ruta válida nunca desborda el arreglo de salida (`trayectoria.h:22-24`). Los tiempos salen de la calibración; si no hay calibración, se usan 50 ms/cm y 10 ms/grado como valores de partida (`calibracion.h:32-35`).

4. Los estados del navegador

El navegador es una **máquina de estados**: una forma de programar en la que el sistema está siempre en uno de unos pocos estados y solo cambia a otro cuando se cumple una regla. Así, en todo momento se sabe qué está haciendo el carro y qué comando tiene permitido.

Estado	Qué significa	Comando
`ESPERANDO`	Ruta cargada pero no empezada	`STOP`
`SIGUIENDO`	Ejecuta el paso actual	El paso
`DETENIDO_OBSTACULO`	Vio algo y se detuvo a decidir	`STOP`
`ESQUIVANDO`	Gira, avanza de lado y mira	Movimiento de esquive
`REINCORPORANDO`	Vuelve a la línea original	Movimiento de vuelta
`AYUDA`	Pide que una persona tome el control	El de la cruceta
`TERMINADO`	Llegó al final	`STOP`
`ERROR`	No pudo continuar	`STOP`

La regla de seguridad está escrita en el código y se comprueba en cada `tick`: en `ESPERANDO`, `TERMINADO` y `ERROR` el comando es **siempre** `STOP`, pase lo que pase antes (`navegador.h:17`; `navegador.c:3772-3775`).

El flujo normal es:

1. `SIGUIENDO`: avanza o gira según el paso.
2. Si la cámara ve un obstáculo cerca: `DETENIDO_OBSTACULO`, pausa de 500 ms para decidir (`NAV_PAUSA_MS`).
3. Si hay espacio: `ESQUIVANDO` (giro de 90°, avance lateral de 40 cm, otro giro).
4. `REINCORPORANDO`: vuelve a la línea, pasando primero el obstáculo para no cruzarlo.
5. Vuelve a `SIGUIENDO` hasta `TERMINADO`.

Si no puede esquivar, por defecto termina en `ERROR`; si la app activa «pedir ayuda», pasa a `AYUDA` y espera a que la persona lo maneje (`navegador.h:18-21`).

5. Cómo sabe dónde está: la pose

El carro **no tiene encoders ni GPS**. Su posición se estima integrando el tiempo que estuvo moviéndose, con la calibración. Eso se llama **pose estimada**: `x`, `y` y rumbo.

• Cada paso guarda cuántos milisegundos hay que mantener el comando. Con los ms/cm se convierte en distancia; con los ms/grado, en ángulo (`navegador.c:381-405`).

• La pose se actualiza en cada `tick` y se usa para planear la esquiva y la vuelta a la línea.

• Un error de calibración se acumula: por eso la pose no es una medida, es una estimación.

Lo que cambió J15: antes, todo el avance se proyectaba con el último rumbo conocido. Ahora, si el giroscopio está fresco, cada incremento de avance se integra con el rumbo observado; el código lo explica en `navegador.c:345-379`. La distancia **sigue siendo por tiempo**; lo que mejora es la dirección. En simulación, el error lateral medio bajó de 13.2 a 1.9 cm y el p95 de 21.9 a 4.6 cm en 400 rutas B6, pero **la prueba física quedó NO MEDIDA** (`docs/reporte_J15.md`).

6. Esquivar sin chocar: las reglas que frenan

El navegador tiene varias redes de seguridad. Son parámetros en `navegador.h`, no magia:

• **Antirrebote en milisegundos:** un obstáculo lejano en los lados debe verse 150 ms seguidos antes de frenar; para volver a avanzar, el frente debe verse libre 250 ms (`NAV_CERCANIA_MIN_DETENER`, `MS_CONFIRMAR_LEJOS`, `MS_CONFIRMAR_LIBRE`).

• **Cercanía:** el detector devuelve 0 a 3. Con 2 o 3 se frena; con 1 solo se informa, salvo que se active «frenar desde lejos en el centro» (`navegador_poner_frenar_lejos_centro`).

• **Hueco de video:** si la cámara deja de mandar cuadros más de 700 ms, el carro se detiene y queda en `ERROR`; no reanuda solo (`NAV_MAX_HUECO_VIDEO_MS`). Antes eran 500 ms y un hueco normal de 503 ms detuvo una ruta real (`navegador.h:71`).

• **Retroceso ciego limitado:** el carro no tiene sensor atrás. Retrocede como máximo 15 cm y 2000 ms, y solo debe usarse con espacio libre detrás (`NAV_RETROCESO_CM`, `NAV_RETROCESO_MAX_MS`).

• **Frenos anti-locura:** un giro planeado de más de 150° que no está en la ruta, un índice que retrocede, más de 2.5 veces la distancia de la ruta o una desviación mayor de 1 m hacen parar y pedir ayuda (`navegador.h:176-180`).

• **Sondeo de obstáculo fijo:** al arrancar no se decide con el primer cuadro; se piden 4 cuadros seguidos. Si el obstáculo ya estaba y el centro está libre, avanza 4 cm y comprueba si crece: si no crece, era un artefacto fijo y se ignora en esa corrida (`NAV_ARRANQUE_CUADROS`, `NAV_SONDEO_CM`).

7. Calibración: por qué el carro navega por tiempo

`CalibracionCarro` guarda (`calibracion.h:52-74`):

• `arranque_ms`: lo que tarda el carro en empezar a moverse.

• `ms_por_cm_x10`: milisegundos por centímetro, multiplicados por 10.

• `frenado_mm`: lo que sigue avanzando después de `STOP`.

• `ms_por_grado_izq_x10` y `..._der_x10`: el giro no es simétrico porque cada pivote usa un motor distinto.

• `frenado_grados_izq_x10` y `..._der_x10`: la inercia del giro.

• `fov_grados_x10`: campo de visión de la cámara, para el giro visual.

• `ms_por_cm_atras_x10`: retroceso, opcional.

• El desplazamiento del centro al girar (en micrómetros por grado), porque el chasis no gira sobre un punto fijo.

La app mide estos valores con pruebas guiadas y los guarda. `cal_es_valida` rechaza rangos absurdos y `cal_validar_para_guardar` exige que el giro quede entre 3 y 20 ms/grado (`calibracion.h:77-83`, `112-117`). La explicación práctica está en `docs/calibracion.md` y en `docs/calibracion_guia.md`.

8. El corredor y la visión

Tres ideas se combinan para no frenar por cualquier cosa:

1. **Detector de color (`obstaculos.c`):** aprende el color del piso en una franja de abajo y marca como obstáculo las celdas que no se parecen. Es rápido (~7 ms) y ve cualquier objeto, aunque no sepa qué es.

2. **Modelo de objetos (`fusion.c` + `cualquiera.c`):** EfficientDet-Lite0 reconoce personas, sillas, botellas... pero tarda ~45 ms y a veces falla. La fusión **nunca baja** lo que ve el detector de color: si el color ve algo cerca, manda el color (`fusion.h:1-20`).
3. **Corredor (`corredor.c`):** en vez de decidir con tres tercios fijos de la imagen, calcula la franja por la que el carro va a pasar, con la perspectiva de la cámara. Un objeto fuera del corredor solo avisa; uno dentro frena.

El detalle de umbrales y resultados está en `docs/obstaculos.md`, `docs/fusion.md` y `docs/giro_visual.md`.

9. El simulador

`simulador_navegador.c` hace de carro y de cámara a la vez:

• recibe el comando que devuelve `navegador_tick` y lo ejecuta durante 50 ms con la calibración;

• puede simular que el carro avanza o gira un porcentaje distinto del planeado, para ver la deriva;

• genera los obstáculos que vería la cámara según eventos programados;

• cuenta las **violaciones de seguridad**: veces que el navegador mandó movimiento en `ESPERANDO`, `TERMINADO` o `ERROR`. Debe ser siempre cero.

`sim_ruta.c` envuelve ese simulador para probar una ruta completa y devolver el recorrido. Es lo que usa la app para «probar la ruta sin el carro» y lo que se usó en J15 para las 400 rutas B6. **No es una medición física**: no reproduce el deslizamiento real, ni el piso, ni la batería.

10. Pruebas y cobertura

La suite del núcleo son **21 programas y 1598 casos** en el último registro (`README.md`, J15). Incluye trayectoria, calibración, obstáculos, corredor, fusión, máscara, giro visual, movimiento de piso, navegador, reincorporación J15, editor de rutas, sesión compartida y los simuladores. El comando es:

```
cd nucleo_c
make test
```

No se volvieron a ejecutar durante el cierre documental por indicación del dueño (`docs/entrega.md` §5). Los casos concretos y qué cubre cada uno están en `docs/referencia/pruebas.md` y `docs/codigo/nucleo_c.md`.

11. Límites que conviene decir en voz alta

• La distancia es **tiempo disfrazado de distancia**. Sin encoders, una rueda que patina no se nota.

• El giroscopio corrige el rumbo, no el deslizamiento lateral puro (`docs/entrega.md` §6).

• La cámara tarda ~250 ms en QVGA/HQVGA; a 38 cm/s eso son varios centímetros recorridos antes de que el STOP llegue.

• El detector de color confunde sombras, brillos y pisos de dos colores; por eso existen la máscara, el corredor y el sondeo.

• El modelo de objetos no se entrenó con el piso ni los objetos del proyecto; reconoce clases generales y a veces inventa (`docs/modelo_objetos.md`).

• La simulación B6 no demuestra el resultado físico; el criterio 5/6 sigue **NO MEDIDO** (`docs/reporte_J15.md`).

12. Para seguir leyendo

• `docs/codigo/nucleo_c.md`: recorrido por archivo y función.

• `docs/navegador.md`: referencia histórica muy detallada de estados y parámetros.

• `docs/obstaculos.md`, `docs/fusion.md`, `docs/giro_visual.md`: visión.

• `docs/calibracion.md` y `docs/calibracion_guia.md`: cómo se miden los valores.

• `docs/referencia/simulador.md`: modelo y límites de la simulación.

9. Aplicación móvil Android

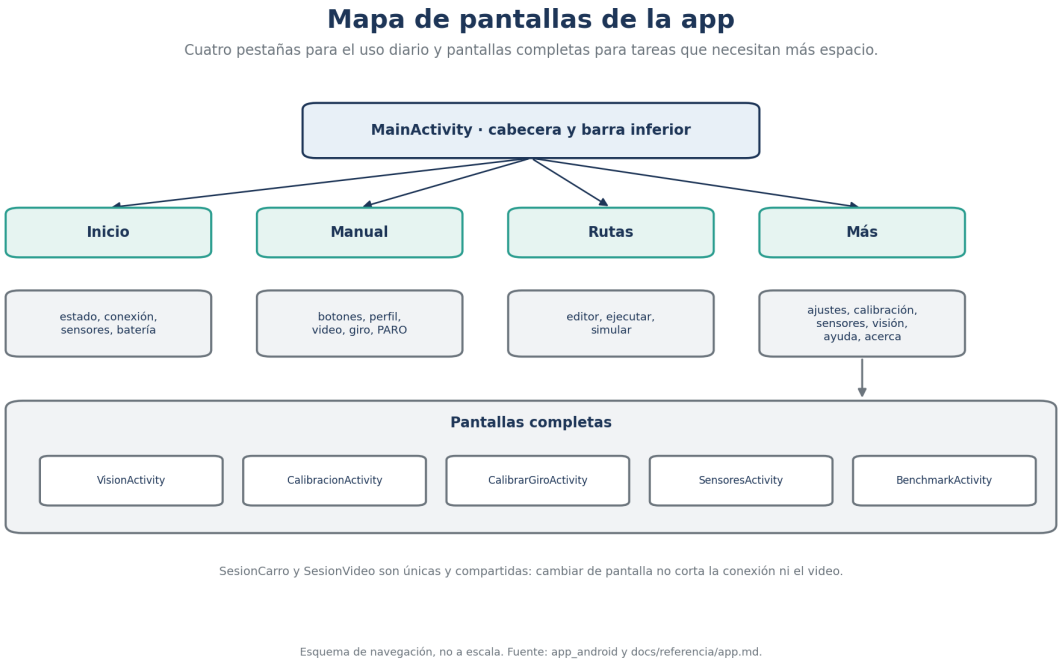
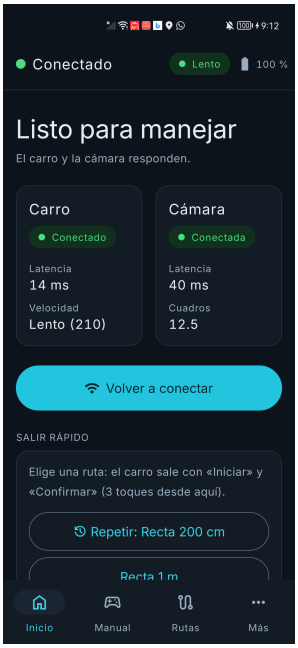


Figura 8: Mapa de pantallas de la app. Cuatro pestañas principales y cinco pantallas completas que comparten la conexión y el video. (Esquema de navegación, no a escala.)



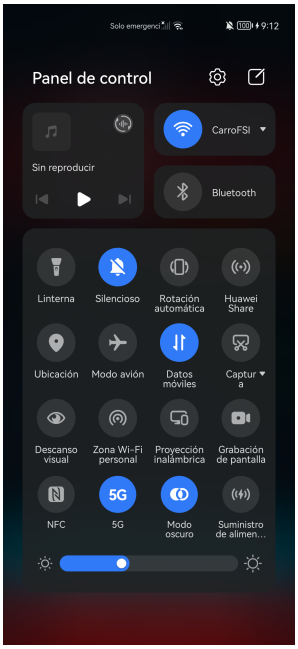
Fotografía: *Pestaña Inicio de la app. Archivo: foto_10_app_conexion.png*

[FOTO PENDIENTE: FOTO_11] — Pestaña Control Manual

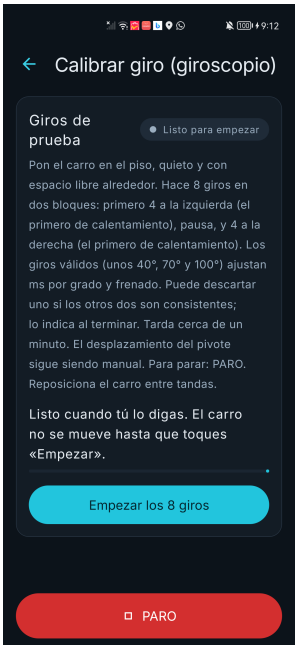
Encuadre requerido: Cruceta direccional, visor de video MJPEG, EBRAKE y perfil
Archivo esperado: docs/imagenes/foto_11_app_manual_video.png



Fotografía: *Pestaña Rutas. Archivo: foto_12_app_editor_rutas.png*



Fotografía: *Pantalla CalibracionActivity. Archivo: foto_14_app_calibracion_recta.png*



Fotografía: *Pantalla CalibrarGiroActivity. Archivo: foto_15_app_calibracion_giro.png*



Fotografía: *Pantalla Ajustes / Red. Archivo: foto_17_app_ajustes_red.png*

Referencia de la aplicación Android

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Código documentado: `app_android/` en `deae8b6`. **Fuentes:** `AndroidManifest.xml`, `app/build.gradle.kts`, `MainActivity.kt`, `ControlLogica.kt`, `Enlace.kt`, `SesionCarro.kt`, `SesionVideo.kt`, `ModoSemiautonomo.kt`, `CamaraObstaculos.kt`, `DetectorTflite.kt`, `app/src/main/cpp/CMakeLists.txt`, `docs/entrega.md` y `docs/pruebas_control.md`.

1. En una frase

La app es el **mando y los ojos** del carro: se conecta por WiFi al WebSocket, manda órdenes, muestra la cámara, dibuja rutas, calibra el movimiento y ejecuta el modo semiautónomo.

Está hecha con vistas clásicas de Android (Kotlin, sin Compose), funciona en un Huawei sin servicios de Google y guarda el trabajo pesado (visión, navegación, calibración) en el código C del núcleo, que se compila dentro de la app por JNI. Así el mismo código que se prueba en el PC es el que corre en el teléfono.

2. Cómo está organizada

• **Una actividad principal** (`MainActivity`) con barra inferior de cuatro pestañas: **Inicio**, **Manual**, **Rutas** y **Más**. Cada pestaña es un `Fragment`. Para quien no viene de Android: una **actividad** es una pantalla completa, y un **Fragment** es una parte de pantalla que se puede cambiar sin cerrar la actividad.

• **Pantallas completas aparte:** `VisionActivity` (cámara y detector), `CalibracionActivity`, `CalibrarGiroActivity`, `SensoresActivity` y `BenchmarkActivity`.

• **Una sola conexión con el carro** para toda la app (`SesionCarro`), compartida por las pantallas.

• **Un solo video de la cámara del carro** (`SesionVideo`), compartido por Inicio, Manual, Visión y Ejecutar ruta.

• **Un solo ciclo semiautónomo** (`ModoSemiautonomo`) que llama al navegador en C cada 50 ms.

La conexión y el video se abren cuando entra la primera pantalla que los usa y se cierran 1,5 s después de que sale la última. Esa espera evita cortar el video al cambiar de pestaña o al girar el teléfono (`SesionCarro.kt:4-7`; `SesionVideo.kt:5-11`; la lógica está en `nucleo_c/sesion_compartida.c`).

3. Las pantallas

Pantalla	Para qué sirve	Archivo principal
Inicio	Estado del carro, conexión, sensores, batería, accesos rápidos	`InicioFragment.kt`
Manual	Botones de movimiento, perfil, video, giro por ángulo, PARO	`ManualFragment.kt`
Rutas	Dibujar/abrir una ruta, verla en la cuadrícula, ejecutarla o simularla	`RutasFragment.kt`, `EjecutarRutaFragment.kt`, `EditorRutaView.kt`
Más	Ajustes, calibración, sensores, visión, ayuda, acerca de	`MasFragment.kt`
Visión	Cámara del teléfono + detector de obstáculos con umbrales	`VisionActivity.kt`
Calibración	Pruebas guiadas de arranque, avance, frenado y giro	`CalibracionActivity.kt`
Calibrar giro	Ocho giros medidos con el giroscopio	`CalibrarGiroActivity.kt`
Sensores	Estado de MPU6050 y VL53L0X, interruptores y prueba de giro	`SensoresActivity.kt`
Benchmark	Compara CPU/GPU/NNAPI del detector de objetos	`BenchmarkActivity.kt`

En **Manual** y **Rutas** la app mantiene la pantalla encendida mientras el carro se mueve (`MainActivity.kt:12`, `336`). El botón de **PARO** de la cabecera hace `STOP` y cancela la ruta (`SesionCarro.pararTodo`, `SesionCarro.kt:97-104`). Si hay una ruta en marcha y se intenta cambiar de pestaña, la app pide confirmación y la detiene (`MainActivity.kt:8`, `193-203`).

4. La conexión con el carro

`ControlLogica` es el corazón del enlace (**`ControlLogica.kt`**):

- • Abre un WebSocket a la dirección configurada (por defecto **`ws://192.168.4.1/ws`**).
- • Manda **`PING`** **cada 800 ms** para mantener viva la conexión y medir la latencia con el **`PONG`** (**`PING_MS`**, línea 43).
- • Repite el comando de movimiento cada 250 ms mientras el botón está apretado. El firmware corta a los 1000 ms sin comandos reales; este heartbeat es el que mantiene el movimiento.
- • Al soltar, manda **`STOP`** de inmediato.
- • Cada 4 **`PING`** (3,2 s) vuelve a pedir el perfil deseado, por si el carro volvió a AUTO por el watchdog (**`ControlLogica.kt:194-206`**).
- • Espera **`ACK_MANUAL`**/**`ACK_AUTO`** antes de dar por confirmado el perfil; las rutas no arrancan sin esa confirmación (**`ControlLogica.kt:16-18`**, **`301-327`**).
- • Interpreta **`SENS`**, **`GIRO_OK/TIMEOUT/ERROR`**, **`TOF_PARO`**, **`CAPS`**, **`IMU`**, **`TOF`**, **`ANT`**, **`ARR`** y los ACK de balance.

`EnlaceOkHttp` implementa el WebSocket con OkHttp (**`Enlace.kt`**). Usa **`readTimeout = 0`** porque el carro real puede estar callado sin que la conexión esté muerta (**`Enlace.kt:43-48`**). Cada conexión lleva un número y los avisos de una conexión vieja se descartan (**`Enlace.kt:94-101`**). Si el teléfono pierde la red, la app reintenta cada 2 s (**`SES_REINTENTO_MS`**).

5. El modo semiautónomo

`ModoSemiautonomo` (**`ModoSemiautonomo.kt`**) es el puente entre la app y **`navegador.c`**:

1. Carga la ruta y la calibración en el navegador nativo.
2. Cada **50 ms** arma el resultado de obstáculos, llama a **`NavegadorNativa.tick()`** y manda el comando si cambió.
3. Los obstáculos pueden venir de la cámara del teléfono (**`CamaraObstaculos`**) o de interruptores simulados para probar sin cámara.
4. Si no llegan cuadros nuevos en **1000 ms**, cancela la ruta: sin ojos no se avanza (**`TIMEOUT_CAMARA_MS`**).
5. Antes de arrancar exige que el carro confirme el perfil AUTO (**`PERFIL_ESPERA_MS`**).
6. Si se pierde la conexión, cancela la ruta.

La app **no decide** cuándo girar ni cuánto avanzar: eso lo hace el C. Kotlin solo alimenta el navegador, ejecuta el comando y dibuja el estado. Esa separación permite probar la misma lógica en el PC.

6. Visión: cámara del teléfono y detector

Hay dos fuentes de imagen:

- • **Cámara del teléfono** (**`CamaraObstaculos.kt`**): usa Camera2, elige el tamaño 4:3 más pequeño de al menos 320×240, convierte cada cuadro a RGB y llama al detector en C. Procesa en un hilo aparte y descarta cuadros viejos si llega más rápido de lo que puede.
- • **Cámara del carro** (**`VideoCarro.kt`** + **`SesionVideo.kt`**): recibe JPEG, video crudo o el modo CELDAS por HTTP/UDP desde la Freenove.

El **detector de color** (**`ObstaculosNativa`** → **`obstaculos.c`**) divide la imagen en celdas de 10×10 y marca lo que no se parece al piso. El **modelo de objetos** (**`DetectorTflite.kt`**) usa EfficientDet-Lite0 en TensorFlow Lite, con CPU (el procesador normal), GPU (el chip gráfico) o NNAPI (la API de Android para usar los aceleradores del teléfono). La **fusión** (**`FusionNativa`** → **`fusion.c`**) combina ambos: el color manda en seguridad y el modelo solo puede subir la severidad, nunca bajarla (**`fusion.h:1-20`**).

El modelo está en **`app/src/main/assets/efficientdet_lite0.tflite`** (4.5 MB) y se puede reemplazar por otro archivo en la carpeta privada de la app (**`DetectorTflite.kt:49-50`**). Los resultados del benchmark en el Huawei están en **`docs/benchmark_ELS-NX9.md`**.

7. El puente JNI

`app/src/main/cpp/puente.c` es un archivo grande (≈143 KB) que traduce entre Kotlin y C. Es lo que se llama **JNI** (Java Native Interface): el mecanismo de Android para llamar código nativo escrito en C. No contiene la lógica: convierte arreglos, llama a las funciones del núcleo y devuelve resultados.

Ejemplos de la correspondencia:

Kotlin	C (`puente.c`)	Núcleo
`TrayectoriaNativa.calcular`	`Java_com_fsi_carro_TrayectoriaNativa_calcular`	`trayectoria_calcular_desde_cal`
`ObstaculosNativa.analizarConfig`	`..._ObstaculosNativa_analizarConfig`	`obstaculos_analizar_config`
`NavegadorNativa.tick`	`..._NavegadorNativa_tick`	`navegador_tick`
`FusionNativa.decidir`	`..._FusionNativa_decidir`	`fusion_decidir`
`SesionNativa.entrar`	`..._SesionNativa_entrar`	`sesion_compartida_entrar`

El `CMakeLists.txt` compila `puente.c` junto con los `.c` de `nucleo_c/`, **sin copiarlos** (`CMakeLists.txt:22-46`). El resultado es `libcarro_nativo.so`. Solo se compila para `arm64-v8a` para no inflar el APK (`build.gradle.kts:50-52`).

8. Compilar e instalar

Requisitos: Android Studio con SDK, NDK 30.0.16248370, CMake 4.1.2 y Java 17 (`build.gradle.kts:35-37`, `73-76`).

```
cd app_android
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat assembleDebug
```

El APK sale en `app_android/app/build/outputs/apk/debug/`. La versión se genera sola con el hash de git y el nombre `0.4` (`build.gradle.kts:15-31`): la app muestra `APP:0.4-<hash>`. La versión publicada en J15 es **APP:0.4-074c9fd**, `versionCode` 3, 14 852 386 bytes, SHA-256 `36CE38F2...9513BA` (`docs/entrega.md` §1).

Instalación (no se ejecutó en el cierre documental):

```
adb install -r dist\carro_fsi_0.4-074c9fd.apk
```

La app pide **INTERNET**, **ACCESS_NETWORK_STATE**, **VIBRATE** y **CAMERA**; no usa servicios de Google Play (`AndroidManifest.xml:9-25`). `usesCleartextTraffic` está activo porque el WebSocket es `ws://` sin cifrar (`AndroidManifest.xml:30-31`).

9. Lo que se probó y lo que no

Afirmación	Estado	Evidencia
La app compila y el APK se instala en el Huawei	VERIFICADO en J15	`docs/entrega.md` §3
La APK publicada coincide con el hash	VERIFICADO	`docs/entrega.md` §1-2
El control por WebSocket responde y el watchdog frena	VERIFICADO en J13/J14; probado también con servidor falso	`docs/pruebas_control.md`, `docs/reporte_J14.md`
El detector de obstáculos funciona con la cámara del teléfono	VERIFICADO en pruebas de visión	`docs/vision_huawei.md`, `docs/obstaculos.md`
El modo semiautónomo completo esquiva en el piso real	PARCIAL: B4-B6 físicas; J15 NO MEDIDO	`docs/reporte_J14.md`, `docs/reporte_J15.md`

El benchmark del modelo en el Huawei	VERIFICADO con las cifras de `docs/benchmark_ELS-NX9.md`	Datos en `docs/datos_benchmark/`
La app funciona en otros teléfonos	NO MEDIDO	Solo se probó el Huawei ELS-NX9

10. Límites y cuidados

• **Un solo cliente WebSocket:** si otra app o el navegador se conecta, el firmware cierra el anterior. No usar la página web y la app a la vez.

• **Un solo cliente de video:** la cámara atiende a uno; la app lo comparte entre pantallas, pero no con otro dispositivo.

• **El video no es imprescindible para manejar:** el control va por otro WebSocket; si el video falla, el botón de PARO sigue funcionando.

• **La cámara del teléfono no es la del carro:** en `VisionActivity` se usa la trasera del teléfono; en el modo semiautónomo la fuente puede ser la del teléfono o la del carro, según la pantalla.

• **La app no guarda la contraseña del WiFi del carro**; la red se elige en Ajustes de Android.

• **No se deben borrar los datos de la app** durante las pruebas: ahí viven la calibración y los balances (`docs/entrega.md` §3).

• **El pin de la app** y las direcciones de red se configuran en Ajustes; no se documentan valores reales en este repositorio público.

11. Para seguir leyendo

• `docs/codigo/app_android.md`: recorrido por archivo y función.

• `docs/protocolo.md`: mensajes y ACK que usa la app.

• `docs/operacion.md`: cómo conectar y manejar el carro.

• `docs/calibracion_guia.md`: cómo se calibra desde la app.

• `docs/pruebas_control.md`: pruebas del control y del servidor falso.

• `docs/instalar_app_huawei.md`: instalación en el teléfono del proyecto.

10. Visión por computadora y percepción

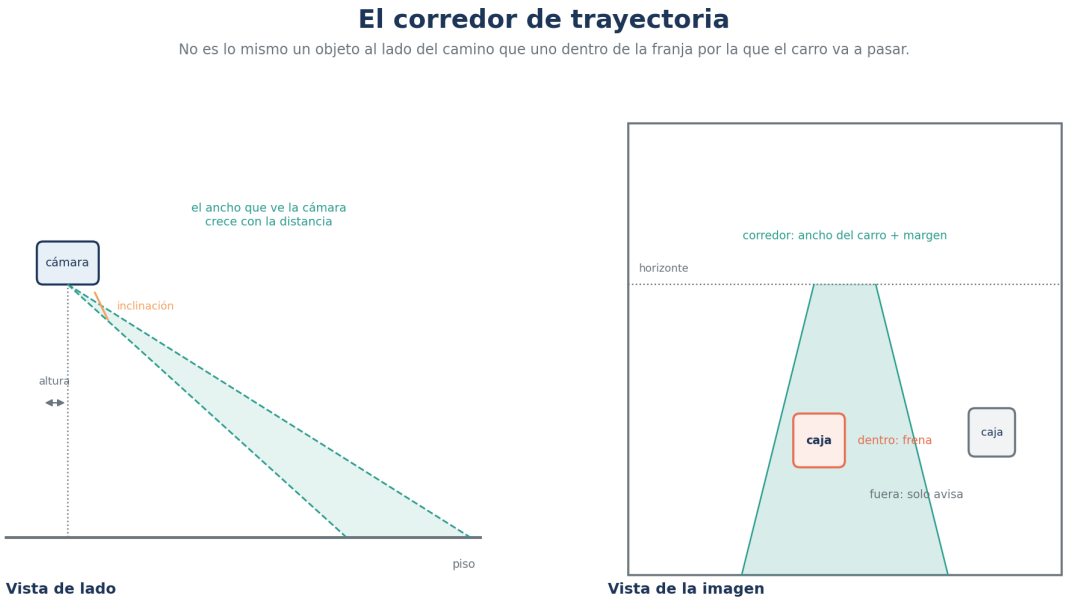


Figura 7: El corredor de trayectoria. La franja de la imagen por la que el carro va a pasar se estrecha con la distancia; un objeto fuera de ella solo avisa. (Esquema geométrico, no a escala.)

[FOTO PENDIENTE: FOTO_13] — Modo semiautónomo asistido

Encuadre requerido: Carro avanzando sobre ruta con detección de obstáculos activa
Archivo esperado: docs/imagenes/foto_13_app_semiautonomo.png

[FOTO PENDIENTE: FOTO_16] — Pantalla VisionActivity

Encuadre requerido: Corredor trapezoidal superpuesto, máscara de piso y cuadro TFLite
Archivo esperado: docs/imagenes/foto_16_app_vision_deteccion.png

Recorrido de código: Corredor de seguridad trapezoidal (`corredor.c`)

Ubicación: `nucleo_c/corredor.h` y `nucleo_c/corredor.c`

Lenguaje: C estándar (C99)

Propósito: Proyectar sobre el plano bidimensional de la imagen de la cámara la franja geométrica de paso del vehículo en función de la altura e inclinación óptica.

---

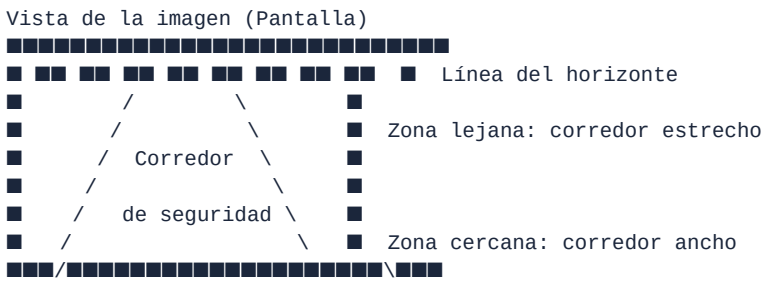
1. Fundamento geométrico

Una cámara orientada hacia adelante con cierta inclinación hacia abajo observa el suelo bajo una proyección en perspectiva:

• Los objetos cercanos aparecen en la parte inferior de la imagen con una escala aparente mayor.

• Los objetos lejanos aparecen en la parte superior (cerca de la línea del horizonte) con una escala menor.

• Un carril recto de ancho constante en el mundo físico (el ancho del carro más sus márgenes de seguridad) no se proyecta como un rectángulo en la pantalla, sino como un **trapecio que se estrecha hacia la parte superior**.



---

2. Parámetros y estructura `Corredor`

Definida en **`nucleo_c/corredor.h`**:

• **`ancho_carro_mm`**: Ancho físico entre las caras exteriores de las ruedas (~180 mm).

• **`margen_lateral_mm`**: Margen de tolerancia a cada lado para evitar rozar obstáculos (~60 mm).

• **`altura_camara_mm`**: Altura del lente de la cámara sobre el nivel del suelo.

• **`inclinacion_grados`**: Ángulo de cabeceo (*pitch*) del eje óptico hacia abajo respecto a la horizontal.

• **`limite_cercano_mm`** y **`limite_lejano_mm`**: Rango de distancias en el que el vehículo evalúa obstáculos (típicamente desde 15 cm hasta 120 cm).

---

3. Funciones principales

3.1 `corredor_iniciar`

Calcula analíticamente los cuatro vértices del trapecio en coordenadas de píxeles: 1. Emplea las ecuaciones de la cámara estenopeica (*pinhole camera model*). 2. Intersecta los rayos ópticos que pasan por los bordes laterales del carro en el suelo con el plano del sensor. 3. Almacena las pendientes de las dos aristas oblicuas del trapecio (lado izquierdo y lado derecho).

3.2 `corredor_contiene_punto`

`nucleo_c/corredor.c:110-145`:

• Recibe las coordenadas de un píxel (x, y) o el centro de un bloque de celdas.

• Evalúa si el punto se ubica verticalmente entre la línea del parachoques y el límite lejano.

• Comprueba si su coordenada horizontal se encuentra comprendida entre el límite izquierdo y derecho del trapecio para esa fila de escaneo específica:

$$X_{\text{izq}}(y) \le x \le X_{\text{der}}(y)$$

• **Retorno:** Devuelve **`1`** si el punto cae dentro de la franja de peligro, o **`0`** si está fuera.

---

4. Importancia en la navegación reactiva

El corredor es la herramienta fundamental para evitar frenados erráticos:

• Una caja o pared situada a 40 cm a la derecha del vehículo es vista claramente por la cámara, pero cae fuera del trapecio proyectado.

• El sistema sabe que esa caja **no obstruye la trayectoria directa del chasis**, permitiendo al carro continuar a velocidad crucero sin detenerse falsamente.

• Solo los objetos que invaden el interior del trapecio activan la secuencia de freno preventivo (**`NAV_DETENIDO_OBSTACULO`**).

---

5. Pruebas unitarias asociadas

• **`nucleo_c/pruebas_corredor.c`**: Evalúa la proyección del trapecio ante distintas alturas e inclinaciones de cámara y comprueba la pertenencia de puntos de prueba en los límites interior, exterior y bordes.

Recorrido de código: Detección de obstáculos (`obstaculos.c`, `mascara_vision.c` y `pisos_sinteticos.c`)

Ubicación: **`nucleo_c/obstaculos.h`**, **`nucleo_c/obstaculos.c`**, **`nucleo_c/mascara_vision.h`**, **`nucleo_c/mascara_vision.c`**, **`nucleo_c/pisos_sinteticos.h`**, **`nucleo_c/pisos_sinteticos.c`**

Lenguaje: C estándar (C99)

Propósito: Segmentar el piso visual, identificar anomalías cromáticas o de profundidad en la trayectoria de avance y confirmar la presencia de obstáculos para la toma de decisiones evasivas.

---

1. Principio de detección por color de piso

En entornos interiores homogéneos (aulas, laboratorios, pasillos), el suelo presenta una tonalidad relativamente uniforme. La estrategia de visión implementada parte de la hipótesis del plano de apoyo: 1. Se aprende el color promedio de referencia del suelo del recinto $(R_{\text{ref}}, G_{\text{ref}}, B_{\text{ref}})$. 2. Cualquier objeto elevado (una caja de cartón, un zapato, una pared) que se interponga en el campo visual reflejará la luz con una intensidad y crominancia distinta a la del suelo. 3. Aquellas regiones que se desvíen de la tolerancia del piso son marcadas como obstáculos potenciales.

---

2. Estructura `ResultadoObstaculos`

Definida en **`nucleo_c/obstaculos.h`**:

```
typedef struct {
    int obstaculo_confirmado;   /* 1 si el obstáculo ha persistido en varios cuadros consecutiv
    int distancia_frente_mm;    /* Distancia estimada al obstáculo más cercano al frente */
    int celdas_bloqueadas;      /* Conteo de celdas anómalas dentro del corredor */
    int lado_libre;             /* Lado recomendado para esquivar: -1 = izquierda, +1 = derecha
    int margen_izq_cm;          /* Espacio libre estimado a la izquierda del obstáculo en cm */
    int margen_der_cm;          /* Espacio libre estimado a la derecha del obstáculo en cm */
} ResultadoObstaculos;
```

---

3. Funciones y algoritmos principales

3.1 `mascara_vision.c`: Segmentación cromática y filtrado

Calcula la distancia de color Euclídea o por Manhattan entre el píxel observado y el piso aprendido: $$\Delta C = |R - R_{\text{ref}}| + |G - G_{\text{ref}}| + |B - B_{\text{ref}}|$$

• Si $\Delta C > \text{UMBRAL_COLOR}$, el bloque se clasifica como *no-piso*.

• Incluye filtrado morfológico de ruido: elimina píxeles anómalos aislados para evitar falsos positivos producidos por brillo de lámparas o reflejos del piso encerado.

3.2 `obstaculos.c`: Análisis espacial y confirmación temporal

• **Intersección con el corredor:** Filtra los bloques no-piso; solo aquellos ubicados dentro del trapecio de trayectoria del carro se consideran peligrosos.

• **Ventana de persistencia temporal:** Para evitar frenados bruscos ante caídas de fotogramas o sombras pasajeras, un obstáculo debe detectarse en al menos 2 o 3 cuadros sucesivos (**`CONFIRMAR_MUESTRAS`**) antes de activar la bandera **`obstaculo_confirmado`**.

• **Evaluación de laterales:** Suma las celdas despejadas a la izquierda y derecha de la caja para recomendar el sentido de giro evasivo con mayor margen de maniobra.

3.3 `pisos_sinteticos.c`: Generador de entornos virtuales de prueba

Dado que no siempre se cuenta con la cámara física montada durante el desarrollo en PC, este módulo genera matrices sintéticas con patrones realistas:

• Pisos de baldosas con líneas de junta oscuras.

• Pisos de madera con vetas de variación cromática.

• Cajas tridimensionales proyectadas con caras iluminadas y sombras proyectadas.

Permite someter los algoritmos de detección a miles de variaciones de contraste de manera automática en las pruebas unitarias.

---

4. Pruebas unitarias asociadas

• **`nucleo_c/pruebas_obstaculos.c`**: Comprueba detección frontal, detección lateral y límites de la matriz.

• **`nucleo_c/pruebas_mascara.c`**: Valida el clasificador binario de color ante variaciones de iluminación.

• **`nucleo_c/pruebas_pisos_dificiles.c`**: Prueba la robustez del detector ante pisos con texturas complejas y sombras.

11. Compilación, instalación y verificación de versiones

Referencia de compilación

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Fuentes: `README.md`, `nucleo_c/Makefile`, `pruebas_firmware/Makefile`, `app_android/app/build.gradle.kts`, `app_android/app/src/main/cpp/CMakeLists.txt`, `scripts/cargar.ps1`, `scripts/verificar_version.py`, `docs/hardware_cargar.md` y `docs/entrega.md`.

1. Qué se compila y con qué

Compilar es traducir el código que escribe una persona a un archivo que la placa o el teléfono pueden ejecutar. El conjunto de programas que hacen esa traducción se llama **toolchain**. El proyecto tiene cuatro productos y cada uno usa una herramienta distinta:

Producto	Herramienta	Resultado
Firmware del carro	`arduino-cli` + core `esp32:esp32` 3.3.12	`.bin` para la DevKit
Firmware de la cámara	`arduino-cli` + core `esp32:esp32` 3.3.12	`.bin` para la Freenove
Núcleo y pruebas de PC	`gcc` + `make`	21 ejecutables de prueba
App Android	Gradle + Android SDK/NDK + CMake	`.apk`

En palabras llanas: `arduino-cli` compila para las placas; `gcc` y `make` compilan para el PC; **Gradle** empaqueta la app de Android; el **SDK** trae las herramientas y bibliotecas de Android; el **NDK** permite compilar C para el teléfono; y **CMake** organiza esa compilación nativa.

La regla del proyecto es que **el código C se compile sin copiarlo**: la app toma los `.c` de `nucleo_c/` desde su carpeta original, y las pruebas de firmware toman `carro.c` desde `firmware_carro/`. Así no hay dos versiones de la misma lógica.

2. Firmware del carro

Requisitos

• `arduino-cli` en el `PATH`.

• Core `esp32:esp32` versión **3.3.12**.

• Librerías: **Async TCP 3.5.0**, **ESP Async WebServer 3.12.0** y **VL53L0X de Pololu 1.3.1** (`docs/pendientes_firmware.md:39`; `firmware_carro.ino:17-24`).

• `firmware_carro/secretos.h` con el nombre y la contraseña de la red. Si no existe, se usa `secretos.ejemplo.h` y el proyecto compila con valores de ejemplo (`config.h:20-36`).

Compilar

```
arduino-cli compile --fqbn esp32:esp32:esp32 firmware_carro
```

Cargar

```
.\scripts\cargar.ps1 nuevo -Puerto COM6
```

`cargar.ps1` compila en una carpeta temporal, obtiene el hash corto del commit, escribe `version_compilada.h` con `FW:<hash>-<aaaammdd>` y sube el binario (`scripts/cargar.ps1:163-173`). También reintenta a 921600, 512000 y 115200 baudios si la primera subida falla (`docs/hardware_cargar.md` §«Uso»).

Verificar sin cargar

```
.\scripts\cargar.ps1 nuevo -SoloCompilar
```

En J14/J15 el firmware compiló al **73-74 % de la flash** (`docs/hardware_cargar.md` §«Qué se probó»; `docs/camara_prueba.md` §«Cargar»). El binario conservado en J15 es `FW:453ed20-20261027`, 1 031 712 bytes, SHA-256 `9FD1BB51...274048E` (`docs/entrega.md` §4). **No se publica** porque contiene la contraseña del WiFi.

3. Firmware de la cámara

Requisitos

• Core `esp32:esp32` 3.3.12.

• Placa **ESP32 Wrover Module** (`esp32:esp32:esp32wrover`, con PSRAM).

• `camara_prueba/secretos.h` (ignorado por git).

Compilar y cargar

```
.\scripts\cargar.ps1 camara -SoloCompilar
.\scripts\cargar.ps1 camara -Puerto COM8
```

`cargar.ps1` copia el sketch a una carpeta temporal y, si existen, los archivos `obstaculos.c/h` y `paquete_celdas.c/h` de `nucleo_c/` para el modo CELDAS (`scripts/cargar.ps1:176-190`). Sin ellos, el sketch compila solo con JPEG y RAW.

La cámara ocupa ~**83 % de la flash** (`docs/camara_prueba.md` §«Cargar»). El `COMPRESOR_JPEGENC` está activo para comparar compresores, pero el compresor por defecto sigue siendo `frame2jpg` (`camara_prueba.ino:51-54`).

4. Núcleo y pruebas de PC

Requisitos

• `gcc` con soporte C99.

• `make`.

• En Windows, un entorno que tenga ambos (MSYS2/MinGW o WSL). El proyecto se desarrolló y probó en este equipo con esas herramientas; la versión exacta no está registrada en el repositorio (**PENDIENTE** de anotar).

Compilar y ejecutar todo

```
cd nucleo_c
make test
```

El `Makefile` compila con `-std=c99 -Wall -Wextra -Wpedantic -Werror` (`nucleo_c/Makefile:4-5`). Cualquier advertencia es error: el código debe compilar limpio. La lista de programas está en `nucleo_c/Makefile:7` y la de ejecución en `72-93`.

```
cd pruebas_firmware
make test
```

Compila `carro.c`, los sensores y `recuperacion.c` con hardware falso, ejecuta las pruebas y verifica el hash de la página web (`pruebas_firmware/Makefile:28-33`). El flag `-DARRANQUE_ESCALONADO_MS=40` compila una segunda variante para probar el arranque escalonado (`Makefile:19-20`).

Limpiar

```
make clean
```

Borra los ejecutables, no las fuentes.

Estado de las pruebas

El último registro es **21 programas / 1598 casos** en el núcleo, **447** en firmware, **46** en el servidor falso y **13** en el verificador, con cero fallos (`README.md`; `docs/entrega.md` §5). No se repitieron durante el cierre documental por indicación del dueño. Los números no se inventan: son los del reporte.

5. App Android

Requisitos

• Android Studio con SDK **compileSdk 37**, build-tools 36.0.0.

• NDK **30.0.16248370**.

• CMake **4.1.2**.

• JDK 17 (**app_android/app/build.gradle.kts:35-37**, **73-76**).

Compilar

```
cd app_android
$env:JAVA_HOME = "C:\Program Files\Android\Android Studio\jbr"
.\gradlew.bat assembleDebug
```

El APK queda en **app_android/app/build/outputs/apk/debug/**. La versión se genera con el hash de git y el nombre **0.4** (**build.gradle.kts:15-31**).

Qué compila el NDK

CMakeLists.txt genera **libcarro_nativo.so** con **puente.c** y estos módulos de **nucleo_c/**: trayectoria, obstáculos, máscara, navegador, calibración, calgiro, fusión, movimiento visual, video stats, prueba de obstáculo, corredor, cualquiera, movimiento de piso, paquete de celdas, sesión compartida, latencia de video, editor de rutas, simulación y simulador (**CMakeLists.txt:25-46**). Solo se compila **arm64-v8a** (**build.gradle.kts:50-52**).

Instalar

```
adb install -r dist\carro_fsi_0.4-074c9fd.apk
```

La app instalada en J15 se verificó por hash y se conservó sin borrar datos (**docs/entrega.md** §3). **No se reinstaló durante el cierre documental.**

6. Verificación de versiones

scripts/verificar_version.py compara el hash del firmware y de la APK con Git. Es de solo lectura: no instala, no mueve y no escribe en la placa.

```
pruebas_firmware\.venv\Scripts\python.exe scripts\verificar_version.py --solo-apk --apk dist\ca
```

El verificador tolera commits posteriores de documentación, entrega y del propio verificador, pero no cambios de producto (**scripts/verificar_version.py:42-60**). El resultado de J15 fue **PASA** para la APK y **pendiente** para el firmware (**docs/entrega.md** §3).

pruebas_firmware\.venv existe para no instalar paquetes en el Python del sistema. Si no existe, se crea con:

```
python -m venv pruebas_firmware\.venv
pruebas_firmware\.venv\Scripts\pip install -r pruebas_firmware\requirements.txt
```

(El archivo de requisitos se consulta en el repositorio; si no existe, el verificador solo usa la biblioteca estándar.)

7. Herramientas de documentación

El manual se genera con **Python 3.9.13** y estas bibliotecas:

• **matplotlib** 3.9.4 para las figuras.

• **reportlab** 4.5.1 para el PDF.

• **PyMuPDF** 1.26.5 para renderizar y revisar el PDF página por página.

No hay **pandoc**, Graphviz ni ImageMagick en este equipo. El script del manual no depende de ellos. Los detalles están en **docs/manual/README.md**.

8. Problemas típicos

Síntoma	Causa probable	Qué hacer
`arduino-cli: no se reconoce`	No está en el `PATH`	Instalarlo y abrir una consola nueva
Falla la subida al 921600	Placa o cable	`cargar.ps1` ya reintenta; si persiste, `-EsperarBoot`
`Falta firmware_carro\secretos.h`	No se creó el archivo	Copiar `secretos.ejemplo.h` y editarlo
No aparece ningún COM	Cable de solo carga o driver ausente	Cable de datos, otro puerto, driver CP210x/CH340
`gcc` o `make` no existen	Entorno C incompleto	Instalar MSYS2/MinGW o usar WSL
La app no compila por NDK/CMake	Versión distinta a la fijada	Instalar NDK 30.0.16248370 y CMake 4.1.2
El APK supera 15 MB	Bibliotecas nativas sin comprimir	`useLegacyPackaging = true` ya está puesto (`build.gradle.kts:58-64`)
El firmware no entra en modo carga	GPIO 2 o 0 a 3.3 V	Desconectar lo que haya en esos pines y reintentar

9. Lo que no se hizo en el cierre documental

• No se compiló ni cargó firmware.

• No se compiló ni instaló la app.

• No se ejecutaron las pruebas (por indicación del dueño).

• No se tocó `original_backup/` ni la copia de OneDrive.

Por eso este documento distingue **comandos verificados por el repositorio** de **resultados históricos**. Cualquier compilación nueva debe anotar fecha, comando y salida en `docs/pendientes_documentacion.md` o en el reporte correspondiente.

12. Pruebas, calidad, integración continua y simulador

Referencia de pruebas

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Fuentes: `nucleo_c/Makefile`, `pruebas_firmware/Makefile`, `README.md`, `docs/reporte_J15.md`, `docs/reporte_J14.md`, `docs/pruebas_control.md`, `docs/validacion_sensores.md` y `.github/workflows/sanitizadores.yml`.

1. Qué se prueba y qué no

El proyecto separa tres tipos de prueba:

1. **Pruebas de PC (software puro):** compilan la lógica real y la ejecutan con hardware falso o datos sintéticos. Son rápidas, repetibles y no necesitan el carro.
2. **Pruebas con servidor falso:** simulan al carro por WebSocket para probar la app y el protocolo sin la placa.
3. **Pruebas físicas:** se hacen con el carro real, en el piso, con personas y obstáculos. Son las únicas que demuestran el comportamiento físico, y son las que menos se pueden repetir.

Este documento describe las dos primeras. Las físicas se resumen al final, con su estado real: **J15 no tuvo ninguna corrida** (`docs/reporte_J15.md` §7).

2. Números registrados

Suite	Programas	Casos	Fallos	Dónde
Núcleo en C	21	1598	0	`nucleo_c/`
Firmware del carro	4 ejecutables	447	0	`pruebas_firmware/`
Servidor falso	1	46	0	`pruebas_firmware/servidor_falso.py`
Verificador de versiones	1	13	0	`scripts/verificar_version.py`
Sanitizadores	CI	—	success	GitHub Actions, run 40613481109

Los 447 del firmware se reparten en **152** de `pruebas_carro`, **246** de `pruebas_sensores`, **15** de `pruebas_escalonado` y **34** de `pruebas_camara` (`docs/reporte_J15.md` §5). Los 1598 del núcleo conservan los 1577 anteriores y suman 21 de J15 (`pruebas_reincorporacion_j15.c`).

Estos números son del último registro, no de una ejecución nueva. No se repitieron durante el cierre documental por indicación del dueño. Cualquier persona que los repita debe anotar fecha, comando y salida.

3. Pruebas del núcleo

Se ejecutan con:

```
cd nucleo_c
make test
```

La lista completa está en `nucleo_c/Makefile:7` y la secuencia de ejecución en `72-93`. Cada programa imprime `OK` o `FALLO` por caso y un resumen al final.

Programa	Qué comprueba

`pruebas_calibracion`	Promedios, medianas, atípicos, ajuste de recta, conversión cm↔ms y validación
`pruebas_calgiro`	Ocho giros guiados, seguridad y resultado de la calibración con giroscopio
`pruebas_trayectoria`	Rutas válidas, saltos, puntos repetidos, demasiados puntos y pasos
`pruebas_obstaculos`	Detección por color, franja de referencia, zonas, cercanía y errores
`pruebas_piso`	Casos de piso con distintos colores, sombras y ruido
`pruebas_fusion`	Fusión detector/modelo y regla «el color manda»
`pruebas_movimiento`	Giro visual entre cuadros, confianza y signo
`pruebas_navegador`	Estados, avance, giros, esquive, reincorporación y seguridad (345 casos en J11)
`pruebas_video_stats`	fps, huecos y ventana de 10 s
`pruebas_prueba_obstaculo`	Cuentas de la prueba de obstáculos
`pruebas_corredor`	Geometría del corredor y cajas dentro/fuera
`pruebas_pisos_dificiles`	Pisos negros, baldosas, franjas y cambios de exposición
`pruebas_cualquiera`	Modo «cualquier objeto»: filtros y persistencia
`pruebas_movimiento_piso`	Flujo del piso contra lo esperado
`pruebas_celdas`	Paquete UDP de celdas: armar, leer y rechazar
`pruebas_sesion_compartida`	Entradas, salidas, gracia y reapertura
`pruebas_latencia_video`	Comparación con y sin video
`pruebas_ruta_editor`	Vértices, deshacer, rehacer, plantillas y texto
`pruebas_mascara`	Artefactos fijos, límites y zonas que dependen del lugar
`pruebas_sim_ruta`	Rutas simuladas con obstáculos
`pruebas_reincorporacion_j15`	21 casos de la corrección de pose de J15

El `Makefile` compila con `-Werror`: una advertencia rompe la compilación. Eso obliga a mantener el código limpio y evita que un aviso se vuelva un fallo silencioso.

4. Pruebas del firmware

Se ejecutan con:

```
cd pruebas_firmware
make test
```

Compilan `carro.c`, `imu.c`, `tof.c`, `giro.c`, `sensores.c` y `recuperacion.c` **sin modificarlos**, con hardware falso (`pruebas_firmware/Makefile:9-27`).

Ejecutable	Qué comprueba
`pruebas_carro` (152)	Comandos, motores, perfiles, balance, cola, watchdog, clientes, keep-alive, DHCP y página
`pruebas_sensores` (246)	MPU6050, VL53L0X, giro por ángulo, parada por láser, mensajes y fallos
`pruebas_escalonado` (15)	La misma lógica con `ARRANQUE_ESCALONADO_MS=40`

`pruebas_camara` (34)	Decisiones de recuperación de la cámara, incluida la vuelta de `millis()`
`verificar_pagina.py`	Hash de la página web del carro

El hardware falso (`sensores_falsos.c`) permite simular el MPU6050 y el VL53L0X, provocar errores de bus, congelar lecturas y controlar el tiempo. Así se prueban caminos que con el hardware real serían difíciles de reproducir.

5. Estrés y concurrencia

Se ejecutó una **prueba de estrés** (mandar muchísimos comandos seguidos para ver si algo se rompe) de **100 000 comandos** con dos **hilos** (dos tareas que corren a la vez dentro del mismo programa), mezclando movimientos, `STOP`, `PING`, giros y consultas. El resultado registrado es **0 STOP perdidos** y estado final correcto (`docs/reporte_J15.md` §5). Es la prueba que respalda la regla de la cola: cuando está llena, se descarta un `PING` o un comando viejo, nunca un `STOP` (`carro.c:399-429`).

Los **sanitizadores** son herramientas que añaden comprobaciones extra al programa para detectar errores de memoria o de concurrencia que, de otro modo, pasarían desapercibidos. Corren en GitHub Actions sobre el último commit con código, `074c9fd`:

- • **ThreadSanitizer** sobre el estrés con giros.
- • **AddressSanitizer + UndefinedBehaviorSanitizer** sobre sensores, carro y navegador.

El resultado fue `success` (run 40613481109). En esta PC Windows ThreadSanitizer no está disponible; por eso se usa Linux en CI.

6. Servidor falso

`pruebas_firmware/servidor_falso.py` imita al carro por WebSocket: acepta los comandos, responde `PONG`, `ACK_*`, `SENS`, `GIRO_*` y `TOF_PARO`, y permite probar la app sin la placa (`docs/pruebas_control.md`). Tiene **46 casos** registrados.

Sirve para:

- • comprobar que la app conecta, manda y recibe;
- • probar el watchdog y la reconexión;
- • simular sensores y fallos sin gastar batería ni mover el carro;
- • repetir un caso tantas veces como haga falta.

No sirve para validar la física, la cámara real ni el WiFi real. Un caso que pasa con el servidor falso puede fallar con el carro si el tiempo o la red son distintos.

7. Verificador de versiones

`scripts/verificar_version.py` tiene **13 pruebas** registradas. Comprueba:

- • que el hash del firmware leído por el puerto serie corresponda a un commit;
- • que la APK contenga el `version_app.txt` correcto;
- • que los bytes instalados en el teléfono sean idénticos a los publicados;
- • que los commits posteriores solo toquen documentación, entrega o el propio verificador.

El caso de J15 con `--solo-apk` dio **PASA** y dejó el firmware como pendiente (`docs/entrega.md` §3).

8. Pruebas físicas: lo que sí se hizo

Las pruebas físicas de J13/J14 están en `docs/reporte_J13_fisico.md` y `docs/reporte_J14.md`. Las principales:

- • **B0-B3:** conexión, mandos, watchdog, giros y rectas.
- • **B4:** rectas de 150 cm con desvíos de 5.5 a 20 cm.
- • **B5:** ajuste de balance y recta.

• **B6:** cuatro corridas con una caja a ~100 cm. La segunda probablemente tocó la caja; la cuarta la evitó pero quedó ~40 cm a la derecha. Las corridas 1 y 3 hicieron dos esquivas y terminaron con rumbos estimados de 117/119°.

• **B7, B8, B9 y B2:** no se realizaron por decisión del dueño (`README.md`, J14).

Esos resultados son **históricos** y no se atribuyen a la APK nueva de J15.

9. J15: lo que no se midió

• **Cero corridas físicas.** No hay mediciones nuevas de contacto, desvío, rumbo, sensores ni anticipación.

• El criterio de **5 de 6 corridas sin tocar la caja y con desvío final menor de 25 cm** queda **NO MEDIDO**.

• La simulación B6 dio 400/400 sin contacto, error lateral medio 1.9 cm y p95 4.6 cm, pero es **SIMULADO** (`docs/reporte_J15.md` §4).

• No se creó v1.1; v1.0 sigue en `eb88151`.

10. Lo que ninguna prueba de PC cubre

• El deslizamiento real de las ruedas.

• La vibración del chasis y el ruido del giroscopio en movimiento.

• La luz del lugar, el color real del piso y los reflejos.

• La latencia real del WiFi y del video en el teléfono.

• La corriente y la temperatura del driver.

• El comportamiento de la batería al caer su voltaje.

• La sujeción de la cámara y su orientación exacta.

Por eso, cuando alguien pregunte «¿está probado?», la respuesta correcta es: **la lógica está probada en PC; el comportamiento físico tiene pruebas parciales de J13/J14 y J15 quedó pendiente.**

11. Cómo repetir las pruebas sin romper nada

1. Las pruebas de PC no tocan hardware: se pueden correr en cualquier momento.
2. Antes de una prueba física, revisar `docs/guia_conexion_J15.md` y `docs/seguridad.md`.
3. No operar el carro sin autorización del dueño.
4. Anotar fecha, comando, versión de la app, versión del firmware y resultado.
5. Si algo falla, no «arreglarlo» en la documentación: anotarlo en `docs/pendientes_documentacion.md` con archivo y línea.

12. Para seguir leyendo

• `docs/codigo/pruebas_firmware.md`: cómo están hechas las pruebas y qué simula el hardware falso.

• `docs/pruebas_control.md`: pruebas de la app con servidor falso.

• `docs/validacion_sensores.md`: pruebas físicas de los sensores.

• `docs/reporte_J14.md` y `docs/reporte_J15.md`: resultados y límites.

Referencia del simulador de rutas

Estado: documento de referencia · **Fecha:** 27 de octubre de 2026 · **Rama:** `docs/documentacion-completa`

Código documentado: `nucleo_c/` en `deae8b6`. **Fuentes:** `simulador_navegador.h/.c`, `sim_ruta.h/.c`, `pruebas_reincorporacion_j15.c`, `navegador.h`, `docs/reporte_J15.md` y `docs/navegador.md`.

1. En una frase

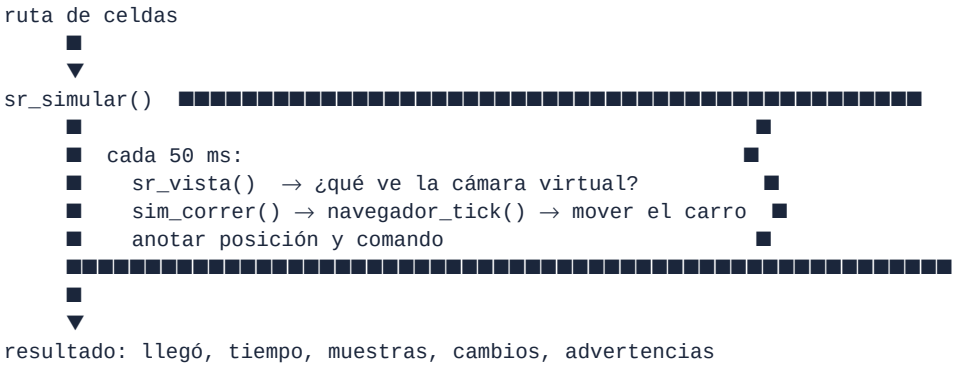
El simulador es un **carro de mentira dentro del PC**: recibe los mismos comandos que el carro real, se mueve con la misma calibración y le cuenta al navegador lo que vería una cámara y un láser. Sirve para repetir una ruta cien veces sin gastar batería, sin mover el carro y sin riesgo de chocar.

No demuestra cómo se comporta el carro de verdad. Demuestra que **la lógica del navegador hace lo que se espera en un mundo simplificado**. La diferencia entre las dos cosas es el centro de este documento.

2. Las dos piezas

Archivo	Qué hace	Se usa desde
`simulador_navegador.c`	Hace de **carro** (posición, rumbo, motores, frenado) y de **firmware** (giroscopio, láser, mensajes)	Pruebas del núcleo
`sim_ruta.c`	Hace de **cámara** y de **punto de entrada**: recibe una ruta de la cuadrícula y corre el ciclo completo	App, pantalla **Rutas** → **Simular**; pruebas

La idea es la misma que en el resto del proyecto: el navegador (`navegador.c`) no sabe si le habla un carro real o uno simulado. Recibe un comando, devuelve otro y consulta sensores por la misma interfaz. Por eso una prueba de simulación ejercita **el código real** de navegación, no una copia.



3. El carro virtual

`sim_iniciar()` coloca el carro en el centro de una celda con un rumbo (`simulador_navegador.c:53`). A partir de ahí, cada 50 ms `sim_correr()` hace lo que haría el firmware:

- **Avanzar o retroceder:** convierte el tiempo en distancia con `ms_por_cm_x10` de la calibración. Si el carro acaba de arrancar, espera `arranque_ms` antes de moverse; al soltar el comando, sigue un poco por el frenado (`frenado_mm`).
- **Girar:** usa `ms_por_grado_izq_x10` o `..._der_x10` según el lado. El giro no es simétrico porque cada pivote usa un motor distinto.
- **Desplazarse al girar:** aplica el desplazamiento del centro del chasis (`desp_*`), porque el carro no gira sobre un punto fijo.
- **Torcerse:** el desbalance entre los dos lados se puede configurar. Con `desbalance_mil × balance_mil = 1000 × 1000` va recto; si no, se desvía como un carro mal ajustado (`simulador_navegador.c:182`).
- **Fallar a propósito:** `pct_avance` y `pct_giro` permiten que el carro real recorra, por ejemplo, un 95 % de lo planeado. Así se prueba la deriva.

El simulador puede usar una calibración **distinta** de la que cree el navegador (`sim.real`). Eso reproduce el caso incómodo: el carro está mal calibrado, pero el cerebro cree que está bien. Es una de las formas más útiles de encontrar un error de diseño.

4. La cámara virtual

`sr_vista()` (`sim_ruta.c:87`) responde una pregunta sencilla: **si el carro está aquí y mira hacia allá, ¿qué obstáculos ve y en qué zona?**

• Los obstáculos son celdas de la cuadrícula de 20 × 20 cm. Se tratan como cajas de 20 cm de alto.

• Solo se ven si están **adelante**, dentro de unos **±30°** (60° de campo visual) y a menos de **80 cm** de la cara.

• La **zona** (izquierda, centro, derecha) sale de a qué lado del carro cae la celda, con los mismos tercios que usa el detector.

• La **cercanía** sale de la distancia a la cara: 3 hasta 30 cm, 2 hasta 50 cm y 1 más lejos.

• El número de **celdas** que ocuparía la caja crece al acercarse. Eso permite probar las reglas que distinguen un objeto real de un artefacto fijo.

El modelo es **aproximado**: no simula píxeles, ni colores, ni sombras, ni el detector por color. Solo entrega el resultado que el detector habría producido. Por eso no sirve para validar la visión; sirve para validar **cómo reacciona el navegador** a un resultado dado.

En la pantalla **Rutas** de la app se eligen obstáculos tocando celdas (`EditorRutaView.kt:7`). El simulador los convierte a esa vista y anima el recorrido con las muestras. La app usa la calibración y el balance guardados, si existen (`RutasFragment.kt:208-234`).

5. El firmware virtual

Además de la cámara, el simulador puede hacer de firmware con sensores:

Pieza	Qué simula	Interruptor
Giroscopio	Contesta los giros por ángulo, con error de escala, sesgo, ruido y anticipación que se corrige sola	`fw_giroscopio`
Láser	Manda `SENS` cada 100 ms y aplica la parada de emergencia a menos de 120 mm	`tof_activo`
Caja para el láser	Distancia calculada contra una caja del mundo, independiente de la cámara	`laser_externo`
Fallos	El firmware no contesta, el giroscopio falla en el giro N, se pierde un `GIRO_OK`, el láser está caído	`fw_sin_respuesta`, `fw_falla_desde`, `fw_pierde_mensaje`, `tof_perdido`

Con eso se prueban caminos que en el carro serían caros o peligrosos: «¿qué pasa si el giroscopio falla en el tercer giro?», «¿y si el mensaje de fin de giro se pierde pero el carro sí giró?».

6. La simulación de una ruta completa

`sr_simular()` (`sim_ruta.c:164`) es la función que usa la app. Recibe:

• la ruta (celdas vecinas),

• la calibración (o `NULL` para usar valores de ejemplo),

• hasta **16 obstáculos** (celdas),

• un **ruido de giro** en grados,

• una **semilla**.

Y devuelve un `ResultadoSimRuta` con:

Campo	Qué dice
`llego`	Si terminó en `TERMINADO`
`duracion_ms`	Cuánto tardó

`n_comandos`	Cuántas veces cambió el comando
`esquivadas`	Cuántas esquivas hizo
`x_final_mm`, `y_final_mm`	Dónde terminó
`error_final_mm`	Distancia al último punto de la ruta
`muestras[]`	Posición, rumbo, comando y estado cada 100 ms (hasta 1500)
`cambios[]`	Cada cambio de comando con su hora
`advertencias`	Salió de la cuadrícula, no llegó, se cortó por tiempo o usó calibración de ejemplo

El ciclo se corta a los **150 s**, si el navegador llega a `TERMINADO` o `ERROR`, o si se acaba la memoria de muestras. La **semilla** es el número con el que arranca el generador de ruido: con la misma semilla sale exactamente el mismo resultado, y con otra sale una variación distinta. Eso hace que cualquier fallo se pueda repetir y depurar.

`sr_dispersion()` repite la ruta hasta **24 veces** con ruidos distintos y resume cuántas llegaron, el error medio, el máximo y la dispersión entre finales. Es útil para ver si una ruta es robusta o si depende de que todo salga perfecto.

7. El banco de J15: 400 rutas B6

La corrección de pose de J15 se evaluó con `pruebas_reincorporacion_j15.c`, no con el carro. El escenario simulado es:

• una recta de **2 m** (11 puntos de una cuadrícula de 20 cm);

• una **caja de 20 × 20 cm** a **1 m** del punto de partida;

• el chasis aproximado por un rectángulo de **20 × 24 cm**;

• **400 corridas** con semillas distintas;

• video a **10 cuadros/s** con **200-300 ms** de retraso;

• giroscopio con ruido y error de cierre por lado;

• láser con alcance de **1.2 m** y autoridad de parada;

• desbalance del carro entre 1006 y 1018 milésimas y variación de ±5 % en el pivote (supuesta, no medida).

En la tabla, **media** es el promedio del error; **p95** significa que el 95 % de las corridas quedó por debajo de ese error (deja fuera el 5 % peor); **contactos** son las veces que el rectángulo del carro tocó el de la caja.

Resultados registrados (`docs/reporte_J15.md` §4):

Métrica	Antes de J15	Después de J15
Error lateral medio	13.2 cm	**1.9 cm**
Error lateral p95	21.9 cm	**4.6 cm**
Contactos con la caja	—	**0 en 400**
Rutas que llegan	—	400 de 400

Estas cifras son **SIMULADAS**. El propio reporte lo dice: el escenario reproduce errores de decenas de centímetros, pero no reproduce exactamente los 40 cm de B6-4 ni las esquivas múltiples de J14, y la mejora simulada **no demuestra** una mejora física. El criterio de 5 de 6 corridas reales sigue **NO MEDIDO**.

8. Las violaciones de seguridad

El simulador cuenta una cosa que nunca debe pasar: **que el navegador mande un movimiento estando en `ESPERANDO`, `TERMINADO` o `ERROR`** (`simulador_navegador.c:499`). En esos estados el comando debe ser siempre `STOP`.

Ese contador es una prueba de propiedad, no un caso concreto: se revisa en todas las corridas. Si alguna vez sube de cero, hay un fallo de seguridad, aunque la ruta haya llegado al final.

9. Cómo se ejecuta

Las pruebas del núcleo compilan el simulador con el resto:

```
cd nucleo_c
make pruebas_reincorporacion_j15
./pruebas_reincorporacion_j15
```

El programa imprime una línea por escenario con media, p50, p95, máximo, contactos, llegadas, éxitos y esquivas, y termina con `N casos, M fallos`. No se volvió a ejecutar durante el cierre documental por indicación del dueño; los números de la tabla son los del reporte de J15.

Para una ruta cualquiera desde la app: **Rutas** → **dibujar** → **Simular**. La animación usa las muestras reales del simulador, no una interpolación inventada.

10. Lo que el simulador NO prueba

• **Deslizamiento de ruedas.** El carro virtual avanza exactamente lo que dice la calibración (salvo que se cambie `pct_avance` a mano). Un carro real patina y no se enteró.

• **El detector de color real.** La cámara virtual entrega zonas y cercanías ya calculadas; no ve sombras, reflejos ni pisos de dos colores.

• **El modelo de objetos.** EfficientDet no participa en la simulación.

• **La física del choque.** El rectángulo del chasis sirve para contar contactos en el banco J15, pero no hay rebote, ni empuje, ni daño.

• **El WiFi, la batería y el calentamiento.** Nada de eso existe en el mundo simulado.

• **La latencia real de extremo a extremo.** El retraso de video se configura a mano; no se mide.

• **El comportamiento de la cámara montada y en movimiento.** No hay vibración ni cambios de luz.

Por eso la simulación responde «¿la lógica es coherente?», no «¿el carro va a funcionar en el piso?». Para lo segundo hacen falta las corridas físicas, que en J15 fueron cero.

11. Para seguir leyendo

• `docs/referencia/nucleo.md`: el navegador y sus estados.

• `docs/reporte_J15.md`: el banco completo y sus límites.

• `docs/navegador.md`: referencia histórica de parámetros y diagramas.

• `docs/referencia/pruebas.md`: las suites y qué cubre cada una.

• `docs/codigo/nucleo_c.md`: recorrido por las funciones del simulador.

13. Historia del proyecto y decisiones de diseño

Historia y evolución cronológica del proyecto

Materia: Fundamentos de los Sistemas de Información (FSI)

Profesor: Dr. Horacio Rostro González

Fecha: 27 de octubre de 2026

---

1. De «WALL-E» a «el carro»: origen y replanteamiento

El proyecto comenzó originalmente bajo el nombre informal de *WALL-E*, consistiendo en un prototipo elemental de carro robot basado en un sketch monolítico de Arduino para ESP32. Aquel sistema inicial presentaba limitaciones críticas:

• La lógica de movimiento, la gestión del servidor web y el acceso a los pines estaban mezclados en un único archivo.

• No existía retroalimentación sensorial de orientación ni de distancia: los giros y avances se realizaban a ciegas por tiempo estimado.

• Toda la teleoperación dependía de una página web simple propensa a desconexiones.

Con el fin de convertir el prototipo en un proyecto de ingeniería robusto y formal para la materia FSI, se rebautizó el vehículo oficialmente como **«el carro»** (o **carro FSI**), iniciando una profunda reestructuración arquitectónica.

---

2. Línea de tiempo por jornadas de trabajo

[J1 - J10]	[J11]	[J12 - J13]	[J14]
Modularización C99	Sensores I2C	Metrología física	Entrega v1.0
Desacoplamiento hw_...	MPU6050 + VL53L0X	Balance recto (3 cm)	APK < 15 MB
Pruebas en PC sin placa	Giro angular cerrado	Cámara GC0308 y HQVGA	Estrés 100k c
App Android y JNI	Perfiles AUTO/MANUAL	Keep-alive powerbank	Pruebas B0 a

Jornadas 1 a 10: Desacoplamiento arquitectónico y núcleo C

• **Separación de capas:** Se extrajo toda la lógica de control del sketch de Arduino hacia un módulo en C estándar (`carro.c`), interactuando con el microcontrolador únicamente mediante llamadas a funciones de abstracción de hardware (`hw_motor_pwm`, `hw_millis`).

• **Nacimiento del banco de pruebas en PC:** Al no depender de Arduino, se creó `pruebas_firmware/`, permitiendo probar la lógica del carro en computadoras con hardware falso y verificar el watchdog con precisión de un milisegundo.

• **Núcleo de navegación y JNI:** Se desarrolló la biblioteca matemática en C99 (`nucleo_c/`) para calcular trayectorias sobre cuadrícula de 20 cm, importada directamente en la app Android mediante el NDK sin duplicar código.

• **Percepción inicial:** Se implementó el detector de obstáculos por colorimetría del piso (`obstaculos.c`) y la proyección del corredor trapezoidal (`corredor.c`).

Jornada 11: La revolución sensorial (Giroscopio y Láser)

• **Integración I2C:** Se incorporaron en un bus común de 400 kHz un giroscopio MPU6050 (`0x68`) y un sensor ToF láser VL53L0X (`0x29`).

• **Giro en bucle cerrado (`GIRAR:<grados>`):** El carro abandonó los giros ciegos por tiempo; ahora el giroscopio mide la velocidad angular a 100 Hz y corta el motor en el ángulo exacto.

• **Parada de emergencia por láser:** El sensor ToF adquirió autoridad para frenar el vehículo ante colisiones inminentes (`TOF_PARO`).

• **Dos perfiles de velocidad:** Se crearon los modos `MANUAL` (potencia plena 255) y `AUTO` (velocidad moderada 210 para facilitar frenados reactivos).

Jornadas 12 y 13: Metrología y afinación de hardware

• **Calibración de balance en línea recta:** Se descubrió que el motor izquierdo empujaba más que el derecho, provocando un desvío de 14.5 cm en 1.5 metros. Se ajustó el factor de balance a 988 (252 izquierda / 255 derecha), logrando reducir la desviación a solo 3.0 cm.

• **Retos de la cámara Freenove:** Se constató que el sensor GC0308 carece de compresión JPEG por hardware. Se implementó compresión por software en PSRAM y se determinó que la resolución **HQVGA (240×176)** era la óptima (~12.4 fps frente a los 8.8 fps lentos de QVGA). Se diseñó además el protocolo UDP de celdas (`CEL1`).

• **Keep-alive de batería:** Se introdujo un pulso en GPIO 2 para evitar que la powerbank apague su salida por baja demanda de corriente.

Jornada 14: Cierre formal de la versión 1.0 (Commit `eb88151`)

• **Compresión del APK oficial:** Se configuró el empaquetado de Gradle para comprimir bibliotecas compartidas del NDK, reduciendo el APK final a 14.85 MB para respetar el límite de 15 MB.

• **Validación masiva:** Se ejecutaron pruebas de estrés con 100 000 comandos continuos sin registrar pérdidas de comandos de frenado.

• **Pruebas físicas:** Se completaron las pruebas B0 a B6 en pista física. Por decisión del dueño del proyecto, las pruebas B7, B8, B9 y B2 se suspendieron.

Jornada 15: Corrección de pose y diagnóstico del escenario B6

• **El problema del desvío lateral post-esquive:** En simulaciones del escenario B6 (esquivar una caja y continuar), se observó que el carro sufría un desvío lateral final acumulado de 13.2 cm en promedio (p95 de 21.9 cm).

• **Causa raíz identificada:** El navegador proyectaba los incrementos de avance usando el rumbo original fijado al inicio de la ruta; un giro intermedio introducía errores angulares no compensados al volver a avanzar.

• **Solución implementada:** Se integró en `navegador.c` la lectura del rumbo instantáneo reportado por el giroscopio (`SENS`) para proyectar la pose en tiempo real.

• **Resultado simulado:** En 400 rutas simuladas del escenario B6, el error lateral cayó a 1.9 cm promedio (p95 de 4.6 cm), logrando 400 de 400 llegadas exitosas.

• **Estado de cierre:** No se realizaron pruebas físicas en el vehículo real; la validación física quedó catalogada como **NO MEDIDA**.

---

3. Decisiones de diseño y alternativas descartadas

A lo largo del proyecto se evaluaron múltiples enfoques alternativos que fueron formalmente descartados por razones de ingeniería:

1. **Uso de encoders ópticos en las ruedas:**

Descartado. Los motores TT amarillos de bajo costo sufren holguras mecánicas y saltos de paso en los discos perforados. Instalar encoders agregaba cableado frágil y consumo de interrupciones sin resolver el problema del derrape de las llantas en el piso. Se optó por el giroscopio MPU6050, que mide la rotación real del chasis con independencia del patinaje de las ruedas. 2. **Procesamiento de visión en servidor en la nube / PC externa:**

Descartado. Introducía dependencias de conexión a internet externa y latencias de transmisión impredecibles (superiores a 300 ms). Se adoptó una arquitectura 100 % local donde la cámara sirve video dentro de la red del vehículo y el teléfono ejecuta las redes neuronales en su propio procesador. 3. **Parada de emergencia basada exclusivamente en visión:**

Descartado. Las variaciones de iluminación en aulas y la latencia del flujo de video impedían garantizar una distancia de frenado segura ante obstáculos inesperados. Se otorgó autoridad prioritaria al sensor ToF láser VL53L0X, cuya respuesta es determinista en milisegundos. 4. **Mapeo SLAM / Grilla persistente de ocupación:**

Descartado. Mantener un mapa métrico completo consumía demasiada memoria y requería odometría milimétrica. Para el alcance del curso, una política reactiva basada en un corredor trapezoidal proyectado y maniobras de evasión acotadas (desvío de 40 cm y avance de 30 cm) resultó infinitamente más simple, confiable y verificable. 5. **Rampa suave de aceleración (*soft*

***start*):**

Descartado. Al intentar arrancar con valores PWM progresivos desde 0 o 100, la fricción estática del peso del carro hacía que los motores zumbasen sin girar. Se implementó en su lugar un «golpe de arranque» de potencia plena (255) durante exactamente 250 ms (`ARRANQUE_PLENO_MS`), conmutando después a la velocidad crucero.

14. Resultados y estado de validación

[FOTO PENDIENTE: FOTO_09] — Montaje de pruebas de obstáculo

Encuadre requerido: Pista con cinta métrica extendida y caja de 20x20 cm en la ruta
Archivo esperado: docs/imagenes/foto_09_escenario_obstaculo.jpg

Compendio de resultados y estado de validación

Materia: Fundamentos de los Sistemas de Información (FSI)

Proyecto: Carro con navegación asistida

Fecha de consolidación: 27 de octubre de 2026

Criterio editorial: Separación estricta entre hechos medidos en hardware, resultados de simulación en PC y aspectos no medidos.

---

1. Resultados verificados físicamente en hardware

Las siguientes métricas fueron obtenidas mediante mediciones directas sobre el chasis físico, los microcontroladores y el teléfono de pruebas:

Subsistema / Parámetro	Condición de prueba	Resultado medido	Evidencia documental
Balance en línea recta (Perfil MANUAL)	Avance de 150 cm sobre piso plano a potencia 255.	Desviación lateral reducida de **14.5 cm** (con 255/255) a **3.0 cm** aplicando balance 988 (252 izquierda / 255 derecha).	`firmware_carro/config.h:103-104`; `docs/reporte_J13_fisico.md`.
Rendimiento de cámara GC0308 (HQVGA)	Barrido de 100 cuadros, 20 MHz, RGB565, calidad JPEG 50.	**12.4 fps** promedio de salida; tiempo de compresión por software de **56 ms**; edad media del cuadro de **137 ms**.	`docs/camara_prueba.md` §«Lo medido»; `docs/referencia/camara.md:109`.
Rendimiento de cámara GC0308 (QVGA)	Barrido de 100 cuadros, 20 MHz, RGB565, calidad JPEG 50.	**8.8 fps** promedio; tiempo de compresión de **107 ms**; edad media del cuadro de **252 ms**.	`docs/camara_prueba.md` §«Lo medido».
Tamaño del APK oficial	Compilación release con bibliotecas nativas comprimidas.	**14,852,386 bytes** (cumple el límite de 15,000,000 bytes). SHA-256: `36CE38F2...`.	`dist/carro_fsi_0.4-074c9fd.apk`; `scripts/verificar_version.py`.
Tamaño de la imagen de firmware	Compilación Arduino-ESP32 3.3.12 con optimizaciones de espacio.	**1,031,712 bytes**. SHA-256: `9FD1BB51...`.	`dist/firmware_carro_453ed20-20261027.bin`.
Tiempo de bucle del firmware (`LOOP?`)	Ejecución continua con muestreo I2C a 400 kHz y WebSocket.	Ciclo de ejecución inferior a **5.0 ms** en régimen normal; I2C con timeout estricto de 5 ms.	`firmware_carro/config.h:163`; `firmware_carro.ino:535`.
Pruebas de pista superadas (J14)	Pruebas de integración en pasillo con cinta métrica.	Escenarios B0, B1, B3, B4, B5 y B6 ejecutados en hardware en la jornada J14.	`docs/reporte_J14.md`.

---

2. Resultados obtenidos mediante simulación en PC

Las siguientes cifras corresponden a modelos matemáticos y simulaciones Montecarlo ejecutadas en computadora; **no representan mediciones del carro físico**:

Escenario simulado	Modelo computacional	Resultado antes de J15	Resultado con J15	Interpretación técnica
Escenario B6 (400 rutas con obstáculo frontal)	Simulador cinemático en C (`nucleo_c/sim_ruta.c`) con ruido angular parametrizado.	Error lateral medio: **13.2 cm**. Percentil 95 (p95): **21.9 cm**. Llegadas exitosas: 344/400.	Error lateral medio: **1.9 cm**. Percentil 95 (p95): **4.6 cm**. Llegadas exitosas: **400/400**.	Integrar el rumbo real del giroscopio durante el avance posterior al esquive eliminó la acumulación de error lateral simulado.
Escenario Recta sin obstáculo (400 rutas)	Modelo cinemático virtual.	Error medio: 16.2 cm. p95: 27.0 cm.	Error medio: **7.4 cm**. p95: **12.4 cm**.	No degrada la trayectoria básica.
Escenario Trayectoria en L (400 rutas)	Modelo cinemático virtual con pivote a 90°.	Error medio: 8.8 cm. p95: 15.5 cm.	Error medio: **2.6 cm**. p95: **6.0 cm**.	Mejora la alineación posterior al pivote.
Suite de pruebas unitarias del núcleo	21 ejecutables C99 independientes.	—	**1598 casos de prueba, 0 fallos**.	Verificación exhaustiva de algoritmos de trayectoria, celdas, corredor y navegador.
Suite de pruebas de firmware	Módulos C compilados con hardware falso.	—	**447 casos de prueba, 0 fallos**.	Verificación de watchdog a 1000/1001 ms, arranque escalonado y recuperación de cámara.
Prueba de estrés de cola de comandos	Inyección masiva de 100 000 comandos hacia `carro.c`.	—	**100 000 comandos procesados sin perder un solo `STOP`**.	Robustez comprobada de la política de descarte de la cola.

---

3. Estado de validaciones físicas no realizadas (NO MEDIDO)

Con el fin de mantener la honestidad técnica más rigurosa, se señalan explícitamente aquellos puntos que **NO HAN SIDO MEDIDOS FÍSICAMENTE** en el vehículo:

1. **Validación física de la corrección de pose J15:**

La modificación introducida en J15 para integrar el rumbo del giroscopio durante los avances cuenta **únicamente con respaldo en simulación computacional**. En el carro físico se realizaron **cero corridas con esta versión**, y el criterio formal de aceptación de 5 de 6 llegadas consecutivas permanece como **NO MEDIDO**. El dueño del proyecto dejó esta validación como pendiente para continuarse con otro modelo de pruebas (`docs/reporte_J15.md`). 2. **Balance del perfil AUTO en pista real:** El factor de balance de motores para el perfil AUTO (207 a la izquierda y 210 a la derecha) fue escalado matemáticamente a partir del perfil MANUAL. No ha sido medido con cinta métrica sobre el carro físico (**NO MEDIDO**). 3. **Latencia de video de extremo a extremo:** El retraso total desde que la luz incide en el sensor GC0308 de la cámara hasta que los píxeles se iluminan en la pantalla OLED del teléfono Android solo cuenta con una medición aislada informal de ~530 ms. No existe una caracterización metrológica formal (**NO MEDIDO**). 4. **Pruebas físicas B2, B7, B8 y B9:** Fueron suspendidas durante el cierre de J14 por decisión del dueño del proyecto; no se presentan como superadas.

---

4. Referencias visuales

• `docs/figuras/fig_05_pose_j15.png`: Comparación gráfica de pose proyectada antes y después de J15.

• `docs/figuras/fig_06_esquive.png`: Geometría de maniobra de esquive y reincorporación a la línea.

15. Límites, riesgos y trabajo futuro

Límites conocidos del prototipo

1. **Ausencia de encoders ópticos:** El carro carece de medición directa del giro de las ruedas. Toda estimación de desplazamiento lineal depende del modelo temporal ($d = t / ext{ms/cm}$). Si las ruedas patinan por exceso de cera en el suelo, el odómetro acumula error de avance.
2. **Dependencia de la carga de las pilas AA:** Conforme las pilas alcalinas se descargan, la velocidad lineal cae paulatinamente. Se mitiga mediante la regla de los 90 minutos y 20 corridas.
3. **Compresión de video por software:** El sensor GC0308 no tiene silicio JPEG; el límite físico de salida es de ~12.4 fps en HQVGA.
4. **Campo estrecho del sensor láser ToF:** El cono de visión del VL53L0X cubre ~25°; objetos angostos ubicados en los extremos del parachoques requieren ser detectados por visión.

Líneas de trabajo futuro

• Sustitución de la placa de cámara por un módulo con sensor **OV2640 o ESP32-S3**, habilitando compresión JPEG por hardware a 30 fps en resolución VGA.

• Incorporación de una batería LiPo 2S de 7.4 V con circuito regulador step-down conmutado para alimentar motores con voltaje constante independiente del tiempo de prueba.

• Migración hacia algoritmos de fusión EKF (Filtro de Kalman Extendido) integrando acelerometría lineal filtrada.

16. Guía de operación y seguridad física

Guía de operación y rutinas del carro FSI

Materia: Fundamentos de los Sistemas de Información (FSI)

Propósito: Manual de procedimientos prácticos para el encendido, puesta a punto, teleoperación y ejecución de rutas autónomas.

Fecha de vigencia: 27 de octubre de 2026

---

1. Reglas operativas fundamentales

Para garantizar la integridad del hardware y la reproducibilidad de los resultados, se deben respetar estrictamente las siguientes cuatro reglas:

1. **Regla de los 90 minutos y 20 corridas:**

Los motores TT alimentados con pilas AA sufren caídas perceptibles de velocidad y torque a medida que la batería se descarga. **No se deben realizar más de 20 corridas continuas o superar 90 minutos de pruebas con el mismo juego de pilas**. Pasado este umbral, los factores de calibración de avance y giro dejan de ser válidos. 2. **Procedimiento «Colocación y LISTO»:** Antes de iniciar cualquier recorrido autónomo, el carro debe alinearse manualmente sobre la línea de partida con las cuatro ruedas en contacto plano con el suelo. Una vez posicionado, se debe verificar en la pantalla de la app que el giroscopio reporte un rumbo estable de 0.0° (`IMU_CERO`) y que el sensor ToF marque el frente despejado antes de presionar el comando de inicio. 3. **Mecanismos de parada inmediata:** El operador debe mantener siempre el pulgar listo sobre el botón rojo de **Freno de Emergencia (`EBRAKE`)** en la pantalla del teléfono. Si el vehículo no responde a la app, el operador debe apagar físicamente el interruptor del portapilas de motores. 4. **Mantenimiento del piso de prueba:** La tracción depende enteramente de la fricción entre los neumáticos de hule y la superficie. La pista debe estar libre de polvo, grasa o cables sueltos que puedan enredarse en los ejes de los motores.

---

2. Procedimiento de encendido y enlace inicial

Paso 1: Verificación de fuentes de energía

• Confirmar que la powerbank de 5 V cuente con al menos 75 % de carga (3 o 4 LEDs indicadores encendidos).

• Instalar 4 pilas alcalinas tipo AA nuevas o recién recargadas en el portapilas de potencia.

• Mantener el interruptor de los motores en posición **APAGADO (OFF)** durante el arranque inicial.

Paso 2: Conexión de la electrónica

• Conectar el cable USB de la powerbank al puerto micro-USB de la placa ESP32 DevKit. Se encenderá el LED rojo de alimentación de la placa.

• Conectar el segundo cable USB de la powerbank a la placa de la cámara Freenove ESP32-WROVER.

• Esperar aproximadamente 5 segundos a que ambas placas inicialicen sus periféricos y la red WiFi esté lista.

Paso 3: Emparejamiento inalámbrico del teléfono

1. En el teléfono móvil Android, abrir Ajustes de WiFi.
2. Localizar y seleccionar la red inalámbrica emitida por el carro (nombre configurado en `firmware_carro/secretos.h`).
3. Ingresar la contraseña de red predefinida.
4. Esperar a que el teléfono obtenga dirección IP automática por DHCP (habitualmente `192.168.4.10`).

Paso 4: Inicio y conexión de la app

1. Abrir la aplicación **Carro FSI** en el dispositivo móvil.

2. En la pestaña **Inicio**, verificar que la dirección de destino esté configurada en `192.168.4.1`.
3. Presionar el botón **Conectar**.
4. Confirmar que el indicador pase a color verde («Conectado») y que el panel de telemetría comience a reflejar lecturas de latencia de ping (habitualmente entre 15 y 45 ms), ángulo del giroscopio y distancia del láser ToF.
5. Encender el interruptor de las pilas de motores (**ON**). El vehículo está listo para operar.

---

3. Modos de operación

3.1 Modo Manual (Teleoperación)

Ubicado en la pestaña **Manual** de la aplicación móvil.

• **Controles de dirección:** Botones para Adelante, Atrás, Pivote Izquierda y Pivote Derecha. Al mantener presionado un botón, la app transmite ráfagas de comando cada 250 ms. Al soltarlo, se emite un `STOP`.

• **Selector de Perfil:**

• **MANUAL:** Tracción a máxima potencia (255 derecho / 252 izquierdo). Diseñado para desplazamientos ágiles y superar pequeños desniveles.

• **AUTO:** Tracción regulada a 210 de potencia (207 izquierdo). Recomendado para maniobras precisas en espacios reducidos.

• **Transmisión de video:** El centro de la pantalla muestra en tiempo real el flujo MJPEG proveniente de la cámara (`http://192.168.4.2/stream`).

3.2 Modo Semiautónomo (Navegación por Rutas)

Ubicado en la pestaña **Rutas**.

1. **Trazado en cuadrícula:** La pantalla presenta una retícula métrica donde cada celda representa un área física de 20×20 cm.
2. **Definición de waypoints:** Con el dedo, el usuario pulsa sobre las celdas para marcar la trayectoria deseada (punto de inicio, cambios de dirección y destino final).
3. **Simulación previa:** Pulsar el botón **Simular** para visualizar una recreación cinemática del recorrido estimado según los parámetros de calibración vigentes.
4. **Ejecución:** Al pulsar **Ejecutar Ruta**, el navegador nativo en C (`navegador.c`) toma el control:

• El vehículo avanza por segmentos temporizados calculados con la constante de milisegundos por centímetro.

• En cada cambio de dirección, ejecuta pivotes guiados por el giroscopio.

• Si la cámara o el láser ToF identifican una caja u obstáculo en el corredor de seguridad, el carro frena, evalúa el espacio lateral disponible, se desvía 40 cm hacia el lado libre, supera la caja y busca un punto de reincorporación 30 cm por delante de la obstrucción.

---

4. Lista de comprobación antes de una demostración o examen

Antes de la presentación final ante el jurado o profesor, ejecutar meticulosamente esta lista de preparación:

• [ ] **Ajustes de red restablecidos:** En la app, acceder a `Ajustes` → `Red` y verificar que las IPs coincidan con `192.168.4.1` (carro) y `192.168.4.2` (cámara).

• [ ] **Pilas nuevas instaladas:** Colocar un juego de 4 pilas AA alcalinas sin uso previo para asegurar que los giros y avances mantengan el torque calibrado.

• [ ] **Aprender el color del piso del aula:** En la pantalla `VisionActivity`, apuntar la cámara al piso del lugar de la demostración y presionar el botón de muestreo cromático para que el detector reconozca la superficie del recinto.

• [ ] **Prueba de avance recto de 1 metro:** Ejecutar un avance de verificación en línea recta para confirmar que el carro no presenta desviaciones excesivas por balance.

• [ ] **Aseguramiento del teléfono:** Desactivar el bloqueo automático de pantalla del móvil para evitar que se apague en mitad de una ruta autónoma.

Seguridad física, eléctrica y lógica del carro FSI

Materia: Fundamentos de los Sistemas de Información (FSI)

Propósito: Análisis exhaustivo de los riesgos operativos y las capas de protección implementadas en hardware y software.

Fecha: 27 de octubre de 2026

---

1. Seguridad eléctrica: aislamiento y tierras compartidas

1.1 El principio de dos fuentes separadas

El sistema móvil requiere alimentar dos clases de cargas eléctricas con requerimientos radicalmente distintos:

• **Cargas digitales sensibles (microcontroladores y sensores):** Exigen una tensión constante y libre de fluctuaciones (5 V USB regulados internamente a 3.3 V). Son vulnerables a reinicios espurios (*brownouts*) si el voltaje cae por debajo de 2.7 V.

• **Cargas inductivas ruidosas (motores eléctricos TT):** Generan picos de corriente elevados al arrancar (hasta 1.5 A combinados) y ruido inductivo de retorno al frenar o conmutar el sentido de giro.

Por esta razón, **está estrictamente prohibido alimentar los motores desde la misma powerbank o regulador de la placa ESP32**.

```
Powerbank 5 V ■■■■■■■■■■■■ ESP32 DevKit (5 V) ■■■ Sensores I2C (3.3 V)
(Electrónica) ■■■■■■■■■■■■ Freenove Cámara (5 V)
                                  ■
                          GND COMÚN (0 V)
                                  ■
4 Pilas AA (~6 V) ■■■■■■■■ TC1508A (VDD) ■■■■■■■■ 4 Motores TT
(Potencia)
```

1.2 Regla de oro del cableado

ADVERTENCIA CRÍTICA: El terminal positivo (+ ~6 V) proveniente del portapilas de 4 pilas AA debe conectarse ÚNICA Y EXCLUSIVAMENTE al pin VDD del driver TC1508A. Bajo ninguna circunstancia debe tocar ningún pin de la placa ESP32 ni de los módulos sensores. Conectar 6 V a los pines de 3.3 V o 5 V del ESP32 destruirá irreversiblemente el microcontrolador.

1.3 Necesidad del bus de tierra común (GND)

Las señales PWM que controlan la velocidad y dirección de los motores se transmiten como niveles de voltaje (0 V = bajo, 3.3 V = alto) desde los GPIOs del ESP32 hacia las entradas lógicas **`INA`**, **`INB`**, **`INC`** e **`IND`** del driver TC1508A.

Para que el driver reconozca estos niveles, **la tierra de la powerbank y la tierra de las pilas AA deben estar rígidamente unidas**. Si la conexión de tierra común se interrumpe:

• Los voltajes de control flotan sin referencia fija.

• El driver puede entrar en conducción descontrolada, dejando los motores encendidos a plena potencia de manera impredecible.

---

2. Seguridad térmica y mecánica

2.1 Límites de corriente del driver TC1508A

El circuito integrado TC1508A soporta una corriente continua de 1.8 A y picos máximos de hasta 2.5 A por canal:

• Con las cuatro ruedas girando libremente, el consumo típico ronda los 300–500 mA.

• Si el vehículo queda atrapado contra una pared o las ruedas se bloquean mecánicamente, la corriente de bloqueo puede elevarse rápidamente a más de 2 A, provocando el calentamiento acelerado del encapsulado del driver.

• **Protección operativa:** Nunca forzar el avance del carro si se encuentra atascado físicamente. En caso de colisión, presionar de inmediato **`EBRAKE`**.

2.2 Temporizador de corte en giros (`GIRO_TIMEOUT`)

Al ejecutar maniobras de giro angular (`GIRAR:<grados>`), si el carro no logra alcanzar el ángulo objetivo antes de un tiempo límite preestablecido (por ejemplo, si una rueda resbala sobre una superficie jabonosa o queda trabada), el firmware aborta el giro, apaga los motores y emite la alarma `GIRO_TIMEOUT` hacia la app.

---

3. Seguridad lógica y vigilancia por software (Watchdogs)

El sistema integra tres niveles independientes de supervisión automática:

3.1 Watchdog de comunicación (1000 ms)

• **Función:** Supervisa la continuidad del enlace inalámbrico entre la app y el carro.

• **Comportamiento:** Si transcurren **1000 ms** sin que el ESP32 reciba un comando válido de movimiento (`FORWARD`, `REVERSE`, `LEFT`, `RIGHT`), el firmware asume que el teléfono se apagó, la app crasheó o se perdió la cobertura WiFi. En el milisegundo **1001**, corta inmediatamente la señal PWM a los cuatro motores, establece estado `STOP` y restablece el perfil de velocidad a `AUTO` (`carro.c:245-265`).

• **Inmunidad del PING:** Los mensajes de sondeo de latencia `PING` no reinician el temporizador del watchdog, forzando a la app a enviar comandos explícitos de avance.

3.2 Watchdog de hardware del microcontrolador (5000 ms)

• El temporizador de vigilancia por hardware del ESP32 (`esp_task_wdt`) está configurado a 5000 ms (`firmware_carro.ino`).

• En cada ciclo de la función `loop()`, se ejecuta `esp_task_wdt_reset()`.

• Si el hilo principal sufriera un bloqueo indefinido por alguna transacción I2C colgada o fallo de memoria, el microcontrolador se reinicia automáticamente en frío, llevando de inmediato los pines de motor a nivel bajo por hardware.

3.3 Parada de emergencia por proximidad láser (Autoridad ToF)

• En perfil `AUTO`, el sensor láser VL53L0X mide continuamente la distancia hacia el frente.

• Si detecta un objeto dentro de la zona de seguridad mínima predefinida, el firmware interrumpe la tracción de forma autónoma con prioridad sobre las órdenes de la app, deteniendo el vehículo e informando `TOF_PARO:<mm>`.

---

4. Referencias visuales

• `docs/figuras/fig_10_alimentacion.png`: Esquema de distribución eléctrica y advertencia de voltajes.

• `docs/figuras/fig_03_flujo_comando.png`: Arquitectura del watchdog y cola prioritaria de frenado.

17. Preguntas frecuentes de la exposición oral

Guion de exposición y banco de preguntas frecuentes

Materia: Fundamentos de los Sistemas de Información (FSI)

Profesor evaluador: Dr. Horacio Rostro González

Fecha de presentación: 1 de diciembre de 2026

Proyecto: Carro con navegación asistida por visión y sensores

---

1. Estructura sugerida para la presentación oral (15 minutos)

```
[0:00 - 2:00]   Introducción: El reto de la navegación autónoma en hardware accesible.
[2:00 - 5:00]   Arquitectura en tres nodos: ESP32 DevKit, Freenove Cámara y App Android.
[5:00 - 8:00]   Hardware y seguridad: Aislamiento eléctrico, giroscopio MPU6050 y láser ToF.
[8:00 - 11:00]  Algoritmos centrales: Máquina de estados, corredor visual y corrección de pose
[11:00 - 13:00] Demostración en vivo: Enlace, video en tiempo real y ejecución de ruta.
[13:00 - 15:00] Resultados y conclusiones: Separación transparente de lo medido vs simulado.
```

---

2. Guion paso a paso de la presentación

Minuto 0:00 a 2:00 — Introducción y planteamiento del problema

• **Apertura:** Saludo formal al profesor y sínodo. Presentación del equipo y del objetivo del proyecto: *«Diseñar e implementar un vehículo terrestre autónomo capaz de ejecutar trayectorias planificadas y sortear obstáculos dinámicos empleando hardware embebido de bajo costo»*.

• **El reto técnico:** Navegar con precisión en plataformas accesibles sin recurrir a sensores LiDAR industriales costosos ni a encoders ópticos de alta resolución.

• **Tesis de ingeniería:** La combinación de una arquitectura desacoplada, algoritmos cinemáticos en C99 puro y fusión sensorial ligera permite navegación reactiva confiable sobre chasis educativos.

Minuto 2:00 a 5:00 — Arquitectura distribuida del sistema

• **Los tres nodos:**

1. *ESP32 DevKit:* Controlador de tiempo real, tracción PWM, punto de acceso WiFi y muestreo de sensores I2C a 100 Hz.
2. *Freenove ESP32-WROVER:* Nodo de visión independiente con cámara GC0308 y 4 MB de PSRAM para compresión JPEG.
3. *Teléfono Android:* Interfaz de usuario, planificación en cuadrícula de 20 cm e inferencia de modelos neuronales.

• **Por qué tres nodos:** Separar la visión de la tracción evita que la compresión de cuadros degrade los tiempos del PWM de los motores o el muestreo del giroscopio.

Minuto 5:00 a 8:00 — Hardware, alimentación y seguridad

• **El problema del ruido eléctrico:** Los motores TT generan picos de corriente y ruido inductivo capaces de reiniciar microcontroladores (*brownout*).

• **Solución implementada:** Dos fuentes físicamente independientes:

• Powerbank USB de 5 V para la electrónica sensible (ESP32 y cámara).

• 4 pilas AA (~6 V) exclusivas para el driver de motores TC1508A.

• Tierra común (GND) para mantener la referencia lógica de las señales PWM.

• **Sensores a bordo:** MPU6050 (eje Z para giros de precisión) y VL53L0X (láser ToF para parada de emergencia autónoma).

Minuto 8:00 a 11:00 — Algoritmos y calidad de software

• **Núcleo C99 (`nucleo_c`):** Código portátil sin librerías externas que corre tanto en Android (vía JNI) como en PC para pruebas unitarias.

• **Máquina de estados finitos:** 8 estados predecibles (**`ESPERANDO`**, **`SIGUIENDO`**, **`DETENIDO`**, **`ESQUIVANDO`**, **`REINCORPORANDO`**, etc.).

• **Percepción:** Corredor trapezoidal de paso proyectado en la cámara; los objetos fuera del corredor no detienen la marcha.

• **Rigor de ingeniería:** 2045 casos de prueba en PC con 0 fallos, pruebas de estrés con 100 000 comandos continuos y watchdog de 1000 ms.

Minuto 11:00 a 13:00 — Demostración práctica

• Encendido de placas, conexión a la red WiFi del carro (**`192.168.4.1`**).

• Muestra de telemetría viva en la app (ping de ~25 ms, ángulo de giroscopio en 0° y distancia milimétrica del láser).

• Demostración de teleoperación manual fluida y visualización de video MJPEG.

• Trazado de una ruta en la cuadrícula y ejecución asistida.

Minuto 13:00 a 15:00 — Resultados y honestidad científica

• Presentación clara de los logros verificados en hardware: desvío recto reducido a 3 cm a 150 cm mediante balance 988; streaming de video a 12.4 fps en HQVGA.

• Diagnóstico de la jornada J15: en simulación computacional, la pose corregida redujo el desvío lateral post-esquive de 13.2 cm a 1.9 cm.

• Aclaración honesta de que la validación física de J15 queda catalogada como **NO MEDIDA**.

---

3. Banco de preguntas frecuentes (FAQ) y respuestas técnicas

P1: ¿Por qué no instalaron encoders en las ruedas del carro?

Respuesta: Los chasis comerciales con motores TT amarillos tienen engranajes plásticos con holguras mecánicas considerables y las llantas patinan ligeramente sobre pisos pulidos. Los encoders miden el giro del eje del motor, pero no detectan el derrape de la rueda en el suelo, acumulando error de rumbo rápidamente. En su lugar, decidimos instalar un giroscopio digital MPU6050: al ir fijado rígidamente al chasis, mide la velocidad de rotación real del vehículo en el espacio con independencia total del patinaje de las ruedas.

P2: ¿Por qué utilizar dos placas ESP32 distintas en lugar de conectar la cámara y los motores a una sola?

Respuesta: La placa con cámara (Freenove ESP32-WROVER) utiliza un sensor GC0308 que entrega imágenes crudas RGB565 sin comprimir. La compresión a JPEG la realiza el microcontrolador por software, lo cual consume entre 50 y 110 ms de tiempo de procesador por cuadro y hace uso intensivo de la memoria PSRAM. Si corriéramos la compresión JPEG en la misma placa que gestiona los motores y el bus I2C, la función **`loop()`** sufriría pausas inadmisibles de más de 100 ms, provocando retardos en la respuesta de frenado, desbordamientos de la cola WebSocket y pérdida de precisión en la integración del giroscopio.

P3: ¿Qué ocurre si la red WiFi se cae o el teléfono se apaga mientras el carro avanza a máxima potencia?

Respuesta: El firmware implementa un **watchdog de comunicación estricto de 1000 ms** en **`carro.c`**. Mientras el usuario mantiene presionado un botón en la app, el teléfono envía el comando **`FORWARD`** repetidamente cada 250 ms. Si se pierde el enlace WiFi o la app se bloquea, el temporizador del microcontrolador expira y, exactamente en el milisegundo 1001, el firmware apaga el PWM de los cuatro motores y restablece el perfil al modo seguro **`AUTO`**. El comando **`PING`** no reinicia el watchdog, asegurando que solo órdenes explícitas mantengan el movimiento.

P4: ¿Por qué separaron la alimentación en powerbank y pilas AA? ¿No era más fácil usar una sola batería grande con un convertidor reductor (buck)?

Respuesta: Los motores de corriente continua generan ruido electromagnético inductivo muy agresivo y caídas de tensión instantáneas de varios voltios al arrancar bajo carga. Si compartieran la misma batería a través de un convertidor reductor, las caídas de tensión bruscas inducirían fácilmente reinicios intempestivos (*brownouts*) en los microcontroladores y corrupciones en el bus I2C. La separación física garantiza que la electrónica reciba corriente continua pura de 5 V regulados, mientras que el portapilas suministra la corriente de fuerza al driver TC1508A, compartiendo únicamente la línea de tierra (GND) como referencia común.

P5: ¿Por qué la cámara transmite a resolución HQVGA (240×176) en lugar de VGA o QVGA?

Respuesta: En la jornada experimental H3 realizamos un barrido metrológico de 100 cuadros por resolución. En QVGA (320×240), el tiempo de compresión por software era de 107 ms y la edad media del cuadro alcanzaba 252 ms, logrando apenas 8.8 fps. Al cambiar a HQVGA (240×176), la cantidad de píxeles se reduce un 45 %, lo que permitió recortar la compresión a 56 ms y la edad del cuadro a 137 ms, alcanzando 12.4 fps fluidos con calidad JPEG 50. Para el detector de obstáculos y la visualización humana en el móvil, 240×176 píxeles brinda suficiente resolución angular sin penalizar la latencia.

P6: ¿Cómo esquiva obstáculos el carro si no construye un mapa completo de la habitación?

Respuesta: El carro utiliza una estrategia reactiva basada en un **corredor de seguridad trapezoidal**. Mediante calibración geométrica de la cámara (altura de montaje e inclinación angular), el núcleo calcula la región del plano de la imagen por la que el vehículo necesariamente transitará. Si la segmentación del piso o la red neuronal detectan un obstáculo dentro de esta franja, el vehículo se detiene, evalúa la distancia disponible a la izquierda y derecha mediante una cuadrícula local, se desvía 40 cm hacia el lado despejado, supera la caja y avanza 30 cm por delante antes de reincorporarse a la trayectoria original.

P7: ¿Qué problema se detectó en la jornada J15 y cómo se resolvió? ¿Fue probado físicamente?

Respuesta: En las simulaciones del escenario B6 (esquivar una caja de 20 cm en una ruta), se observó que el carro llegaba al final con un error lateral promedio de 13.2 cm. El diagnóstico reveló que el estimador de pose proyectaba los avances asumiendo que el vehículo mantenía el rumbo inicial de la ruta, sin considerar que un giro intermedio introduce pequeñas perturbaciones angulares. En J15 se corrigió el algoritmo en `navegador.c` para integrar el rumbo instantáneo medido por el giroscopio durante el avance. En simulación computacional (400 rutas), el error se redujo drásticamente a 1.9 cm. Sin embargo, para ser rigurosos con la verdad científica, **aclaramos que esta corrección no se llegó a probar en el vehículo físico real (permanece como NO MEDIDA)**.

P8: ¿Por qué escribieron el núcleo de navegación en C puro en vez de hacerlo en Kotlin dentro de Android?

Respuesta: Desarrollar los algoritmos en C99 estándar permitió desacoplar totalmente la lógica matemática del sistema operativo móvil. El mismo código fuente compilado en Android vía JNI se compila de manera nativa en computadoras con GCC, lo que nos permitió crear una batería de 21 suites de prueba con 1598 casos unitarios y ejecutar simulaciones Montecarlo de miles de rutas en pocos segundos sin necesidad de encender un emulador de Android ni agotar las baterías del carro.

P9: Si la cámara y el láser ToF detectan cosas distintas, ¿cuál tiene la autoridad?

Respuesta: El **sensor ToF láser VL53L0X tiene la máxima autoridad de parada física**. La cámara puede fallar ante sombras, cambios de luz o retrasos de red, pero el láser emite fotones a la velocidad de la luz y mide distancias reales en milímetros con determinismo temporal. Si el láser detecta un objeto a menos del umbral de seguridad, el microcontrolador ESP32 frena los motores en el acto (`TOF_PARO`), independientemente de lo que la app o la cámara estén procesando.

18. Solución de problemas y mantenimiento

Guía de solución de problemas (Troubleshooting)

Materia: Fundamentos de los Sistemas de Información (FSI)

Propósito: Matriz de diagnóstico rápido de fallas comunes en hardware, red, firmware y aplicación móvil.

Fecha: 27 de octubre de 2026

---

1. Matriz de síntomas y acciones correctivas

Síntoma observado	Causa más probable	Procedimiento de comprobación y solución
La red WiFi del carro no aparece en el teléfono.	La placa ESP32 DevKit no recibe energía o se encuentra en bucle de arranque.	1. Verificar que el LED rojo de la placa DevKit esté encendido. 2. Probar con otro cable micro-USB (algunos cables comerciales solo transmiten carga y no datos o tienen hilos partidos). 3. Confirmar que la powerbank esté encendida y entregue 5 V. 4. Si el LED parpadea erráticamente, conectar el ESP32 a la PC y abrir el monitor serie a 115200 baudios para inspeccionar mensajes de pánico.
La app conecta pero el video de la cámara aparece en negro o no carga.	La placa de la cámara no logró asociarse al WiFi o tiene asignada otra IP.	1. Comprobar que la placa Freenove tenga su LED encendido. 2. Asegurarse de encender el DevKit antes o casi al mismo tiempo que la cámara. 3. Abrir el navegador del teléfono e ingresar a `http://192.168.4.2/stream`. Si abre en el navegador pero no en la app, verificar en `Ajustes` → `Red` que la URL de video apunte exactamente a esa dirección. 4. Si no responde en el navegador, reiniciar la placa de la cámara (el vigilante interno de `recuperacion.c` se reiniciará solo tras 60 s).
Los motores no se mueven aunque la app indique comando activo.	El portapilas de motores está apagado, las pilas están agotadas o falta la tierra común.	1. Verificar que el interruptor físico del portapilas esté en **ON**. 2. Medir con multímetro el voltaje del portapilas (debe ser $\ge 4.8\text{ V}$). Reemplazar las pilas AA si están descargadas. 3. **Verificar el cable de tierra común (GND):** Comprobar con multímetro en modo continuidad que el pin GND del driver TC1508A esté unido al pin GND de la placa ESP32.
El carro gira sobre su propio eje o va hacia atrás al pulsar Adelante.	Polaridad invertida en los cables de los motores.	Los motores TT se cablean por pares en paralelo. Verificar contra `firmware_carro/config.h`: el motor izquierdo avanza con GPIO 26 (`PIN_MOTOR_IZQ_B`) y el derecho con GPIO 25 (`PIN_MOTOR_DER_A`). Intercambiar los cables rojo y negro del canal invertido en la bornera del driver.

El carro se desvía fuertemente hacia un lado al avanzar en línea recta.	Asimetría mecánica en las reductoras o balance descalibrado.	1. Revisar que las ruedas de hule estén bien encajadas en el eje y que ningún cable esté rozando los neumáticos. 2. Acceder a `CalibracionActivity` en la app o consultar el factor con `BAL?`. 3. Si se desvía a la derecha, reducir el factor de balance izquierdo con `BAL_AUTO:NNN` (ej. bajar de 986 a 920). Si se desvía a la izquierda, aumentarlo hacia 1000.
El carro avanza durante 1 segundo exacto y se frena solo.	Disparo del Watchdog de comunicación (1000 ms).	Esto es un comportamiento normal de protección cuando se pierde el flujo de comandos. Ocurre si: (a) el usuario soltó el botón de la app, (b) el teléfono saltó automáticamente a otra red WiFi con internet (desactivar «Asistente WiFi» en Android), o (c) hay alta congestión de radio.
El ángulo del giroscopio cambia rápidamente sin que el carro se mueva.	Sesgo (*bias*) mal calibrado por vibración durante el arranque.	El MPU6050 calibra su sesgo al encender. Si el chasis se movió o vibró durante esos primeros 2.5 segundos, el sesgo queda corrupto. **Solución:** Colocar el carro completamente inmóvil sobre el suelo y pulsar el botón **Calibrar IMU** en la pantalla `SensoresActivity` (comando `IMU_CAL`).
El sensor ToF reporta distancia errática o 0 mm.	Suciedad en el lente óptico o cable I2C flojo.	1. Limpiar suavemente con un paño seco la superficie del sensor VL53L0X. 2. Enviar el comando `CAPS?` desde la app. Si responde `TOF=0`, el microcontrolador no detectó el sensor en el bus (revisar conexiones físicas de SDA en GPIO 21 y SCL en GPIO 22).
La powerbank se apaga sola cada 30 a 60 segundos.	El circuito de bajo consumo de la powerbank interpreta que no hay carga.	La electrónica de control consume poca corriente en reposo. Comprobar que el pin keep-alive (GPIO 2) esté configurado y emitiendo pulsos periódicos de 50 ms cada 2 s (`KEEPALIVE_PERIODO_MS`), o utilizar una batería portátil con modo continuo (*low current mode*).

Mantenimiento preventivo y conservación del carro FSI

Materia: Fundamentos de los Sistemas de Información (FSI)

Propósito: Guía de buenas prácticas mecánicas, eléctricas y metrológicas para prolongar la vida útil del vehículo y preservar la repetibilidad de las pruebas.

Fecha: 27 de octubre de 2026

---

1. Mantenimiento del sistema de alimentación

1.1 Cuidado de las baterías de potencia (Pilas AA)

• **Tipo de celdas recomendadas:** Utilizar pilas alcalinas estándar de 1.5 V (4 en serie = 6.0 V nominales) o acumuladores recargables de NiMH de buena calidad (1.2 V $\times$ 4 = 4.8 V nominales).

• **Prohibición de mezclas:** Nunca mezclar pilas nuevas con usadas ni de marcas o químicas distintas; la celda más débil se sobrecalentará y limitará la corriente del conjunto.

• **Prevención de sulfatación:** Retirar siempre las 4 pilas AA del portapilas al finalizar cada sesión de trabajo. Si el chasis se almacena con las pilas colocadas durante varias semanas, el riesgo de fuga química de electrolito sulfatará las terminales

metálicas y arruinará el contacto eléctrico.

1.2 Cuidado de la batería de control (Powerbank)

• Para almacenamiento prolongado (más de 1 mes), conservar la batería entre el 40 % y el 60 % de carga (2 LEDs encendidos).

• Cargarla al 100 % el día previo a cualquier sesión de laboratorio o exposición formal.

• Desconectar los cables micro-USB de las placas DevKit y Freenove cuando el vehículo no esté en uso.

---

2. Mantenimiento mecánico del chasis y motores

2.1 Motores TT y reductoras plásticas

• Los cuatro motores amarillos incorporan engranajes internos de nylon y poliacetal. **Bajo ninguna circunstancia se deben forzar o hacer girar manualmente las ruedas a gran velocidad cuando el carro está apagado**, ya que el par invertido transmitido desde la rueda hacia el piñón del micromotor puede romper los dientes plásticos de la reductora.

• **Inspección de ejes:** Periódicamente, retirar las cuatro ruedas de hule e inspeccionar los ejes de plástico para retirar pelos, polvo o fibras de alfombra que suelen enrollarse entre el buje del motor y la llanta, provocando fricciones asimétricas.

2.2 Limpieza de neumáticos

• La tracción del chasis depende de la adherencia del compuesto de caucho de las llantas.

• Si las ruedas acumulan polvo, el coeficiente de fricción cae y el vehículo patina en los pivotes angulares.

• Limpiar la banda de rodadura de las cuatro ruedas con un paño ligeramente humedecido en alcohol isopropílico o agua jabonosa antes de calibrar rumbos.

2.3 Apriete de tornillería

• Las vibraciones generadas por la marcha continua tienden a aflojar las tuercas de los separadores de latón que sostienen los dos niveles del chasis y los soportes de la cámara.

• Revisar y apretar suavemente con destornillador de cruz la tornillería antes de cada jornada de pruebas.

---

3. Limpieza de sensores ópticos

1. **Lente de la cámara Freenove (GC0308):** Limpiar suavemente con una toallita de microfibra para lentes fotográficos. Huellas dactilares o grasa sobre el lente reducen el contraste de la imagen y perjudican la inferencia del modelo TensorFlow Lite.
2. **Sensor ToF láser VL53L0X:** El sensor dispone de dos ventanas ópticas diminutas (un diodo emisor láser VCSEL y un arreglo receptor SPAD). El polvo depositado sobre estas ventanas produce reflexiones parásitas internas que provocan lecturas de distancia erróneamente cortas. Limpiar con aire comprimido suave o hisopo de algodón seco.

---

4. Almacenamiento y transporte

• Transportar el carro en una caja rígida acolchada para evitar que los módulos sensores montados sobre espaciadores sufran golpes mecánicos o desalineaciones en su eje de puntería.

• Proteger especialmente el soporte superior del teléfono móvil para no deformar el ángulo de inclinación calibrado de la cámara.

19. Glosario de términos y unidades

Glosario técnico de términos y unidades

Materia: Fundamentos de los Sistemas de Información (FSI)

Propósito: Definición rigurosa y accesible del vocabulario técnico, acrónimos y magnitudes empleadas en la documentación del proyecto.

Fecha: 27 de octubre de 2026

---

1. Conceptos de hardware y electrónica

Término	Definición y contexto en el carro
ESP32	Familia de microcontroladores de 32 bits de doble núcleo con conectividad WiFi y Bluetooth integrados, fabricados por Espressif. Es el procesador central del sistema.
DevKit	Placa de desarrollo comercial que incorpora el microcontrolador ESP32, regulador de voltaje a 3.3 V, interfaz USB-serie y pines accesibles para prototipado.
WROVER	Variante del módulo ESP32 que añade 4 u 8 MB de memoria RAM pseudoestática externa (PSRAM). Es la placa empleada en el nodo de la cámara Freenove.
Driver / Puente H	Circuito integrado de potencia que permite aplicar voltaje a un motor de corriente continua en ambos sentidos de giro mediante interruptores electrónicos de estado sólido. En el carro se utiliza el chip **TC1508A**.
PWM (Pulse Width Modulation)	Modulación por ancho de pulsos. Técnica digital que enciende y apaga rápidamente un pin para regular la velocidad de giro de los motores según el porcentaje de tiempo en alto (ciclo de trabajo).
Bus I2C (Inter-Integrated Circuit)	Bus serie sincrónico de dos hilos: línea de datos (`SDA`, GPIO 21) y línea de reloj (`SCL`, GPIO 22). Permite conectar múltiples sensores compartiendo los mismos cables mediante direcciones hexadecimales únicas.
MPU6050	Módulo sensor que integra un acelerómetro de tres ejes y un giroscopio de tres ejes (GY-521). En este proyecto se utiliza exclusivamente la medición de velocidad angular sobre el eje vertical Z para conocer la orientación del chasis.
VL53L0X	Sensor de distancia láser basado en tecnología de tiempo de vuelo (Time-of-Flight). Mide el tiempo que tarda un pulso de luz infrarroja invisible en rebotar contra un obstáculo frontal.
ToF (Time-of-Flight)	Principio físico de medición de distancia fundamentado en calcular la distancia a partir de la velocidad de la luz y el tiempo transcurrido entre la emisión y recepción del pulso lumínico.
Powerbank	Batería recargable portátil con salida regulada de 5 V mediante puerto USB, empleada exclusivamente para energizar los microcontroladores y sensores del carro.
Tierras comunes (GND)	Interconexión física de las líneas de retorno eléctrico (0 voltios) de la powerbank y del portapilas de motores para establecer una referencia de potencial idéntica en todo el sistema.

Keep-alive (GPIO 2)	Pulso digital de 50 ms emitido cada 2 segundos por el microcontrolador para forzar un consumo transitorio y evitar que los circuitos ahorradores de la powerbank apaguen la alimentación.

2. Redes y protocolos de comunicación

Término	Definición
Punto de Acceso (AP)	Modo de funcionamiento inalámbrico en el cual el ESP32 crea su propia red WiFi independiente (`192.168.4.1`), permitiendo la conexión de clientes sin depender de un router externo.
Estación (STA)	Modo de red en el cual un dispositivo opera como cliente inalámbrico conectado a un punto de acceso. La cámara y el teléfono funcionan como estaciones.
DHCP	Protocolo de configuración dinámica de host. Servicio que asigna automáticamente direcciones IP a los dispositivos que se enlazan a la red del carro (rango configurado: `.10` a `.20`).
WebSocket	Protocolo de red sobre TCP que establece una conexión bidireccional continua y de baja latencia entre el teléfono y el microcontrolador (`/ws`), permitiendo el envío de comandos y telemetría sin renegociar conexiones.
MJPEG (Motion JPEG)	Flujo de video transmitido por HTTP donde cada cuadro es emitido como una fotografía JPEG independiente delimitada por cabeceras multipart.
RGB565	Formato de almacenamiento de color donde cada píxel ocupa 16 bits (2 bytes): 5 bits para el canal rojo, 6 bits para el verde y 5 bits para el azul.
HQVGA	Resolución de imagen de 240×176 píxeles (High Quarter VGA). Resolución estándar de la cámara en este proyecto.
Celdas UDP (`CEL1`)	Paquete binario compacto de 1244 bytes enviado por el puerto UDP 4210 que resume la imagen de la cámara en una matriz de 24×17 bloques de 10×10 píxeles promediando su color RGB.

3. Algoritmos, control y navegación

Término	Definición
Pose	Vector matemático (x, y, θ) que describe la posición bidimensional del centro del vehículo en coordenadas métricas y su orientación angular (rumbo) respecto al sistema de coordenadas de partida.
Rumbo (Heading)	Ángulo de orientación horizontal del vehículo respecto al eje inicial, expresado en grados sexagesimales.
Waypoint	Punto de paso o coordenada objetivo en el plano de navegación trazado en la cuadrícula de la app.
Corredor de seguridad	Franja de paso trapezoidal proyectada en la perspectiva de la cámara que delimita el ancho real del vehículo más un margen lateral preventivo. Un obstáculo solo detiene la marcha si se localiza dentro de este corredor.
ZUPT (Zero Velocity Update)	Algoritmo que actualiza el cálculo del sesgo del giroscopio cuando se comprueba que los motores llevan más de 400 ms apagados y las lecturas no presentan variaciones significativas.

Watchdog	Temporizador de seguridad por software o hardware que supervisa el correcto funcionamiento del sistema. Si no es reiniciado periódicamente por una señal válida, apaga los actuadores o reinicia la placa para evitar estados fuera de control.
JNI (Java Native Interface)	Mecanismo que permite al código escrito en Kotlin/Java en la app Android invocar directamente bibliotecas nativas de alto rendimiento compiladas en C (`nucleo_c`).
NDK (Native Development Kit)	Conjunto de herramientas para compilar código C/C++ en la plataforma Android.

---

4. Unidades de medida empleadas

Símbolo	Magnitud física y equivalencia
ms	Milisegundo (10^{-3} segundos). Unidad estándar de temporización de motores y bucles de control.
μs	Microsegundo (10^{-6} segundos). Unidad de medición de tiempo de ejecución del bucle `loop()`.
mm	Milímetro (10^{-3} metros). Unidad de resolución del sensor ToF láser VL53L0X y del arrastre por inercia (`frenado_mm`).
cm	Centímetro (10^{-2} metros). Unidad de trabajo de la cuadrícula de la app (20 cm por celda).
ms/cm	Milisegundos por centímetro. Inverso de la velocidad lineal del carro; constante de calibración de avance.
ms/grado	Milisegundos por grado sexagesimal. Constante cinemática para calcular la duración de giros en pivote por tiempo.
°/s	Grados por segundo. Velocidad angular reportada por el giroscopio MPU6050.
LSB	Least Significant Bit (Bit menos significativo). Cuentas digitales del convertidor analógico-digital del MPU6050 (sensibilidad: 65.5 LSB por °/s en rango $\pm 500^\circ/\text{s}$).
Hz / kHz / MHz	Hertz, kiloHertz y MegaHertz. Frecuencias del PWM (1500 Hz), bus I2C (400 kHz) y reloj del CPU (160 MHz).
dBm	Decibelio-milivatio. Unidad de potencia de transmisión de radio WiFi (13 dBm configurados en el ESP32).

20. Apéndices técnicos

Apéndice A: Estructura del repositorio y mapa de archivos

El repositorio oficial se hospeda en GitHub (`github.com/fernandosegrr/fsi_proyect`). La rama oficial de entrega documental es `docs/documentacion-completa`.

Apéndice B: Tabla maestra de comandos y respuestas WebSocket

Comando	Parámetros	Respuesta típica	Propósito	
`FORWARD`	Ninguno	—	Avance con rampa y balance activo	
`REVERSE`	Ninguno	—	Retroceso con balance	
`LEFT` / `RIGHT`	Ninguno	—	Pivote en el lugar (un motor activo)	
`STOP` / `EBRAKE`	Ninguno	—	Freno inmediato a los cuatro motores	
`PING`	Ninguno	`PONG`	Sondeo de latencia (no renueva watchdog)	
`MODO_MANUAL`	Ninguno	`ACK_MANUAL`	Potencia 255/252	
`MODO_AUTO`	Ninguno	`ACK_AUTO`	Potencia 210/207	
`GIRAR:<deg>`	5 a 200	`GIRO_OK` / `GIRO_TIMEOUT`	Giro en bucle cerrado con giroscopio	
`BAL_AUTO:NNN`	800 a 1000	`ACK_BAL_AUTO:NNN`	Calibra factor de balance AUTO	
`CAPS?`	Ninguno	`CAPS:IMU=1,TOF=1`	Consulta de periféricos I2C	
`IMU_CAL`	Ninguno	`ACK_IMU_CAL`	Calibra sesgo en reposo	
`TOF_USO:<0	1>`	0 o 1	`ACK_TOF_USO:<n>`	Activa/desactiva autoridad láser

Apéndice C: Asignación de pines físicos (ESP32 DevKit)

- **Motor Izquierdo INA / INB:** GPIO 27 / GPIO 26 (`PIN_MOTOR_IZQ_A`, `PIN_MOTOR_IZQ_B`).
- **Motor Derecho INC / IND:** GPIO 25 / GPIO 33 (`PIN_MOTOR_DER_A`, `PIN_MOTOR_DER_B`).
- **Bus I2C Sensores:** SDA = GPIO 21, SCL = GPIO 22 (400 kHz).
- **Keep-alive Powerbank:** GPIO 2 (pulso de 50 ms cada 2 s).

Apéndice D: Sumas de verificación criptográficas (SHA-256)

- APK Oficial: `36CE38F2ABD11C1030A7DE7B8A23009A865A01365900192E69CED9CDDE9513BA` (`dist/carro_fsi_0.4-074c9fd.apk`, 14,852,386 bytes).
- Firmware Esperado: `9FD1BB5103D1AD66652ADEA68B009D54D30B5C5B6F07F8CA7488A4FBF274048E` (`dist/firmware_carro_453ed20-20261027.bin`, 1,031,712 bytes).

Apéndice E: Observaciones y puntos pendientes

Para la lista de 17 fotografías por tomar, consúltese `docs/lista_de_fotos.md`. Para las aclaraciones de contradicciones históricas y aspectos pendientes de confirmación, consúltese `docs/pendientes_documentacion.md`.